

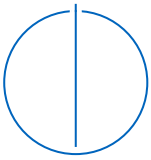
SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Scheduling for Deterministic Virtualization of Virtualized Protection and Control Workloads on COTS Hardware

Leon Simoniants





SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Scheduling for Deterministic Virtualization of Virtualized Protection and Control Workloads on COTS Hardware

Author:	Leon Simoniants
Examiner:	Prof. Pramod Bhatotia
Supervisor:	Peter Okelmann, Dr. Anatole Lefort
Submission Date:	21.09.2026

Abstract

Virtualizing protection and control functions on COTS platforms promises more flexible and modular substation systems, but current environments still depend on specialized hardware because deterministic real-time behavior under virtualization remains challenging. This thesis therefore investigates whether PREEMPT_RT-based KVM/QEMU virtualization can provide sufficiently low and stable latency for representative vPAC workloads, and which system configuration mechanisms are important for achieving such behavior.

A representative protection-processing pipeline is implemented, and, with it, host and guest configurations are evaluated using end-to-end Ethernet measurements on a dedicated COTS testbed. The evaluation considers guest scheduling, CPU isolation mechanisms, network-IRQ priorities, co-location of real-time and fair-scheduled workloads, VM-exit behavior, one-vCPU and two-vCPU guest topologies, long-running execution, and multi-VM consolidation. Across six one-hour measurements, 86.4 million packet exchanges occur without packet loss or exceeding the 5 ms protection target, with the dense end-to-end latency distribution remaining around 40 µs to 46 µs. The implemented protection computation itself requires only approximately 1.19 µs per invocation, indicating most of the measured latency arises outside the protection computation itself. The two designed VM vCPU topologies are compared, and the trade-offs are detailed. Regarding latency, both achieve similar results. Multi-VM measurements demonstrate the viability of the system running multiple concurrent vPAC workloads of up to eight VMs within the tested bounds.

The results demonstrate that low, reproducible latency for vPAC workloads is feasible on the evaluated COTS platform when the configuration is appropriate and deliberate. The evidence provided is empirical; no formal guarantees are given. Source code and configuration files are publicly available [1].

Contents

Abstract **ii**

1 Introduction **1**
1.1 Motivation and Problem Statement 1
1.2 Research Questions 2
1.3 Approach 3
1.4 Contributions 3
1.5 Thesis Structure 4

2 Background **5**
2.1 IEC 61850, Modern Substations, and vPACs 5
2.1.1 Sampled Values (SV) and Frequency 6
2.1.2 Generic Object-Oriented Substation Events (GOOSE) and Latency 6
2.1.3 vIED / vPAC Idea 6
2.2 IED Structure and Protection Processing Pipeline 7
2.2.1 Hot-Path Protection Pipeline 8
2.2.2 Auxiliary Functionality 9
2.3 Real-Time Linux and PREEMPT_RT 10
2.4 KVM/QEMU Virtualization 12
2.4.1 Virtual Machines and the KVM/QEMU Stack 12
2.4.2 vCPUs and Host Threads 13
2.4.3 Virtual I/O and virtio 13
2.4.4 VM Exits and Real-Time Implications 14
2.5 Real-Time Isolation Techniques 14
2.5.1 CPU Affinity and Dedicated Cores 15
2.5.2 Kernel-Level CPU Isolation 16
2.5.3 Interrupt Placement 17
2.5.4 CPU Idle States and Frequency Scaling 17
2.5.5 Separation of Real-Time and Housekeeping Work 18
2.6 Real-Time Latency Evaluation 18
2.6.1 Latency Distributions and Tail Behavior 18
2.6.2 Scheduling-Latency Measurement 18

3 System Design **20**
3.1 Scope and Design Goals . . . 20
3.2 System Architecture . . . 21
3.2.1 Measurement and Actuation Side . . . 21
3.2.2 Virtualized IED Execution . . . 22
3.2.3 End-to-End Measurement and Thesis Scope . . . 23
3.3 VM vIED Processing Architecture . . . 24
3.3.1 Single Protection Hot Path . . . 24
3.3.2 Layered Data Flow . . . 24
3.3.3 Separation of Auxiliary Functionality . . . 24
3.4 IED Deployment Model . . . 25
3.5 VM CPU Topologies . . . 25
3.5.1 One-vCPU Topology . . . 26
3.5.2 Two-vCPU Topology . . . 26
3.5.3 Isolation and Resource Trade-off . . . 27
3.6 Scheduling and Isolation Principles . . . 27
3.6.1 Guest Scheduling Hierarchy . . . 27
3.6.2 Host-Side Isolation . . . 28

4 Implementation **29**
4.1 Host and Guest Configuration . . . 29
4.1.1 Software Platform . . . 29
4.1.2 Reproducible System Provisioning . . . 31
4.1.3 Low-Latency Kernel Configuration . . . 31
4.2 VM Resource and CPU Placement . . . 34
4.2.1 One-vCPU Placement . . . 35
4.2.2 Two-vCPU Placement . . . 35
4.2.3 Multi-VM Placement Changes . . . 36
4.2.4 Additional VM Resource Configuration . . . 36
4.3 Guest Real-Time Scheduling and IRQ Configuration . . . 37
4.3.1 Protection Process Scheduling . . . 37
4.3.2 Low-Latency Network IRQ Configuration . . . 37
4.3.3 Priority Hierarchy . . . 38
4.4 Protection Workload Implementation . . . 39
4.4.1 Protection Hot Path . . . 39
4.4.2 Measurement Processing . . . 39
4.4.3 Protection Functions . . . 40
4.4.4 Hot-Path Boundaries and IEC 61850 Abstraction . . . 40

4.5 Measurement and Microbenchmark Infrastructure 41
4.5.1 End-to-End Latency Measurement 41
4.5.2 Standalone Processing Microbenchmarks 42
4.6 Multi-VM Test Support 43
4.7 Reference System Configurations 43

5 Evaluation **45**
5.1 Experimental Methodology 45
5.1.1 Physical Testbed 45
5.1.2 Workload and Experiment Procedure 46
5.1.3 Reported Metrics 46
5.1.4 Measurement Instrumentation 47
5.2 Scheduling and Isolation Mechanisms 47
5.2.1 Guest PREEMPT_RT under CPU and I/O Contention 47
5.2.2 Network IRQ Scheduling 50
5.2.3 Fair-Task Contention on the RT vCPU 52
5.2.4 Resulting Scheduling Configuration 57
5.3 Processing Costs and Virtualization Effects 57
5.3.1 Representative vPAC Processing Costs 58
5.3.2 VM-Exit Effects 60
5.4 One-vCPU vs. Two-vCPU Topologies 68
5.4.1 Co-location of Real-Time and Fair-Scheduled Work 68
5.4.2 End-to-End Latency under Bounded RT Load 71
5.4.3 Overload, Priority Starvation, and Failure Containment 73
5.4.4 Topology Trade-off 75
5.5 Final End-to-End Latency 76
5.5.1 Long-Run Measurement Method 76
5.5.2 Latency Distribution and Reproducibility 77
5.5.3 Temporal Stability and Rare Transients 78
5.5.4 Long-Run Findings and Implications 81
5.6 Multi-VM Scalability 81
5.6.1 Multi-VM Experiment Configuration 81
5.6.2 Consolidated Multi-VM Results 82
5.6.3 Scalability Findings 84

6 Discussion **88**
6.1 Determinism and Real-Time Guarantees 88
6.2 One-vCPU vs. Two-vCPU Trade-offs 89
6.3 Remaining Tail Latency 90

6.4 Configuration Guidance 91
6.5 Limitations and Threats to Validity 92

7 Related Work **94**
7.1 Virtual IED and vPAC Systems 94
7.2 Real-Time Linux 96
7.3 Real-Time Virtualization 97
7.4 Comparison with This Work 98

8 Conclusion and Future Work **99**
8.1 Summary and Answers to the Research Questions 99
8.2 Future Work 101

Abbreviations **103**

List of Figures **105**

List of Tables **107**

Bibliography **109**

1 Introduction

1.1 Motivation and Problem Statement

Traditionally, protection and control functions in electrical substations have been implemented on dedicated Intelligent Electronic Devices (IEDs). These devices consist of specialized hardware and software to execute protection functions with strict timing requirements. This approach, however, ties the IEDs and their protection functions to vendor-specific physical devices. An alternative deployment model for protection and control can be offered by virtualizing these functions and executing them on commercial off-the-shelf (COTS) compute platforms. Thus, protection from IEDs can be implemented as software workloads and consolidated on shared computing infrastructure.

Such a software-based, virtualized approach to IEDs can offer several advantages. Many virtualized IEDs can share the same physical platform, while virtual machines (VMs) provide isolation between independently deployed workloads. IEDs can be deployed alongside auxiliary functionality, such as monitoring and logging services. Furthermore, computational resources can be allocated through software rather than being fixed by the hardware of individual IEDs. This increases deployment flexibility and decreases reliance on numerous distinct dedicated physical IEDs for protection.

However, by virtualizing and consolidating IEDs onto a general-purpose computing platform, the execution environment changes, but requirements for protection still remain: protection functions are latency-critical. Incoming substation data must be processed, evaluated, and, where required, result in a trip decision within a sufficiently short response time. Therefore, a virtualized platform must continue to adhere to that short response time. The problem widens, as the behavior must remain stable not only during common execution cases, but also in the tail of the latency distribution and under possible interference from other system activity.

General-purpose operating systems and virtualization introduce numerous potential sources of such interference. Within a VM, auxiliary workloads may execute alongside the latency-critical protection path, delaying it. At the virtualization layer, execution of a virtual CPU (vCPU) can be affected by the host in several ways, including through scheduling and virtualization events. Hardware power-management mechanisms and the placement of tasks and interrupts across physical CPUs (pCPUs) further influence

the response times experienced. As a result, achieving predictable latency depends on the overall system configuration and is not solely a property of the protection functions themselves.

Linux with the PREEMPT_RT real-time extensions provides mechanisms to reduce latency and assign explicit real-time priorities to latency-critical workloads. Combined with KVM/QEMU virtualization and several kernel features, a Linux system therefore serves a potential basis for running virtualized protection and control (vPAC) workloads on COTS hardware. Nonetheless, the sole availability and potential do not establish that a particular configuration yields sufficiently predictable behavior. Therefore, it is essential to experimentally evaluate different system configurations, their trade-offs, and remaining overheads.

Thus, this thesis aims to investigate whether PREEMPT_RT-based KVM guests can provide sufficiently low and stable latency for representative vPAC workloads on COTS hardware and which system configurations are suitable for doing so. The investigation considers both application-level end-to-end latency and several individual sources of interference. In particular, it studies real-time scheduling and isolation mechanisms, representative vPAC protection-processing costs, virtualization events, and alternative vCPU configurations. Finally, the resulting system is evaluated under representative workloads and, where applicable, multiple concurrently deployed virtual IEDs. Note that the objective is not to establish a formal worst-case end-to-end latency bound. Rather, the objective is to experimentally characterize the achievable real-time behavior, identify relevant sources of latency, and derive suitable configuration recommendations for vPAC workloads.

1.2 Research Questions

Based on the motivation and problem described above, this thesis investigates the real-time behavior of vPAC workloads from multiple complementary perspectives. The objectives considered are captured by the following research questions:

RQ1 Can PREEMPT_RT-based KVM guests provide sufficiently low and stable end-to-end latency for representative vPAC workloads?

RQ2 Which scheduling and isolation mechanisms are important for maintaining low and stable latency under load?

RQ3 What latency is contributed by representative protection processing, and how do virtualization events such as VM exits affect guest timing?

RQ4 What are the trade-offs between one-vCPU and two-vCPU VM configurations?

RQ5 Do the evaluated configurations remain viable when multiple IED VMs are deployed concurrently?

The research questions guide the experimental evaluation presented later in Chapter 5. Each question is addressed through a corresponding set of experiments, while their combined results are used to derive the final system-level conclusions.

1.3 Approach

To investigate our research questions, we use an experimental approach. We start with a KVM/QEMU-based vPAC workload and iteratively analyze and reconfigure the execution environment, considering both host- and guest-side mechanisms. The goal is to reduce latency and interference sources. Mechanisms we consider include PREEMPT_RT scheduling, CPU and interrupt isolation, vCPU allocation, and execution delays because of virtualization.

The evaluation proceeds at multiple levels. At first, we evaluate scheduling and isolation mechanisms to determine which system configurations are required for stable real-time execution. Then we compare one-vCPU and two-vCPU VM topologies to understand the trade-offs between them. Simultaneously, we research, implement, and benchmark representative vPAC processing to quantify the latency contribution of protection computations. Finally, by analyzing guest-visible scheduling latency alongside VM-exit activity across different workloads, we investigate effects specific to virtualization.

In the end, we benchmark the resulting configurations by measuring end-to-end latency at the application level. We conduct longer-running measurements to analyze tail behavior and identify observed maximum latencies. To assess the feasibility of the evaluated configurations in terms of their latency characteristics in a multi-IED setup, we test them under multiple concurrently active protection workloads.

1.4 Contributions

The main contributions of this thesis are:

- An experimental characterization of end-to-end latency, tail behavior, and reproducibility for representative PREEMPT_RT-based vPAC execution in KVM/QEMU VMs.

- A mechanism-oriented evaluation of scheduling and virtualization effects relevant to low-latency vPAC execution, spanning guest kernel behavior, network-interrupt request (IRQ) scheduling, co-location of real-time and fair-scheduled workloads, fair-server behavior, and different VM-exit paths.
- Design and comparison of one-vCPU and two-vCPU VM configurations for vPAC workloads, including their trade-offs in resource efficiency, workload separation, and overload behavior, as well as an experimental evaluation of their consolidation behavior under multiple concurrently deployed IED VMs.
- Description, implementation, and benchmarking of a representative vPAC processing pipeline derived from open-source implementations, consisting of measurement preprocessing and several ANSI protection functions.
- Practical system configuration recommendations derived from the experiment results and their identified limitations.

1.5 Thesis Structure

The rest of the thesis is structured as follows: Chapter 2 provides the background information necessary to understand the work implemented and evaluated in this thesis. Chapter 3 provides a high-level overview of our system, detailing the design principles that guide it. Chapter 4 dives deeper into the concrete implementations of the described system and accompanying components. Chapter 5 subsequently evaluates the implemented system based on the previously mentioned research questions. Afterward, Chapter 6 discusses the evaluation results. And finally, Chapter 7 details related work, and Chapter 8 concludes this thesis.

2 Background

2.1 IEC 61850, Modern Substations, and vPACs

Modern digital electrical substations follow the international standard IEC 61850. This standard was designed to meet the unique requirements of substation automation. It defines an Ethernet-based communication system and universal data models, which are key to interoperable and interchangeable equipment in substations [2].

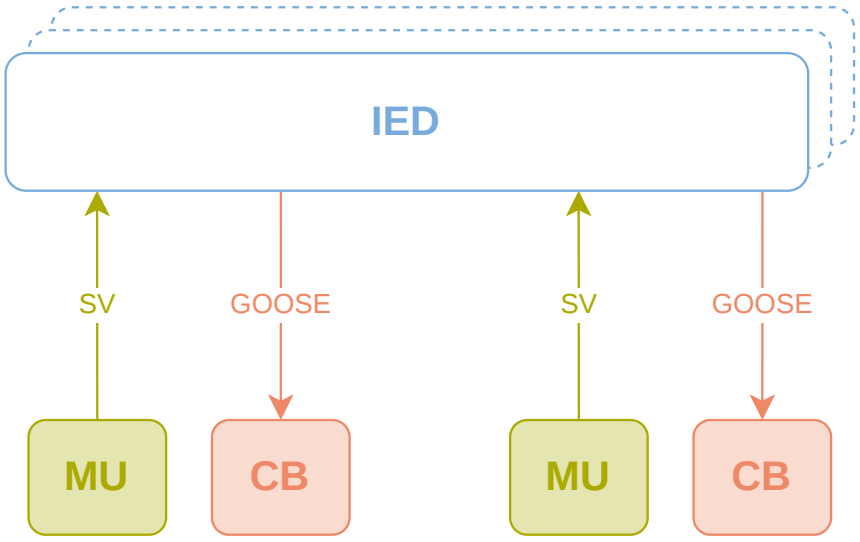


Figure 2.1: Simplified IEC 61850-based substation architecture and the communication path relevant to this thesis.

To understand the architecture important for this thesis, consider the simplified substation system depicted in Figure 2.1. There are three levels of communication defined in IEC 61850, of which two are key for our work, and we look a little closer at these:

The **process level**: Contains sensors, merging units (MUs), and actuators. The sensors

measure analog current and voltage samples and send these to the MUs. The MUs convert these analog samples into digital format, create Sampled Values (SV) packets containing them, and send them over Ethernet to the IEDs. With this data, the IEDs can perform calculations to detect faults and, if one is detected, send a trip signal. The actuators are circuit breakers (CBs) that receive this signal and consequently isolate the fault [2, 3].

One layer higher is the **bay level**: This level contains the aforementioned IEDs, whose purpose is to receive data from the process level and determine faults.

The connection between MUs, CBs, and IEDs is defined by two IEC 61850 network protocols: SV for connecting MUs to IEDs and Generic Object-Oriented Substation Events (GOOSE) for connecting IEDs to CBs.

2.1.1 Sampled Values (SV) and Frequency

SV data transmission occurs at a constant bandwidth. We consider the IEC 61850-9-2LE suggested sampling rate of 80 samples/cycle at 50 cycles/second (50Hz system frequency) [2]. This corresponds to a sampling rate of 4000 samples per second, i.e., MUs send out an SV packet every 250 µs.

2.1.2 Generic Object-Oriented Substation Events (GOOSE) and Latency

The trip signal is communicated via a GOOSE message and requires latency between 4 ms and 1 s [2]. Going forward, our latency requirement, starting from the registration of a faulty state (i.e., an SV packet sent by a MU) until a trip signal arrives at a CB, will be defined as 5 ms.

2.1.3 vIED / vPAC Idea

Currently, IEDs adhere to the IEC 61850 standard but remain expensive, vendor-specific hardware devices. The virtualization of these substation functions shifts the hardware and logic to software-based workloads that execute on COTS hardware (cf. Chapter 7). Therefore, the protection functions are now realized as a vPAC workload inside one or more VMs. This invites potential benefits such as more flexible deployment and maintenance, and the consolidation of IEDs onto fewer common platforms. Consequently, this changes the execution environment of IEDs: The virtualized IEDs now depend on operating-system mechanisms such as interrupt handling and scheduling. In contrast to the classical subsystem architecture (Figure 2.1), an abstracted example vPAC architecture is shown in Figure 2.2.

This thesis focuses on the computational and scheduling behavior of this vPAC architecture. We are not trying to completely re-engineer a production-ready IED, but

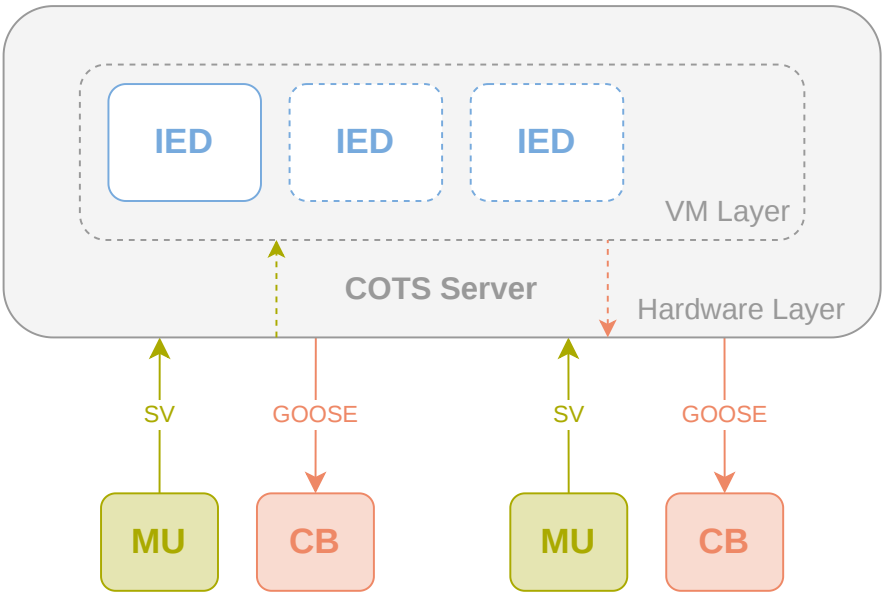


Figure 2.2: Example vPAC substation architecture.

rather study whether the latency-critical protection path from SV generation to the trip signal can be realized within our constraints in a COTS KVM/QEMU PREEMPT_RT environment. The concrete IED internal protection-processing pipeline examined is described in the following section.

2.2 IED Structure and Protection Processing Pipeline

To understand how IED processing itself influences our trip decision latency, it is important to understand how an IED is typically structured, what it contains, what properties it exhibits, what computations must be performed, and how long they take. To achieve this goal, we have researched multiple open-source reference implementations of vPACs and virtualized IEDs and synthesized our findings here [4–9]. Therefore, this section is concerned with understanding a reference IED structure.

After the measurement data has been delivered to an IED inside of an SV packet, the IED is tasked with determining whether a trip signal needs to be emitted. By doing so, the IED fulfills its protection task. Although the internal organization and structure of protection devices and production-ready IEDs may differ across implementations, the open-source reference implementations investigated in this thesis exhibit a common

processing structure. We define this processing path as the *Hot-Path*; i.e., the processing and computations necessary from SV packet arrival up to trip decision and possible trip signal emission from the IED. An IED may also contain auxiliary functionality such as logging and monitoring, but this functionality is not important for the latency-critical hot-path calculations and is therefore treated separately. In the following section, we will elaborate on the different stages in the hot-path, with the general structure of an IED being illustrated in Figure 2.3.

2.2.1 Hot-Path Protection Pipeline

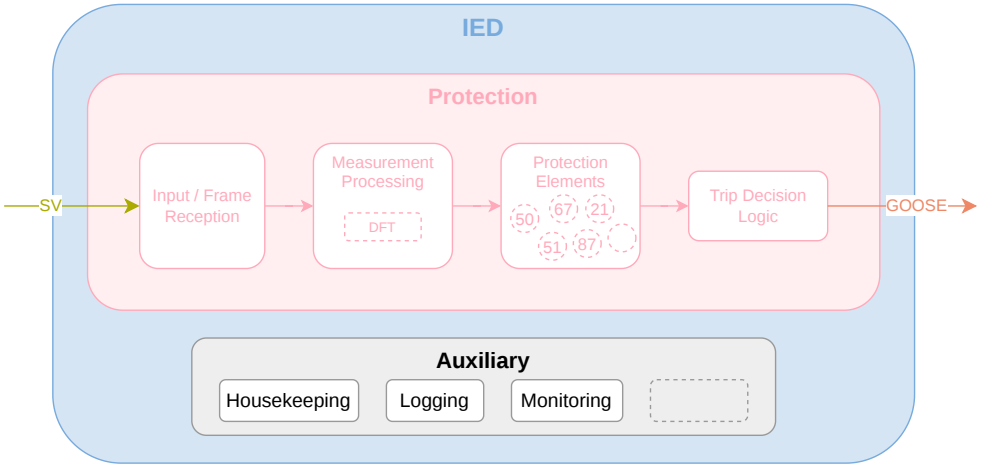


Figure 2.3: Protection processing path inside an IED. The solid path represents the latency-critical protection path (hot-path), while auxiliary functionality executes outside the hot-path.

At a high level, the protection pipeline consists of four stages: input reception, measurement preprocessing, execution of protection elements, and a trip decision. In the following, the stages are described in more detail.

Input Reception

This stage is concerned with receiving the measurement data. SV packets from the merging units are accepted, and they carry digitized current and voltage samples. These samples are typically not consumed directly by the following protection algorithms, but are first handed off to a measurement preprocessing layer.

Measurement Preprocessing

The measurement preprocessing stage derives the quantities required by the protection elements from the digitized current and voltage data in the SV packets. Depending on the specific protection element, these derived values may include current and voltage magnitudes, phase angles, or complex phasors. A common approach in the investigated implementations is to maintain and update a sample window and perform a Discrete Fourier Transform (DFT) to estimate the phasors. For our system, using 80 samples/cycle (see Section 2.1), our cycle sample window consists of 80 samples. After computing the required quantities, they can be provided to the protection stage and the corresponding protection elements.

Protection Stage

The protection stage consists of individual protection elements that use configured protection settings and derived quantities from the measurement layer to compute trip decision indicators. Based on the investigated implementations, we consider the five representative protection elements described in Table 2.1, with ANSI 51 later implemented in DTL and IDMT variants, resulting in six benchmarked functions.

The exact internals of each function are not of significant importance; what is important to note, though, is their similar characteristics. From a software perspective, the different functions are not very distinct. All functions operate on a small, fixed number of quantities; all elements operate on their own limited, bounded state, such as accumulated timers [10]; and result in only a small set of outputs, such as, e.g., trip signals [4–9]. Essentially, the ANSI protection functions themselves are relatively small, bounded in-memory computations that, for decisions, do not require any other services from the operating system or memory management.

Trip Decision Stage

The outputs of the executed ANSI protection elements in the previous stage are subsequently combined by specific decision logic, including, e.g., combinations of functions and blocking conditions. In a complete IEC 61850 implementation, the resulting decision state can be mapped to a trip output such as a GOOSE trip message.

2.2.2 Auxiliary Functionality

As described and shown in Figure 2.3, an IED not only contains hot-path processing but can also include auxiliary mechanisms, such as logging or configuration interfaces. Furthermore, if an IED lives inside a VM, kernel housekeeping is also an important

ANSI	Protection function	Basic processing principle
50	Instantaneous overcurrent	Immediately trips when the current magnitude exceeds a configured pickup threshold.
51	Time overcurrent	Similar to ANSI 50 overcurrent pickup, but with a time characteristic; trips after a configured time delay. Both a fixed definite-time delay and an inverse-time function are considered.
67	Directional overcurrent	Restricts the overcurrent tripping decision to a specific fault direction using directional information.
21	Distance protection	Estimates the apparent impedance from the relay location to the fault point. Trips if the resulting impedance lies within a configured zone.
87	Differential protection	Compares current measurements from different ends of the protected equipment and essentially trips if they differ more than a configured threshold.

Table 2.1: Representative protection functions considered in this thesis.

factor. As we have seen, the complete hot-path computations necessary for trip decisions can generally be separated into bounded computations utilizing preallocated data structures in memory. The computation kernels do not rely on any other kernel services, such as file access or network communication. Since these auxiliary functionalities can introduce significant variability in execution times and latency, they should be kept separate from the latency-critical hot-path.

2.3 Real-Time Linux and PREEMPT_RT

General-purpose Linux prioritizes overall throughput, responsiveness, and fairness. The problem is that, with our latency-critical protection workload, our primary concern is low latency: a high-priority task (i.e., our protection function) becomes runnable and executes with a small, and in the best case, predictable delay. This latency is called *scheduling latency*; it is the difference in time between the time when a task becomes runnable and the time when it is actually scheduled to run.

In general-purpose Linux, numerous sources contribute to unpredictable latency,

most notably kernel activity, interrupt handling, sections of the kernel where preemption is disabled, and the execution of other processes [11–13].

PREEMPT_RT is a patch for the Linux kernel, which was mainlined in 2024 with kernel version v6.12 [14], that makes the kernel nearly fully preemptible. This goal is achieved by drastically reducing the amount of non-preemptible kernel code and sections, making IRQ handlers threaded, and modifying several other kernel synchronization mechanisms so they can also be scheduled normally. As a result, a high-priority real-time task can preempt a lower-priority task in situations where a conventional kernel might have had to wait, delaying execution [11–13].

Shortly mentioned above is IRQ handler threading. In a regular Linux kernel, the majority of interrupt handling occurs in interrupt context and can therefore not be scheduled like normal processes. In PREEMPT_RT Linux IRQs become threaded, meaning the IRQ handler kernel threads can be assigned CPU affinities and scheduling priorities, just like regular threads. Some low-level interrupt handling necessarily remains outside of scheduling and preemption (notably the top halves of IRQs), but the majority of IRQ processing can be controlled by the scheduler. By default, threaded IRQs receive the real-time scheduling policy `SCHED_FIFO` with a real-time priority of 50 [12]; more on this in the following paragraphs.

This IRQ handler scheduling priority assignment will become critical for our workload, since it is packet-driven. For our workload, we need to receive packets from outside, meaning that simply assigning the protection application a high priority is insufficient, since the IRQ thread responsible for delivering the packets might not execute, delaying our hot-path execution. Thus, a relative-priority design for the application and its associated threads and processes is relevant.

For the latency-critical application threads in our research, the `SCHED_FIFO` real-time scheduling policy is used. Linux assigns RT-tasks fixed real-time priorities, ranging from 1-99. Any thread with an RT-priority outranks any other tasks with default Linux priorities (`SCHED_OTHER`) or `nice` values. Any RT-thread with a higher priority preempts another RT-thread with a lower priority. `SCHED_FIFO` does not periodically stop a running task after a predefined time slice, in contrast to a time-sharing scheduling policy. A task may continue executing until it blocks, yields, or is preempted by a higher-priority thread.

One important caveat is introduced with this scheduling behavior, though. Since a high-priority task either voluntarily stops or is preempted by a higher-priority task, a faulty or CPU-heavy `SCHED_FIFO` thread can consume an entire CPU and prevent any other tasks from running. PREEMPT_RT foresaw this and provides mechanisms for containing such starvation:

Traditionally, two system parameters were relevant: `sched_rt_period_us` and `sched_rt_runtime_us`. The former defined a period in µs, whereas the latter specified how

much CPU time within that period may be consumed by RT tasks in µs. Note that this included all RT-tasks, not one specifically. Consequently, if the real-time runtime in a period had been exhausted, RT tasks could be temporarily throttled until the next period. A `sched_rt_runtime_us` value of `-1` represented an unlimited runtime, essentially disabling this throttling mechanism [15, 16]. This mechanism remains relevant when `CONFIG_RT_GROUP_SCHED` is enabled.

Recent kernels behave differently when `CONFIG_RT_GROUP_SCHED` is disabled, as in the guest kernel evaluated in this thesis. In this configuration, the parameters primarily remain relevant to `SCHED_DEADLINE` task admission control. Instead, recent kernels use a per-CPU *fair server*, implemented using deadline-server scheduling, to provide execution runtime to fair-scheduled tasks [15, 17].

The fair server is only relevant when fair-scheduled runnable work actually requires CPU time, otherwise real-time tasks are permitted continued execution. The fair-server similarly exposes per-CPU period and runtime parameters through `/sys/kernel/debug/sched/fair_server/cpuX/`. The default configuration reserves 50 ms within a 1 s period for fair-scheduled execution.

In general, mechanisms that may interrupt or throttle RT tasks typically protect the system from rogue threads, but can be very undesirable and hinder a constant low-latency workload on dedicated CPUs.

The mechanisms described above will be important when configuring our system, as they make the kernel largely preemptible and IRQ activity schedulable. This prevents our hot-path and related threads from incurring significant and unpredictable latency penalties. The interaction of these mechanisms forms the basis for the guest scheduling configurations evaluated in later chapters.

2.4 KVM/QEMU Virtualization

2.4.1 Virtual Machines and the KVM/QEMU Stack

Typical applications on a system run *bare-metal*. Whilst this is great for low-latency applications and system reasoning, this also has drawbacks: namely, less isolation or deployment flexibility. A VM provides a software-defined execution environment that emulates a physical system for the operating system. The guest operating system and applications running inside the VM operate on virtualized CPUs, memory, and devices. This allows for multiple VMs to execute independently on one single physical host, whilst retaining isolation.

On Linux, the Kernel-based Virtual Machine (KVM)[1] provides a kernel module

[1] `https://linux-kvm.org/`

for hardware-assisted virtualization. It allows guest code to mostly execute directly on the pCPU without the need for slow software emulation or interpretation. Some instructions still persist that require intervention from the virtualization layer; these cause the control to return from the guest to the host.

By itself, KVM does not provide a complete virtual computer necessary for VMs. Device models, storage, and networking remain necessary and are provided by QEMU[2]. With KVM hardware acceleration, QEMU serves as a user-space VM monitor, whilst KVM accelerates CPU virtualization. Therefore, QEMU configures and emulates the virtualized hardware environment for the VMs and repeatedly enters KVM to execute guest code.

2.4.2 vCPUs and Host Threads

A guest operating system observes one or more vCPUs as its processor cores. From the guest's perspective, these behave as actual pCPUs. However, on the host, each vCPU is represented and executed by a dedicated *vCPU thread* belonging to QEMU. This means the vCPUs are regular schedulable threads from the host's perspective and can be assigned an affinity, scheduling policy, and scheduling priority. Whenever such a vCPU thread is scheduled to run, it enters KVM and executes the guest vCPU's instructions on the scheduled host pCPU. This also means, though, that the host scheduler may deschedule a vCPU, thereby stalling and delaying its progress. Thus, careful consideration of mechanisms to prevent this behavior is warranted.

In addition to vCPU threads and general processing, the VM might also need other I/O and services, such as e.g., networking and storage. For these tasks, QEMU contains other threads responsible for them. In particular, QEMU emulator threads and I/O threads can execute tasks such as device emulation. Therefore, for low-latency VM workloads, their placement and scheduling on pCPUs might be of interest, since VM guest work may include networking or kernel housekeeping, which requires emulation.

2.4.3 Virtual I/O and virtio

Since a VM requires virtualized devices to communicate with the host system, fully emulating them may introduce significant overhead due to repeated virtualization-layer intervention. Paravirtualization is a concept that aims to address legacy hardware emulation issues, e.g., when writing to storage disks, by making the guest system aware that it is running inside a virtualized environment. The guest is modified with special drivers that enable more efficient, streamlined operations and communication between the host and guest systems.

[2] `https://www.qemu.org/`

The virtio framework[3] provides such paravirtualized devices for common functionality, such as networking and storage. Depending on specific device configurations, processing may occur in QEMU userspace or be accelerated via kernel mechanisms such as `vhost`.

For reasoning about the network path relevant to our work, our virtual network interface involves interactions among the guest network driver, virtual device queues, host-side processing, and the actual host physical network interface.

2.4.4 VM Exits and Real-Time Implications

As previously mentioned, VM execution sometimes needs to exit to the host to perform certain functions, such as device or instruction emulation. Such a transition is typically called a *VM exit*. After the event has been handled, the virtualization layer can resume the execution of the guest VM. The resume is analogously called *VM entry*. VM exits may be caused by various reasons, such as the execution of privileged instructions.

Since these VM exits must leave guest execution, they may cause delays visible in our guest system, and not every VM exit has the same cost. Some exits can be handled in the host kernel, and VM execution can resume thereafter without the need to schedule the QEMU userspace process. On the other hand, events can occur that require KVM to return control to the QEMU userspace threads, so that, e.g., devices can be properly emulated. In general, userspace QEMU exits are undesirable due to the higher delays incurred within the VMs [18].

This distinction is relevant for our real-time workload, since any exit may temporarily interrupt our protection workload. If the event can be handled quickly by KVM, the resulting interruption may remain small, whilst if QEMU threads must execute, the incurred delay also depends on host mechanisms, such as scheduling.

2.5 Real-Time Isolation Techniques

Real-time latency-critical workloads use real-time scheduling mechanisms such as policies and priorities, but these alone do not prevent unrelated activity from sharing CPU processing time. Numerous events can still delay our latency-critical task, e.g., kernel activity, interrupts, housekeeping tasks, or processor power-saving states. Typically, real-time systems combine scheduling and isolation mechanisms to reserve selected CPUs for low-latency execution. Unrelated activity is directed toward so-called *housekeeping* CPUs.

[3] `https://docs.oasis-open.org/virtio/virtio/v1.2/virtio-v1.2.html`

Linux provides several mechanisms that aid in our goal. Some of which operate at boot time and reduce activity on selected CPUs, whereas others can control the placement of processes on specific CPUs. In Table 2.2, we summarize particularly relevant mechanisms to system configurations considered in our work.

Mechanism	**Example**	**Purpose**
CPU affinity	CPU mask	Restricts a process or thread to selected CPUs.
`isolcpus`	`isolcpus=<cpus>`	Removes selected CPUs from normal scheduler load balancing.
`nohz_full`	`nohz_full=<cpus>`	Reduces periodic scheduler-tick activity on selected CPUs.
`rcu_nocbs`	`rcu_nocbs=<cpus>`	Offloads RCU callback processing from selected CPUs.
`kthread_cpus`	`kthread_cpus=<cpus>`	Restricts eligible kernel threads to a designated CPU set (housekeeping).
IRQ affinity	CPU mask / `irqaffinity`	Controls on which CPUs hardware interrupt processing is allowed.
CPU idle policy	`idle=poll`	Avoids deep idle transitions by actively polling while a CPU would otherwise be idle.
Frequency policy	performance governor	Reduces latency and execution-time variation caused by dynamic CPU frequency changes.

Table 2.2: Selected Linux mechanisms relevant to isolating latency-critical execution.

In the following subsections, we briefly introduce the mechanisms required to understand the configurations later in our work. Not all of these mechanisms are required in every real-time configuration, but they provide various controls to address different potential sources of latency.

2.5.1 CPU Affinity and Dedicated Cores

In general, the Linux scheduler is free to migrate a runnable task between compatible CPU cores. For a latency-sensitive task, this makes it more difficult to control whether and which other tasks share its execution resources, adds overhead from context

switches, and can introduce latency from core-specific caches that do not contain application-relevant data, resulting in cache misses.

CPU affinity allows restricting processes and threads to specific sets of CPUs, preventing the scheduler from moving them to other CPUs. CPU affinity by itself does not make the selected CPUs exclusive to that process or thread; kernel threads and interrupts may still execute on those CPUs, potentially causing the task to stop executing. The restriction of a process to specific CPU cores is referred to as *pinning a process*.

This mechanism allows assigning CPU cores specific roles, which are used later in our work on both the host and the guest. One example is directing housekeeping or auxiliary work to one set of CPUs, whilst pinning low-latency work on dedicated specific CPUs.

2.5.2 Kernel-Level CPU Isolation

Whilst CPU affinities allow pinning certain threads to specific CPUs, their execution can still be interrupted by background activity originating from the operating system itself. To isolate a CPU core from disturbances, the Linux kernel provides several boot-time mechanisms [19].

`isolcpus`. The `isolcpus` kernel parameter can be used to exclude a set of CPUs from regular Linux scheduling. This means that the Linux scheduler no longer places tasks on these CPUs automatically, and subsequent manual pinning can be used to place specific tasks on them.

But importantly, this mechanism does not prevent all kernel execution on such isolated CPUs; it only prevents interference from regular scheduling.

`nohz_full`. The `nohz_full` kernel parameter allows for reducing periodic tick activity on selected CPUs. Linux typically uses periodic scheduler ticks for accounting and scheduling. It is also important to note that the periodic scheduler tick is removed only if two conditions are met: a single task is runnable on that CPU, and the time stamp counter (TSC) is stable [20, 21]. The TSC requirement is typically not a problem on modern systems. However, pinning or having more than one task running on a `nohz_full` CPU will enable the periodic tick again.

This mechanism is useful if a CPU is intended to run a single long-running task, since otherwise the periodic kernel interruptions add small but unnecessary latency.

`rcu_nocbs`. For synchronization in many kernel subsystems, Linux utilizes Read-Copy-Update (RCU). Completion of RCU operations can require callbacks, possibly on

an intended isolated CPU if it participated in the workload. The `rcu_nocbs` parameter offloads these callbacks from the selected set of CPUs to dedicated kernel threads.

For isolated CPUs, this parameter prevents RCU callback work from competing with our isolated and pinned processes. The callback threads instead execute on housekeeping CPUs.

`kthread_cpus`**.** The kernel creates various worker threads that may execute on the intended isolated CPUs. The kernel parameter `kthread_cpus` allows restricting these kernel threads to a specific set of CPUs, typically the housekeeping CPU set.

2.5.3 Interrupt Placement

Even when CPUs have been successfully isolated for specific workloads, hardware devices can generate IRQs independently, in which case IRQ processing may compete with our isolated workload on that CPU.

For this case, Linux allows control of interrupt affinities. Essentially, this is similar to controlling CPU affinities, but for IRQ processing. Commonly, configurations route unrelated interrupts to housekeeping CPUs and, at most, permit only relevant interrupt processing on the corresponding isolated CPU cores.

2.5.4 CPU Idle States and Frequency Scaling

In favor of power conservation, modern processors dynamically adjust their operating frequency and can enter idle states, commonly referred to as C-states. These various states introduce measurable increases in latency and latency variability, which are unfavorable for our latency-critical protection workload; therefore, disabling them is reasonable [22–24].

One mechanism is to restrict or avoid processor idle states. The kernel parameter `idle=poll` keeps a CPU polling instead of entering the normal hardware idle states, avoiding the corresponding idle-state exit latency at the cost of substantially increased power consumption [19, 22, 23].

Another mechanism is regulating dynamic frequency scaling. Configuring performance-oriented frequency policies can reduce variability in observed latency caused by dynamic frequency scaling by requesting the highest frequency permitted by the policy [24].

Both of these options give rise to a latency–energy tradeoff. At the cost of higher power consumption, the system achieves lower, less variable latency. In general, this trade-off must not be taken in all cases, and decisions can be workload- and goal-dependent.

2.5.5 Separation of Real-Time and Housekeeping Work

The above-described mechanisms share the objective of reducing the number of unrelated activities that can interrupt and delay specific important processes.

Commonly, the idea is to further separate latency-critical real-time workloads from general housekeeping activities by assigning them to distinct CPU pools (cf. Chapter 7). This principle can reduce the number of execution contexts to consider and yield a more clearly defined structural decomposition of a system. Housekeeping typically refers to tasks that are not directly related to mission-critical work, such as logging or kernel housekeeping.

2.6 Real-Time Latency Evaluation

For real-time workloads and systems, evaluating average latency is important but not sufficient on its own. Average latency evaluation is important for understanding expected system functionality, but rare, substantially larger delays are also significant. For protection workloads, exceeding the chosen application-level protection target can be more critical than small variations in common-case latency. Therefore, the evaluation of real-time behavior has to consider the complete latency distribution. Special attention should be given to upper percentiles and maximum observed latencies.

2.6.1 Latency Distributions and Tail Behavior

Throughout our work, latency distributions are characterized using statistics such as the median and upper percentiles. Comparatively rare, high-latency observations are referred to as the *tail latency*.

Jitter is another related term, which in general describes the variation in latency between repeated executions. For real-time workloads, a system with lower jitter but slightly higher average latency might be better suited than one with lower latency but higher latency variance.

This distinction is important for our work because, for most trip decisions or scheduling, latencies may fall within a narrow range, but rare outliers can produce significant delays. Following this logic, both the magnitude and frequency of these outliers are relevant to assessing a system's or configurations' compatibility with the requirements.

2.6.2 Scheduling-Latency Measurement

The scheduling latency introduced in Section 2.3 is measured independently from the trip latency of our system measurements. A commonly used tool in literature for

measuring thread wake-up latency in PREEMPT_RT is `cyclictest`[4] [11]. It repeatedly sleeps on a thread for a specific duration and measures the difference between the actual wake-up time and the expected wake-up time as the scheduling or wake-up latency.

These measurements are useful for isolating and decreasing operating-system and virtualization latency overhead from the total trip decision latency.

[4] `https://man.archlinux.org/man/cyclictest.8.en`

3 System Design

3.1 Scope and Design Goals

In our thesis work, we focus on the system environment and on the mechanisms that influence the real-time behavior of our vPAC workload. As described in Subsection 2.1.2, our trip decision latency bound is set at 5 ms. Therefore, the primary concern is the latency between the protection-relevant input becoming available and the circuit breaker receiving a corresponding trip signal. We primarily investigate guest scheduling, vCPU placement, interrupt handling, virtualization effects, and execution characteristics of the protection hot-path itself.

Our trip decision latency measurements include the latency and delays introduced by the host-side networking, as our vPAC host system will receive SV packets and send out corresponding GOOSE trip messages. Investigation of the host-side networking paths and the data distribution between the host and the virtualized IEDs inside VMs is outside of the primary scope of this thesis and is addressed by companion work [25]. Our work assumes that the required data can be delivered to the target IEDs and investigates the other aforementioned aspects. However, the networking path still remains part of the measured system and therefore still contributes to the observed latency.

The system design is guided by the following goals:

- **Use of established commodity infrastructure.** The design should rely on COTS x86 hardware and widely available Linux virtualization mechanisms. In particular, the system is based on KVM, QEMU, and libvirt rather than specialized real-time hypervisors or dedicated protection hardware.
- **Low end-to-end latency.** The latency observed should fall well within our target bound of 5 ms.
- **Stable tail behavior.** The system should not only achieve a suitable average latency but also limit the tail latency caused by the evaluated mechanisms in both frequency and magnitude.
- **Isolation of the protection hot path.** Latency-critical protection processing should be protected from unrelated system activity as far as practical. Auxiliary

functionality should not be executed synchronously within the protection hot-path.

- **Reproducible system configuration.** Evaluated mechanisms and parameters should be explicitly configured rather than left to the dynamic scheduler. This allows the execution environment to be reproduced and compared.
- **Support for multiple independently deployable IEDs.** The architecture should allow several vPAC workloads to coexist on the same physical server while preserving clear isolation.

The architectural choices presented in the following sections are derived from these design goals. The goals aim to provide a representative, controlled execution environment in which we can systematically and repeatedly evaluate system configurations using standard, readily available Linux and virtualization mechanisms.

3.2 System Architecture

As described, our goal is to measure the total trip decision latency as the difference between the time protection-related information becomes available (i.e., a merging unit (MU) sends an SV packet) and the time a trip decision is received in an actuator (i.e., a circuit breaker (CB) receives GOOSE trip signal) (cf. Section 2.1). Therefore, the system considered in this thesis consists of two physical hosts connected through an Ethernet link, simulating an IEC 61850 substation design. One host represents the measurement and actuation devices (MU as the data sender and CB as the trip data receiver). The second host represents the COTS virtualization host that executes vPAC workloads. Figure 3.1 depicts the resulting high-level system architecture.

3.2.1 Measurement and Actuation Side

The first physical host acts as both an MU and a CB. A *sender* periodically transmits packets with protection-relevant data to the vPAC host, which represents the MU. On the return path, a *receiver* listens for trip decision signals, which represents the CB. Conceptually, this corresponds to the substation processing path introduced in Chapter 2.

The physical host is not intended to reproduce every component of a complete substation; instead, the sender and receiver interfaces provide a controlled means to simulate protection data and responses. Further details, such as timestamping mechanism, sender/receiver implementations, and packet representation, are described in Chapter 4.

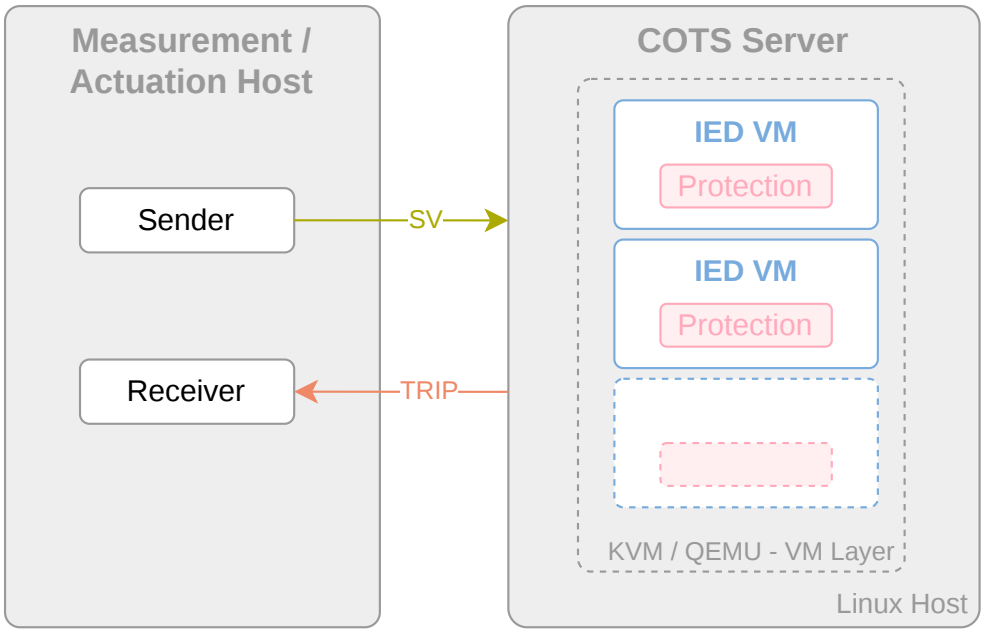


Figure 3.1: High-level system architecture used in this thesis. One physical host emulates the measurement source and trip receiver, while a second physical host acts as the COTS virtualization host where virtualized IEDs execute the protection workload as VMs and return a response to the first host.

3.2.2 Virtualized IED Execution

The second physical system represents the COTS server on which representative vPAC workloads execute, and system configurations should be evaluated.

Linux, KVM, and QEMU provide the virtualization environment for VMs. Each virtualized IED executes inside a separate VM. This preferred deployment model of one IED per VM is discussed in Section 3.4.

Within each IED, protection processing follows the pipeline introduced in Section 2.2 and implementation details are described in Section 4.4. In general, the virtualized IED receives incoming measurements, executes the protection workload, and possibly emits a trip signal, which is sent back to the first host

Essentially, the system transaction considered throughout our thesis work is:

1. Measurement packet.

2. VM reception.
3. Measurement processing.
4. Protection evaluation.
5. (optional) Response trip packet.

3.2.3 End-to-End Measurement and Thesis Scope

As we have defined our latency goal as the total time from the SV packet to the trip decision arriving, this enables the following measurement protocol: measure the timestamp when the sender emits a packet and the timestamp when the receiver receives the corresponding trip packet. It is imperative that the trip decision timestamp corresponds to the timestamp of the emitted measurement that contained or triggered a fault state, to get correct measurement results. This, for example, becomes important if the measurement includes sending a fault packet only on every n-th packet. In that case, the first $n-1$ timestamps are irrelevant, and the correct measured system latency is

$$T_{\mathrm{E2E},n} = t_{\mathrm{receive},n} - t_{\mathrm{send},n}.$$

Since the timestamps in our system are taken in userspace applications on the external measurement machine, the resulting trip decision latency includes all processing between these two points. Conceptually, the latency can be decomposed as

$$T_{\mathrm{E2E}} = T_{\mathrm{forward}} + T_{\mathrm{VM}} + T_{\mathrm{return}},$$

where T_{forward} and T_{return} describe the complete path from the sender to the VM and from the VM to the receiver respectively. T_{VM} describes the latency introduced while processing the request inside the VM.

The forward and return terms, therefore, include the Ethernet transmission time and the networking paths through both physical hosts on the send and receive sides.

The term T_{VM} contains the guest-visible processing, which is the primary interest in this thesis. This term includes interrupt, scheduling, and virtualization delays, as well as hot-path processing. Selected components of this term are investigated separately.

In the interest of providing a complete experimental reference transaction, the networking contributions to the end-to-end measurements are retained. However, as mentioned previously, they are not the primary focus of this thesis work.

3.3 VM vIED Processing Architecture

In Section 2.2, we introduced the commonly found IED structure, which maps naturally to our VM idea. We instantiate the general model by designing the protection-critical hot path as a single Linux application within each IED. The process therefore owns the complete protection processing path from input reception to output generation.

3.3.1 Single Protection Hot Path

The latency-critical hot-path is organized as a sequential processing pipeline. This also occurs naturally, as each stage requires the outputs of a previous stage. The individual ANSI protection functions are treated as computational modules within this application. All stages and ANSI functions are therefore not independently scheduled operating-system processes. Therefore, this allows protection computation to occur without introducing additional context-switching latency or complicated process scheduling and inter-process communication.

Alternative designs could possibly break this structure up into several processes or threads and schedule them independently. Such an organization may possibly prove beneficial if certain protection elements have different priorities or operate at different rates. For the workload considered in this thesis, however, a sequential processing model provides a simple and more explicit path, allowing a more direct characterization of execution time and scheduling behavior. The ANSI protection functions and workload we consider consist of bounded computations over a small shared state.

3.3.2 Layered Data Flow

Although the hot-path executes within a single process, the layered structure of the protection workload allows it to be logically separated into multiple layers. This separation is beneficial because it allows for the design of boundaries between the different stages, which are supported by the data consumed by those stages. The protection functions do not directly operate on the values included or the network representation of the data. This keeps the stages independent of one another. The key details of the precise implementation are described in Chapter 4.

3.3.3 Separation of Auxiliary Functionality

As previously mentioned, an IED may contain additional auxiliary functionality such as logging, monitoring, or configuration interfaces. These services may be important for an actual deployment, but do not participate in the IED's protection functionality.

Consequently, the main goal is for auxiliary functionality not to disrupt the timely execution of the hot-path. If information from the hot-path must be exported to auxiliary services, such as logging a trip decision, the hot-path should be allowed to perform only a timely, bounded operation, while the potentially slower process (i.e., writing to disk) is performed separately.

This separation is of a logical nature and does not prescribe any particular CPU allocation scheme to VMs. The following sections, therefore, consider how VM and vCPU resources can be assigned such that this distinction can be upheld and represented in the execution environment.

3.4 IED Deployment Model

As mentioned in Subsection 3.2.2, we chose to adopt the model of one VM representing one virtualized IED. However, we can also consider alternative models, such as consolidating multiple IEDs within a single VM. Questions such as isolation, resource ownership, and specific IED process models then arise.

The model we chose to follow provides clear ownership of resources such as memory, CPU time, operating system state, and device-specific software. Furthermore, it allows for a clear environmental boundary for virtualized IED vendors. Vendors can use independent software stacks and configurations without requiring them to share the same guest environment with other vendors. The same boundary also applies to auxiliary functionalities.

In general, this model simplifies isolation and makes the behavior of each virtualized IED simpler to reason about. Consolidation of multiple IEDs on one hardware computing system is also not prevented, as a multi-core server can still host multiple independently isolated VMs.

However, running multiple IEDs inside of a single VM remains a valid alternative. Especially when the key focus of a system is increasing consolidation density rather than maintaining a separate operating-system boundary for every virtualized IED. We therefore do not consider such an alternative approach as inherently inferior, but it is outside of the scope of this thesis and is left as a possible direction for future investigation.

3.5 VM CPU Topologies

In Section 2.5, we have introduced mechanisms to aid in the separation between latency-critical and housekeeping activity. This separation can and should also be applied inside a virtualized guest IED. In this thesis, we primarily consider two possible VM CPU

topologies: a *resource-efficient one-vCPU* and an *isolation-oriented two-vCPU* topology. Both are compatible with the previously introduced one IED per VM deployment model (cf. Section 3.4).

3.5.1 One-vCPU Topology

In the one-vCPU topology, the complete guest must execute on a single vCPU. Namely, housekeeping activity, auxiliary IED activity, the protection hot-path, and relevant interrupt activity must share the same virtual processor. On the host side, the corresponding vCPU thread can be pinned to a dedicated, isolated pCPU to prevent execution contention, essentially running a VM with one full pCPU.

The main idea of this topology is consolidation density. If IEDs are deployed using this topology, more IED VMs can fit onto the same physical core count. Priority execution of the latency-critical hot-path and related IRQ threads over lower-priority auxiliary activity is established by PREEMPT_RT scheduling policies and priorities. However, all guest work must ultimately share and make progress on the same processor, so the available CPU time must accommodate both the latency-critical RT workload and auxiliary activity.

3.5.2 Two-vCPU Topology

The two-vCPU topology enables the guest to utilize two vCPUs and therefore assign explicit roles to them. One vCPU can be used for the housekeeping and auxiliary workloads, whilst the second vCPU can dedicate all of its CPU time to mission-critical RT workloads.

The main idea of this topology, in contrast to the one-vCPU topology, is the separation principle. In the same way that our model separates housekeeping from latency-critical protection computations, we separate the actual work performed into two distinct virtual processors. The RT vCPU is pinned to an isolated pCPU and the VM is thus able to allocate a full pCPU to the RT workload, whilst the housekeeping vCPUs of multiple VMs can be consolidated on a pCPU pool if sufficient capacity is available.

This topology follows a common real-time, low-latency isolation pattern, in which our aforementioned distinction between housekeeping and low-latency workloads is supported by utilizing two distinct vCPUs pinned to two distinct sets of pCPUs (cf. Chapter 7). However, this additional vCPU does not inherently make the protection workload execute faster. The main purpose is to provide an additional processing resource with which housekeeping tasks can make progress without competing for CPU time with the hot-path.

3.5.3 Isolation and Resource Trade-off

The two considered topologies, therefore, represent an inherent trade-off between resource efficiency and isolation. Neither topology is assumed to be superior at design time; the choice depends on the specific use cases. If the protection and housekeeping workloads remain sufficiently small, the one-vCPU topology may provide suitable real-time behavior with better resource efficiency. If stronger isolation or additional auxiliary or housekeeping capacity is desired, the two-vCPU topology may become more attractive. The actual latency behavior and resulting trade-offs are evaluated experimentally later in Chapter 5.

3.6 Scheduling and Isolation Principles

The low-latency protection processing architecture we have established and detailed in this chapter requires a corresponding scheduling and isolation strategy. The main objective is the timely execution of protection and related functionality, without unrelated work interfering. The design is therefore a combination of PREEMPT_RT fixed-priority scheduling within the guest, and explicit CPU and interrupt placement on both the guest and the host.

3.6.1 Guest Scheduling Hierarchy

Inside each VM, the protection application executes using a high-priority `SCHED_FIFO` policy. However, the protection application depends on receiving the measurement packets from the kernel. Therefore, the network IRQ threads required for this path are treated as part of the execution chain. The intended priority relationship is

$$\begin{aligned} \text{required network IRQ threads} &> \text{protection application} \\ &> \text{auxiliary real-time activity} \\ &> \text{ordinary housekeeping.} \end{aligned}$$

Assigning the relevant network IRQ threads a higher priority than the protection application ensures that packet delivery can complete even if the protection task or other real-time work is runnable. This enables further subdivision or inclusion of real-time tasks, while ensuring that whenever packets arrive, they are processed by the IRQ threads and made available to the protection application. This becomes especially important under high RT CPU utilization, where otherwise lower-priority network IRQ threads could be delayed by other running real-time work. Once the required

packet processing has finished, the protection application becomes the highest-priority runnable process and can perform protection computations.

3.6.2 Host-Side Isolation

To ensure that the guest-side scheduling hierarchy fulfills its intended purpose, we must ensure that the latency-critical vCPU thread is not interfered with by unrelated activity on the host system. Thus, the design assigns real-time vCPUs dedicated isolated pCPUs. Furthermore, unrelated host processing, kernel activity, and interrupt handling should be directed towards housekeeping pCPUs on the host. QEMU emulator threads are similarly kept separate from the pCPUs executing the RT vCPUs. The only thing remaining executing on a dedicated, isolated RT pCPU should be the RT vCPU thread.

Likewise, power-management behavior is treated as another part of this isolation strategy. Latency-critical pCPUs should avoid transitions into idle states to avoid introducing additional wake-up latency.

The exact kernel parameters used, scheduling priorities assigned, and configurations applied are detailed in Chapter 4.

4 Implementation

In this chapter, the concrete implementation details of the physical systems, the guest VMs and the representative vPAC workload are described. In the following sections, source code and configuration artifacts are mentioned, including *Nix* system configuration files, libvirt VM XML files, and `cloud-init` VM provisioning files, all of which are publicly available [1].

4.1 Host and Guest Configuration

As detailed in Section 3.2, our system design consists of two machines, each with 12 CPU cores: one serves as the sender of measurement data and the receiver of trip signals, and the other serves as the configurable COTS platform running vPAC workloads. We implement this design using ***River*** as the measurement and actuation system and ***Christina*** as the virtualization host. Both machines are connected via a direct Ethernet connection. Going forward, we will refer to the two machines as *Christina* and *River* respectively. Both machines are running the standard Linux kernel version 6.18.45 without PREEMPT_RT with NixOS[1][2] version 26.05 as the operating system.

4.1.1 Software Platform

Christina Virtualization

On *Christina*, VMs execute through KVM/QEMU managed by libvirt[3]. Libvirt is an open-source API that serves as a VM manager on top of KVM/QEMU. It allows defining VM configurations as structured extensible markup language (XML) files. The specific version details for the software used are listed in Table 4.1.

[1] `https://nixos.org/`
[2] `https://nixos.org/manual/nixos/stable/`
[3] `https://libvirt.org/`

Component	Version
QEMU	10.2.4
libvirt	12.2.0
KVM	Linux 6.18.45 (`kvm_intel`)

Table 4.1: Virtualization software versions on Christina.

Christina Networking

As detailed in Section 3.1, optimizing the network path and packet delivery to the VMs on *Christina* is out of scope for this thesis, but is investigated in companion work [25]. However, for our measurements and research, the packets must still arrive in the VMs, and the consequent trip-decision packets must still be sent from the VMs. Both of these actions require networking decisions in *Christina*.

We distinguish between two types of network traffic: protection-critical networking and ordinary networking, which may be used for monitoring and configuration. It is not necessary to include standard networking capabilities in the VMs; however, for managing the VMs and conducting experiments, they have been helpful.

Generally, to enable any type of networking between the host and guests, we use standard Linux bridges[4]. To differentiate latency-critical from general network traffic, we instantiate two separate bridges. The low-latency bridge directly forwards packets between the VMs and the network interface on *Christina*, which connects the system to *River*. We use the low-latency connection as a pure layer 2 connection, disabling DHCP and IPv6 router advertisements to reduce extraneous packets flowing on that link. The general-purpose bridge connects the VMs and the host over separate interfaces, enabling networking functionality besides low-latency packet flow.

VM Guest Environment

As previously introduced in Section 3.5, we are primarily concerned with investigating two specific VM CPU topologies, namely the one-vCPU and the two-vCPU configuration. Both of these configurations use the same Ubuntu[5] 26.04.1 LTS operating system and PREEMPT_RT Linux kernel version 7.0.0-31-realtime.

The networking configuration applied in *Christina* manifests in the VMs as two separate classes of network interfaces: two general-purpose interfaces connected to the general-purpose bridge and the low-latency interface `ens8`.

[4] `https://docs.kernel.org/networking/bridge.html`
[5] `https://ubuntu.com/`

4.1.2 Reproducible System Provisioning

Because the host systems run NixOS, this enables declarative and reproducible system configurations using NixOS configuration files[6].

Since the VMs are managed via libvirt, specific XML files describe the configurations of VMs, including resource allocation and devices. We used a clean Ubuntu cloud image as the base, where cloud-init[7] YAML files configure the VM environment properly. This configuration includes installing a PREEMPT_RT kernel, modifying specific kernel parameters, configuring networking, and registering a startup service to apply pinning and priority mechanisms to selected threads. The exact system configurations are detailed later in this chapter; essentially, cloud-init fully initializes the VM's environment.

Once again, the instantiation of the one-vCPU and the two-vCPU VMs differs only in intentional, topology-required aspects.

4.1.3 Low-Latency Kernel Configuration

This subsection details the specific kernel configuration parameters used in the host systems and VMs as introduced in Section 2.5, realizing the isolation principles described in Section 3.6. In particular, these configurations allow us to implement our distinction between housekeeping and isolated real-time CPUs on both the hosts and within the guest VMs.

Christina and River host systems

Table 4.2 details what kernel parameters are enabled on both hosts *Christina* and *River*, depicted in the table as columns ***C*** and ***R*** respectively. A checkmark in a row indicates that the parameter is enabled on that host system. These system parameters are enabled through the NixOS configuration files.

Furthermore, both hosts use the performance governor and restrict systemd services to the housekeeping CPUs (i.e., 0–7). This configuration is also enabled through NixOS configuration files.

Since *River* is included in the measured end-to-end latency, it receives sensible parameters as well, in an effort to attempt to reduce unnecessary *River*-side latency interference.

Christina is the actual system under study, thus it receives additional latency-oriented tuning intended to reduce potential host-side interference with VM execution.

[6] `https://nixos.org/manual/nixos/stable/#ch-configuration`
[7] `https://cloud-init.io/`

Parameter	Host value	Role in implementation	*C*	*R*
`nosmt`	enabled	Disable simultaneous multithreading	✓	✓
`isolcpus`	`8-11`	Reserve latency-critical CPU cores	✓	✓
`nohz_full`	`8-11`	Reduce scheduler ticks on isolated cores	✓	✓
`rcu_nocbs`	`8-11`	Offload RCU callbacks from isolated cores	✓	✓
`irqaffinity`	`0-7`	Direct default IRQ handling to housekeeping cores	✓	✓
`kthread_cpus`	`0-7`	Direct kernel threads to housekeeping cores	✓	✓
`intel_idle.max_cstate`	`0`	Disable the `intel_idle` driver; redundant while `idle=poll` is active	✓	✓
`idle`	`poll`	Avoid idle-state wake-up latency	✓	✓
`nowatchdog`	enabled	Disable lockup-watchdog activity	✓	
`tsc`	`reliable`	Mark the TSC as reliable	✓	
`skew_tick`	`1`	Desynchronize periodic CPU ticks	✓	
`rcu_nocb_poll`	enabled	Poll offloaded RCU callbacks	✓	
`audit`	`0`	Disable kernel auditing activity	✓	

Table 4.2: Host kernel parameters used for CPU isolation and latency-oriented configuration.

VM guests

Table 4.3 details what kernel parameters are enabled inside the VMs in both the one-vCPU and the two-vCPU configurations. A checkmark in a row indicates that the parameter is enabled in that VM configuration. These system parameters are enabled through GRUB configuration, specifically by setting the parameters in `GRUB_CMDLINE_LINUX_DEFAULT`.

Furthermore, as described in Section 2.3, RT throttling mechanisms, such as the fair server, can interfere with our protection processing. On the one-vCPU topology we therefore disable fair server intervention by setting `/sys/kernel/debug/sched/fair_server/cpu0/runtime` to `0`. This setting is initialized through cloud-init. In the two-vCPU topology, the workload classes are separated via the additional vCPU, thus the fair server can remain enabled.

Parameter	Guest value	Role in implementation	1-vCPU	2-vCPU
`isolcpus`	`1`	Reserve the latency-critical CPU		✓
`nohz_full`	`1`	Reduce scheduler ticks on the RT CPU		✓
`rcu_nocbs`	`1`	Offload RCU callbacks from the RT CPU		✓
`irqaffinity`	`0`	Keep default IRQ handling on housekeeping CPU		✓
`kthread_cpus`	`0`	Keep kernel threads on housekeeping CPU		✓
`intel_idle.max_cstate`	`0`	Disable the `intel_idle` driver; redundant while `idle=poll` is active	✓	✓
`idle`	`poll`	Poll while idle; avoid guest idle/HLT wake-up latency	✓	✓
`nohibernate`	enabled	Disable hibernation support	✓	✓
`nowatchdog`	enabled	Disable lockup-watchdog activity	✓	✓
`tsc`	`reliable`	Mark the TSC as reliable	✓	✓
`skew_tick`	`1`	Desynchronize periodic CPU ticks		✓
`rcu_nocb_poll`	enabled	Poll offloaded RCU callbacks		✓
`audit`	`0`	Disable kernel auditing activity	✓	✓

Table 4.3: Guest kernel parameters used for the one-vCPU and two-vCPU configurations.

In the two-vCPU configuration, we configure additional parameters to isolate the real-time vCPU, which we can later dedicate to our latency-critical workload. This isolation is similar to the isolation detailed earlier for the host system pCPUs.

Since `idle=poll` disables the normal CPUIdle drivers on x86, the simultaneously configured `intel_idle.max_cstate=0` does not further restrict idle behavior in the evaluated configuration.

Multi-VM CPUIdle Change

To enable the evaluation of multiple concurrent two-vCPU topology VMs, whilst allowing housekeeping vCPU consolidation across VMs on shared pCPUs, one more detail must be described. The two-vCPU topology utilizes `idle=poll` to avoid exiting vCPU execution when idle. However, this is a system-level parameter, and whilst beneficial toward wake-up latencies for the real-time vCPUs, this hinders effective housekeeping vCPU consolidation across VMs, as the host system cannot distinguish whether the guest vCPU thread is running actual tasks, or idle polling.

For the two-vCPU case, therefore, a desirable solution would be polling while idle on the real-time vCPU, whilst the housekeeping vCPU is allowed to exit guest execution. Thus, we remove the `idle=poll` option from the kernel configuration and reinstate the CPUIdle subsystem. The CPUIdle mechanism itself exposes per-CPU resume-latency constraints via PM QoS [23]. In the corresponding `sysfs` implementation, the special value `n/a` maps internally to a zero-latency requirement [26]. We use the `haltpoll` CPUIdle governor, which interprets this zero-latency requirement by selecting its polling idle state [27, 28].

Concretely, the `cpuidle_haltpoll` driver is loaded during guest startup. The latency constraint of the real-time vCPU is then configured by writing `n/a` to `/sys/devices/system/cpu/cpu1/power/pm_qos_resume_latency_us` (vCPU1 is the real-time vCPU in this case). The housekeeping vCPU retains default `haltpoll` behavior and may halt when no work is runnable. This mechanism enables our desired "per-vCPU idle polling" for the two-vCPU topology.

It is important to note that this change was introduced to the two-vCPU topology only as part of the multi-VM experiments, and was not yet included in the configurations of single-VM measurements and experiments.

4.2 VM Resource and CPU Placement

As mentioned in the last section, *Christina* has 12 pCPUs. We divide the available processors into two sets: housekeeping cores and isolated VM dedicated cores. However, `isolcpus` and other isolation mechanisms do not automatically place our VMs where

we want them. Libvirt allows us to explicitly pin the vCPU and QEMU emulator threads belonging to the VMs to specific pCPUs in the XML configuration files.

The general idea is that guest vCPUs are mapped to corresponding KVM/QEMU vCPU threads, and we map these to specific dedicated pCPUs on the host via libvirt configuration. This enables the vCPU thread and thus the vCPU itself not to have to compete with *Christina's* ordinary host processes. Furthermore, whilst pinning the QEMU emulator threads to dedicated pCPUs is not a fundamental requirement, it aids in conducting controlled experiments. Giving the emulator threads their own dedicated pCPUs instead of consolidating them on the housekeeping cores isolates them from variable host load, removing this aspect as a possibly confounding factor in later investigation.

In the following subsections, the specific pinnings for the one-vCPU and two-vCPU topologies, as introduced in Section 3.5, will be detailed.

4.2.1 One-vCPU Placement

In the one-vCPU configuration, all guest execution necessarily shares the sole available processor, including the real-time workload, IRQ processing, housekeeping, and background activity. Since the latency-critical protection application executes on this vCPU, it is pinned to a dedicated isolated pCPU. The guest vCPU0 is pinned to the host pCPU10. The QEMU emulator is separately pinned to pCPU11.

4.2.2 Two-vCPU Placement

In the two-vCPU configuration, we distinguish between guest housekeeping and latency-critical protection workloads by specifically allocating two vCPUs. Similarly to the reason for pinning the QEMU emulator thread to its own dedicated pCPU, we also pin the housekeeping vCPU thread to its own pCPU. Firstly, this introduces more controlled experiments and allows for more direct comparison of the different topologies. Variations in host load may affect the guest's vCPU execution, introducing measurement differences due to unrelated host contention. Secondly, both vCPUs are still part of the same VM, which shares a single kernel, memory, and devices. Therefore, delays to guest housekeeping may still introduce indirect effects on the rest of the VM and the protection application.

The mapping occurs analogously to the one-vCPU configuration, with the housekeeping vCPU0 pinned to pCPU9 and the latency-critical vCPU1 pinned to pCPU10. Table 4.4 shows the exact mappings for both configurations.

Configuration	VM execution context	Physical CPU
1-vCPU	vCPU0 (RT)	pCPU10
1-vCPU	QEMU emulator	pCPU11
2-vCPU	vCPU0 (housekeeping)	pCPU9
2-vCPU	vCPU1 (RT)	pCPU10
2-vCPU	QEMU emulator	pCPU11

Table 4.4: VM execution-context placement on *Christina* for the one-vCPU and two-vCPU configurations.

4.2.3 Multi-VM Placement Changes

For the multi-VM experiments, the host CPU allocation is adapted to improve consolidation. The latency-critical vCPU threads remain backed by dedicated pCPUs, however, the split of host housekeeping and dedicated isolated cores is different now. *Christina* reserves pCPUs 0–2 for housekeeping, whilst the remaining pCPUs 3–11 are used as the isolated experimental CPU pool. The corresponding host isolation masks are adjusted accordingly.

For the one-vCPU topology, each VM receives a dedicated pCPU for its vCPU execution. The remaining QEMU emulator and support threads for all VMs are consolidated onto a single shared pCPU. Therefore, pCPUs 3–10 can host up to eight independent VMs in the evaluated configuration, with the virtualization support processing consolidated on pCPU11.

For the two-vCPU topology, the latency-critical vCPU1 likewise receives its own dedicated pCPU. However, the VMs housekeeping vCPU0 threads are similarly consolidated onto one shared pCPU. The evaluated configuration, therefore, allows hosting up to seven concurrent two-vCPU VMs, with the RT vCPUs pinned to pCPUs 3–9, the housekeeping vCPUs consolidated on pCPU10, and virtualization support consolidated on pCPU11.

4.2.4 Additional VM Resource Configuration

Additional VM configurations are briefly described in this subsection. VMs receive 4 GiB of fixed guest memory backed by huge-pages in order to reduce TLB cache misses. The memory is locked on the host to prevent it from being swapped out of host RAM, which could introduce large delays. Furthermore, memory sharing between VMs is disabled to enable simpler, more isolated memory behavior. The CPU is passed through to the VMs. Since we want VM memory to remain fixed, to prevent

introducing unpredictable dynamic behavior, memory ballooning is disabled. Lastly, to reduce unnecessary activity, unused virtual devices, such as USB and the Performance Monitoring Unit (PMU), are disabled to minimize the total device model of the VMs.

4.3 Guest Real-Time Scheduling and IRQ Configuration

This section details the configuration and principles used in the VM topologies to execute the protection workload.

4.3.1 Protection Process Scheduling

As described in Section 3.3, our design instantiates the IED protection functionality consolidated as a single application. By choosing this design, our implementation does not require different scheduling priorities for different stages of the hot-path. We elevate the process itself to be scheduled with the real-time scheduling policy `SCHED_FIFO` with priority `95`. Any ordinary and auxiliary guest processes remain under the normal Linux scheduling policy `SCHED_OTHER`. This lets the protection application preempt any ordinary guest workload and any real-time workload with a lower priority whenever it becomes runnable.

This scheduling configuration is equivalent for both one-vCPU and two-vCPU VMs. On the two-vCPU VMs, the protection process is additionally pinned to the dedicated, low-latency vCPU1.

4.3.2 Low-Latency Network IRQ Configuration

However, our protection application can only execute once the corresponding measurement packets are delivered. In Section 3.6, we detailed this fact and presented a scheduling hierarchy including packet delivery, which will be instantiated in this section.

As described in Section 2.3, IRQ handlers execute as schedulable threads under PREEMPT_RT. Since we have a dedicated interface `ens8` for latency-sensitive network flow inside the VM, we can elevate the priority of the corresponding interface IRQ handlers, enabling the instantiation of our proposed priority hierarchy and therefore timely execution of packet delivery, whilst preventing ordinary network traffic from interfering with our latency-critical workload.

This elevation of priority is achieved via a startup service script, which is registered at VM creation time by cloud-init. Both receive and transmit network IRQ threads are elevated to `SCHED_FIFO` at priority `98` at system boot.

This scheduling configuration is again equivalent for both one-vCPU and two-vCPU VMs. On the two-vCPU VMs, the network IRQ threads are additionally pinned to the dedicated, low-latency vCPU1. This should help prevent auxiliary and housekeeping work from interfering with our latency-critical packet delivery path. If the critical IRQ handler threads are not pinned to the RT vCPU1, one potential source of interference, even with proper scheduling priorities configured, may be through VM exits (cf. Subsection 2.4.4); vCPU0 could VM exit because of auxiliary work, whilst latency-critical packets arrive and are thus not able to be delivered, because vCPU0 is not running, consequently delaying our latency-critical workload.

4.3.3 Priority Hierarchy

With the above-mentioned configurations, the VMs instantiate the proposed priority architecture using the specific scheduling policies and priorities depicted in Table 4.5.

Execution context	**Policy**	**Priority**	**Role**
`ens8` network IRQ threads	`SCHED_FIFO`	98	Critical network receive/transmit processing
Protection application	`SCHED_FIFO`	95	Latency-critical protection hot path
Auxiliary/synthetic RT workload	`SCHED_FIFO`	51–94	Optional lower-priority real-time activity
Other threaded IRQs	`SCHED_FIFO`	50	Unmodified non-critical device IRQ threads
Ordinary guest tasks	`SCHED_OTHER`	–	Non-real-time housekeeping and auxiliary work

Table 4.5: Scheduling hierarchy used inside the PREEMPT_RT guests.

The priorities are chosen relative to one another to follow this hierarchy. It is not imperative that the specific chosen priorities are the same in another system. However, it is noteworthy, as mentioned in Section 2.3, that IRQ handler threads are `SCHED_FIFO` priority `50` by default under PREEMPT_RT.

4.4 Protection Workload Implementation

This section is concerned with detailing the implementation of a representative IED protection workload as described in Section 3.3 and Section 2.2. We implemented the concrete hot-path workload as the C program `hot_replier`.

`hot_replier` itself contains the full synchronous VM hot-path from receive, over measurement preprocessing, over protection computation to a potential response output. The processing omits auxiliary functionality, such as logging, and the computations performed are themselves compact and bounded. This form of implementation represents the central protection execution path of an IED. The following sections further describe implementation details.

4.4.1 Protection Hot Path

The application communicates directly using Ethernet frames through the low-latency interface `ens8`. The frames that arrive at the protection application are not directly IEC 61850 SV frames, but their payloads are SV-inspired. Essentially, this means we are not sending actual SV frames, just the sample information required by the measurement preprocessing layer. We are not trying to benchmark full IEC 61850 SV protocol processing, but rather emulate measurement samples being carried over Ethernet to the virtualized IEDs. Similarly, the output trip signals in our case are not actual GOOSE trip L2 frames, but a frame used to delimit the end of processing. This communication representation deliberately abstracts the IEC 61850 encoding layer while retaining protection-relevant processing stages.

Furthermore, each iteration of the protection computation emits a trip-signal frame. This implementation choice is deliberate to allow observation of the end-to-end latency for every frame received. Otherwise, only frames satisfying specific protection conditions would yield a latency sample.

The reception of L2 frames in the `hot_replier` is blocking. The application waits for incoming frames, then executes the entire pipeline synchronously. This is a natural implementation in which packet arrival triggers a protection iteration.

4.4.2 Measurement Processing

After packet reception, the measurement samples are handed off to the measurement preprocessing layer. The incoming payload includes SV-like instantaneous current and voltage samples. The protection elements, however, require derived quantities. The measurement preprocessing layer is located between the packet reception and protection computation layers and is dedicated to computing the required quantities.

The preprocessing layer uses an 80-sample window and computes quantities such as magnitude, phase angle, and complex phasors over a fixed-size sample history for four voltage and current channels.

To compute the quantities, the measurement layer uses DFT, which repeatedly requires sine and cosine coefficients. Instead of recalculating them inside every iteration, we precompute the possible coefficients and reuse them. This is a regular and useful optimization, since recomputing them does not add useful protection functionality; it only adds avoidable repeated trigonometric work.

Sample history and measurement preprocessing state are kept in static, fixed-size buffers.

4.4.3 Protection Functions

After measurement preprocessing and computation of the required quantities, a common representation is passed to a dispatcher, which evaluates all protection elements sequentially. The protection functions themselves do not require any knowledge of the external pipeline or the network format.

We implemented the six ANSI protection elements as functions, which are described in Section 2.2. The internal state of the protection functions is similarly kept in static, fixed-size buffers.

4.4.4 Hot-Path Boundaries and IEC 61850 Abstraction

Inside the VM path, we measure and time:

- The L2 receive,
- payload/sample extraction,
- measurement-buffer update,
- DFT / phasor processing,
- all six protection calculations,
- the decision aggregation,
- and the L2 response send.

We deliberately omit certain aspects from the model, such as full SV and GOOSE construction, configuration-file access, event logging, database or file I/O, and possibly other auxiliary IED services. We are not trying to completely reproduce a

commercial IED. The primary goal is to construct a representative latency-critical vPAC computation path, which we can control and whose system interactions with PREEMPT_RT/KVM we can study.

4.5 Measurement and Microbenchmark Infrastructure

4.5.1 End-to-End Latency Measurement

To measure end-to-end latency according to the proposed design in Section 3.2, we implement the sender and receiver as two separate C programs, namely `hot_sender` and `hot_receiver`. Both applications execute on *River*.

`hot_sender`. The sender application operates directly on the low-latency Ethernet interface on *River* using a raw socket, which connects to *Christina*. It constructs the L2 payload with a custom EtherType and assigns a monotonically increasing `frame_id` to each frame. Additionally, representative current and voltage data, and a sender timestamp are embedded in the frame. The timestamp is obtained via `CLOCK_MONOTONIC_RAW` and embedded immediately before the frame is sent. After construction, the frame is emitted over the interface. Frame transmission is paced using a busy-wait loop rather than a sleep-based wake-up mechanism, avoiding dependency on scheduler wake-up latency for workload generation.

`hot_receiver`. The receiver also uses a raw L2 socket and listens for the custom EtherType. That is because the frame sent back as a trip signal includes the same EtherType from the sender. Additionally, the response frame includes the previously embedded `frame_id` for association and sender timestamp. Furthermore, echoing these arguments, allows the toleration of out-of-order arrival and, more importantly, the identification of lost frames. When the frame arrives, the receiver takes a timestamp, again using `CLOCK_MONOTONIC_RAW`, and since both timestamps are taken on the same physical machine, they use the same clock. Therefore, no extra synchronization is necessary, and the end-to-end latency can be computed as

$$T_{\text{E2E}} = t_{\text{receive}} - t_{\text{send}}.$$

During a run, the timestamps are stored in memory. At the end of the run, the receiver records the latency and timestamps for each `frame_id` and then writes the results to a CSV file. This avoids introducing I/O interference in our measurements.

To avoid introducing further interference into our measurements, all applications (i.e., `hot_sender`, `hot_replier`, and `hot_receiver`) lock all memory in RAM via `mlockall(MCL_CURRENT | MCL_FUTURE)` and pre-fault used memory.

This end-to-end measurement, as designed in Subsection 3.2.3, approximately includes: the userspace send and kernel network stack from *River*, the time for physical Ethernet traversal, the *Christina* kernel network stack and bridge, further virtio and guest networking, the corresponding protection processing, and the similar response path. Therefore, this measurement is not solely constituted of the VM execution time, but of our previously mentioned goal of full trip decision latency (cf. Section 3.1).

4.5.2 Standalone Processing Microbenchmarks

To aid our second goal of understanding latency introduced by the protection computation itself, we implemented microbenchmarks for our respective measurement preprocessing computations and ANSI protection functions. Two standalone benchmarks were implemented to characterize and isolate the computational cost of these components from the larger end-to-end path.

The common timing procedure is illustrated in Algorithm 1. Timestamps are obtained before and after a batch of repeated function invocations, and the resulting batch duration is stored as a single timing sample. Batching the benchmarked functions is important because the individual functions execute extremely quickly, often in the order of tens to hundreds of nanoseconds. If we time only one function call between two timestamp retrieval instances, the timing mechanism itself would be large relative to the operation under test. Essentially, this design allows us to amortize the timestamping overhead.

Algorithm 1 Batched microbenchmark timing procedure.

```
for i ← 1 to N_samples do
    t_start ← ClockGetTime(CLOCK_MONOTONIC_RAW)
    for j ← 1 to N_batch do
        r ← BenchmarkFunction
        Consume(r)
    end for
    t_end ← ClockGetTime(CLOCK_MONOTONIC_RAW)
    samples ← samples ∪ {t_end − t_start}
end for
```

Similar to the end-to-end related applications, the microbenchmark applications lock and pre-fault all memory. Additionally, the microbenchmarks warm up caches

by repeatedly executing the target function before recording samples. Furthermore, benchmark function results are consumed through volatile sinks, so the compiler cannot simply optimize away the work. Statistics are computed and written to disk only after the benchmark completes.

4.6 Multi-VM Test Support

In addition to the kernel configuration changes and CPU placement changes introduced in Subsection 4.1.3 and Subsection 4.2.3, respectively, the end-to-end measurement infrastructure introduced in the preceding sections was extended slightly to support multiple concurrently active VMs. The general structure remains unchanged; however, now one measurement frame is transmitted to all participating VMs, and the corresponding results must be associated with them. It is important to note that this version of the test binaries was used only for the multi-VM experiments.

Sending The `hot_sender` remains unchanged; it simply transmits the measurement frames to the Ethernet broadcast address `ff:ff:ff:ff:ff:ff`. The low-latency Linux bridge on *Christina* consequently fans out each frame to every connected VM.

Computation Each VM is assigned a unique low-latency MAC address whose final byte encodes its VM_id. The `hot_replier` now uses this MAC address as the source of its response.

Response The `hot_receiver` was extended with an additional argument for the expected number of replying VMs. The response processing remains the same, whilst the VM_id is additionally extracted from the source MAC address. With the extra argument, the `hot_receiver` can preallocate all necessary storage for the entire run across all expected participating VMs. This also allows us to continue to explicitly identify missing responses. Thus, the resulting output now, in addition to the already existing sample data, also contains the responding VM_id of this data.

4.7 Reference System Configurations

Both configurations use the same physical host and guest environments described in Section 4.1. In particular, the baseline virtualization host *Christina* uses the non-PREEMPT_RT NixOS kernel. The detailed host and guest kernel parameters are listed in Table 4.2 and Table 4.3, respectively.

Similarly, both VMs run Ubuntu 26.04.1 LTS with the 7.0.0-31-realtime PREEMPT_RT kernel and use 4 GiB of fixed, locked, huge-page-backed memory. Real-time fair-server intervention is disabled in the one-vCPU topology by setting `/sys/kernel/debug/sched/fair_server/cpu0/runtime` to `0`. The protection application executes using `SCHED_FIFO` priority 95, and the latency-critical `ens8` network IRQ threads at priority 98. The complete scheduling hierarchy is given in Table 4.5.

The main differences between the two VM configurations are summarized in Table 4.6.

Property	**1-vCPU configuration**	**2-vCPU configuration**
Guest CPU topology	One vCPU	Two vCPUs
Latency-critical vCPU	vCPU0	vCPU1
Guest housekeeping	Shares vCPU0 with the RT workload	Primarily assigned to vCPU0
Host vCPU placement	vCPU0 $\rightarrow$ pCPU10	vCPU0 $\rightarrow$ pCPU9, vCPU1 $\rightarrow$ pCPU10
QEMU emulator placement	pCPU11	pCPU11
Protection process affinity	vCPU0	vCPU1
`ens8` IRQ affinity	vCPU0	vCPU1
Guest RT CPU isolation	Not applicable	vCPU1 isolated from guest housekeeping
Protection scheduling	`SCHED_FIFO`, priority 95	`SCHED_FIFO`, priority 95
Critical network IRQ scheduling	`SCHED_FIFO`, priority 98	`SCHED_FIFO`, priority 98
RT fair-server intervention	Disabled	Default enabled

Table 4.6: Reference one-vCPU and two-vCPU system configurations used in the evaluation.

The exact host-side CPU mappings are detailed in Table 4.4. The primary difference between the two configurations lies in the amount of allocated vCPUs and the resulting processing structure. All other implementation parameters are kept equivalent where possible.

The configurations summarized in this section represent the reference for the single-VM configurations used in the associated evaluation experiments. For the multi-VM scalability evaluation in Section 5.6, selected CPU-placement changes and two-vCPU idle-handling are adapted as described further in Subsection 4.1.3 and Subsection 4.2.3.

5 Evaluation

5.1 Experimental Methodology

Unless stated otherwise, all experiments use the reference system configurations defined in Section 4.7. Thus, specific CPU placement, scheduling, and virtualization configurations, as well as other configurations, are not repeated for each experiment. Individual evaluation sections solely state the parameters deliberately changed from this reference configuration.

5.1.1 Physical Testbed

The evaluation uses the two physical systems introduced in Section 4.1. Both systems use the same base server platform and processor. The hardware that is relevant to the measurements is briefly summarized in Table 5.1.

Property	**Christina**	**River**
System	Supermicro SYS-110P-WTR	Supermicro SYS-110P-WTR
Processor[a]	Intel Xeon Gold 5317 @ 3.00 GHz	Intel Xeon Gold 5317 @ 3.00 GHz
CPU topology	1 socket, 12 physical cores	1 socket, 12 physical cores
Low-latency NIC	Intel Ethernet Controller X550	Intel Ethernet Controller X550
Interface	`eno1`	`eno1`
NIC driver	`ixgbe`	`ixgbe`
Physical link	10 Gb/s full-duplex Ethernet	10 Gb/s full-duplex Ethernet
Role	KVM/QEMU virtualization host	Traffic generation and latency measurement

[a] `https://www.intel.com/content/www/us/en/products/sku/215272/intel-xeon-gold-5317-processor-18m-cache-3-00-ghz/specifications.html`

Table 5.1: Physical hardware used for the evaluation.

The two systems are directly connected over an Ethernet link via their `eno1` interfaces, without any additional Ethernet switch in the measured path.

On *River*, the `hot_sender` and `hot_receiver` applications are pinned to CPU 8 and CPU 9, respectively, with the goal of reducing possible latency interference on the *River* host-side. Furthermore, *River* uses the standard host kernel rather than a PREEMPT_RT kernel, and no real-time scheduling policies or priorities are assigned.

5.1.2 Workload and Experiment Procedure

The end-to-end measurement implementation is detailed in Section 4.5. As described in Subsection 2.1.1, the sender is configured to send out frames approximately every 250 µs. This corresponds to approximately 4000 transmitted frames per second.

The transmitted protection-relevant samples are synthetic inputs. Their purpose is to apply the same measurement and protection processing to each set of packets and to trigger the emission of a consequential trip. Typically, and unless stated otherwise, a short end-to-end experiment consists of 100,000 transmitted frames and is repeated three times per experiment.

Experiments that require the generation of additional stress or CPU computation typically utilize `stress-ng`[1]. Unless stated otherwise, the CPU stressor method utilized is `--cpu-method=matrixprod`; additional specific parameters used are stated for each experiment.

Pinning processes to specific CPU cores and setting specific scheduling policies and priorities are common mechanisms. These tasks are performed with the `taskset`[2] and `chrt`[3] utilities, respectively.

5.1.3 Reported Metrics

End-to-end results are primarily characterized using the median and percentile latencies. Observed maxima, tail latencies, and lost frames are included where useful.

Latency distributions are visualized using empirical cumulative distribution functions (ECDFs), where the dense part of the distribution is of interest. Time-series plots are used where specific effects can be observed over time. Rare events that are not readily visible in the corresponding plots but are of interest are reported in a compact table alongside the figure.

It is imperative to note that observed and reported maximum latencies are empirical measurements and must not be interpreted as formal guarantees or hard latency bounds.

[1] `https://github.com/ColinIanKing/stress-ng`
[2] `https://man7.org/linux/man-pages/man1/taskset.1.html`
[3] `https://man7.org/linux/man-pages/man1/chrt.1.html`

5.1.4 Measurement Instrumentation

The primary, end-to-end latency measurements are collected without additional tracing of other tools. If additional tools are required for specific experiments, they are introduced in the corresponding sections. One important note is that detailed tracing, for example via the `perf` utility, can itself perturb the measurements.

5.2 Scheduling and Isolation Mechanisms

> **RQ2:** Which scheduling and isolation mechanisms are important for maintaining low and stable latency under load?

This section evaluates the scheduling and isolation mechanisms underlying the reference configuration introduced in Section 4.7. Unless stated otherwise, experiments use the two-vCPU topology. As detailed, this topology separates the guest housekeeping from the latency-critical vCPU. The one-vCPU and two-vCPU configurations are compared directly in Section 5.4.

We consider the evaluation of three aspects. First, the generic and PREEMPT_RT guest kernels are compared under ordinary CPU and I/O-contention conditions. Secondly, we evaluate the priority assigned to the `ens8` low-latency network IRQ thread in the presence of competing real-time work. Finally, we investigate the interaction between real-time execution and fair-scheduled tasks on the isolated vCPU.

5.2.1 Guest PREEMPT_RT under CPU and I/O Contention

The goal of this experiment is to isolate and evaluate the effect of the guest kernel. We compare pairs of two-vCPU and one-vCPU VMs, which only differ in the kernels used: 7.0.0-31-generic and 7.0.0-31-realtime. Essentially, we compare PREEMPT_RT and non-PREEMPT_RT VMs in both topologies. Apart from that, the configuration, such as scheduling policies and priorities, remains the same.

We perform two sets of experiments, and the following subsection covers the first: we evaluate the performance of end-to-end timings, and additionally conduct scheduling latency experiments with `cyclictest` (cf. Subsection 2.6.2) for the two-vCPU topology. Both measurements are performed without and with an additional 95 % `SCHED_OTHER` CPU workload pinned to the low-latency vCPU1.

Two-vCPU PREEMPT_RT performance under CPU load

The protection `hot_replier` was intentionally kept scheduled with `SCHED_FIFO` priority 95 in the generic kernel. Fixed-priority real-time scheduling is also available without

PREEMPT_RT. Therefore, this experiment can isolate the effects of the actual kernel, rather than mixing kernels with application scheduling policies.

For the PREEMPT_RT guest, the `ens8` low-latency network IRQ threads were deliberately left at the default PREEMPT_RT priority 50. Elevated IRQ priorities are evaluated separately in Subsection 5.2.2. This does not demonstrate a problem with our designed priority hierarchy, since the replier spends most of its time blocked, waiting for incoming frames, and therefore does not starve the network IRQ process.

The CPU load is generated via `stress-ng` using the `--cpu-load=95` argument. The `cyclictest` measurements were performed on the low-latency vCPU using `SCHED_FIFO` priority 95 and a 1 ms wake-up interval for a duration of 120 s per run.

Results

Figure 5.1 compares the end-to-end distributions. The plot depicts the ECDFs up to the 99th percentile, showing the bulk of the distribution. All three separate runs were combined into a single dataset for each group. Inside the groups themselves, the distributions were equivalent. The table below the plot shows the ranges of calculated quantities from the three experimental runs per group. The maximum observed latency was 280.7 µs. No frame exceeded 1 ms in any of these runs.

Both kernels remain highly stable under the ordinary CPU workload. The scheduler-latency measurements via `cyclictest` show the same quantitative result. Only single-digit-to-low-double-digit microsecond maxima were reported for both kernels. Thus, the measurements do not show a latency advantage from PREEMPT_RT in this scenario.

The result follows logically from the scheduling and isolation configuration. The benchmarked protection process executes under a higher-priority `SCHED_FIFO` policy, whereas the generated `SCHED_OTHER` CPU load runs under a lower-priority policy. Additionally, the applied isolation configuration already removes most unrelated activity from the critical vCPU1. Furthermore, pure CPU stress load runs primarily in userspace and does not deliberately invoke kernel and interrupt paths for which PREEMPT_RT is expected to provide its largest preemptibility benefits. Thus, PREEMPT_RT does not really have any interference to eliminate in this experiment. This result also shows that PREEMPT_RT is not necessarily an inherently lower-overhead kernel. For this workload, the generic guest actually achieves slightly lower end-to-end latency.

However, the workloads in VMs may be diverse, as virtualized IEDs may contain auxiliary functionality such as remote management and logging. Therefore, we additionally perform a second experiment using a synchronous disk I/O workload that more heavily exercises the kernel and interrupt activity. Such activity is relevant to auxiliary IED functionality, such as logging, although the stressor is deliberately more intensive, intended as an adverse condition to expose scheduling behavior.

PREEMPT_RT performance under I/O load

The workload is generated using the `stress-ng` HDD stressor with 4 kB writes and synchronous-data semantics. Simultaneously, `cyclictest` again measures guest-visible wake-up latency at `SCHED_FIFO` priority 95 with a 1 ms activation interval. Three 120 s runs are performed per condition.

In this experiment, both one-vCPU and two-vCPU topologies are evaluated. In the one-vCPU topology, both the disk workload and `cyclictest` necessarily must share vCPU0, whereas the disk workload is evaluated on both vCPU0 and vCPU1 on the two-vCPU topology.

Results

The results are detailed in Table 5.2. In contrast to pure CPU contention, synchronous disk I/O exposes a difference between the kernels. The dense parts of the scheduling-latency remain similar; however, the generic kernel exhibits a noticeably larger upper tail when the disk load is colocated.

In the one-vCPU configuration with colocated disk activity, the generic kernel reaches a pooled 99.99th percentile of 11 µs, compared with 7 µs under PREEMPT_RT. The maxima of the runs range from 18 µs to 27 µs, whereas the PREEMPT_RT maxima remain between 8 µs and 11 µs. The difference is further noticeable in the frequency of these elevated observations: the generic kernel contains 41 observations above 10 µs, 15 above 15 µs, and four above 20 µs across 360,000 samples. However, under PREEMPT_RT, only one observation exceeds 10 µs, the 11 µs maximum.

A very similar behavior is observed in both two-vCPU configurations, particularly for percentiles and the long tail. For the disk workload executing on the RT vCPU1, the generic kernel contains 51 observations above 10 µs, including 22 above 20 µs, whereas only three observations exceed 10 µs under PREEMPT_RT and none exceed 15 µs.

Moving the disk workload to the housekeeping vCPU most clearly benefits the generic kernel in the percentiles, supporting the general separation principle applied in the two-vCPU topology. However, such a uniform advantage is not displayed in the PREEMPT_RT case. But the main kernel-dependent result and advantage is the reduction of the upper tail of scheduling latency in this experimental scenario.

Additionally, interrupt counters were observed before and after each `cyclictest` run, which confirm substantial interrupt generation by the workload.

The observed reduction in the upper tail of wake-up latency is consistent with the mechanisms introduced by the PREEMPT_RT patch. Namely, kernel execution preemptibility is enhanced, and interrupt handling is largely moved into schedulable, preemptible threads. The high-priority `cyclictest` task can therefore preempt a

Topology	Guest kernel	Disk-I/O placement	Mean	p99	p99.9	p99.99 [µs]	Run maxima
1 vCPU	Generic	None	2.007	2	3	6	10–18
1 vCPU	PREEMPT_RT	None	2.007	2	4	9	7–13
1 vCPU	Generic	Shared vCPU0	3.275	5	6	11	18–27
1 vCPU	PREEMPT_RT	Shared vCPU0	3.179	4	5	7	8–11
2 vCPU	Generic	None	2.018	2	5	6	7–19
2 vCPU	PREEMPT_RT	None	2.007	2	4	7	9–11
2 vCPU	Generic	RT vCPU1	2.356	6	7	14	27–35
2 vCPU	PREEMPT_RT	RT vCPU1	2.391	6	7	8	10–14
2 vCPU	Generic	housekeeping vCPU0	2.075	3	4	7	21–24
2 vCPU	PREEMPT_RT	housekeeping vCPU0	2.222	4	5	9	12

Table 5.2: Guest-visible scheduling latency with and without synchronous disk-I/O activity. Each condition pools three 120 s `cyclictest` runs with approximately 120,000 wake-ups per run. `cyclictest` executes with `SCHED_FIFO` priority 95. Run maxima report the range of the individually observed maxima rather than an averaged maximum.

larger fraction of otherwise interfering kernel activity. But it is important to note that the measurements do not trace individual latency samples to their specific causes. Nevertheless, it seems feasible that the repeatedly observed difference between the kernels demonstrates a related advantage.

Thus, the combined experiments of this subsection do not show that PREEMPT_RT is intrinsically faster. Under baseline and ordinary CPU contention, both kernels exhibit similar scheduling latency distributions. However, under sustained I/O-intensive activity, PREEMPT_RT retains a tighter observed `cyclictest` latency tail. Since reducing latency variability and limiting upper latency tails are important objectives of real-time systems, as described in Section 2.6, this provides compelling motivation for retaining PREEMPT_RT in the guest configuration.

5.2.2 Network IRQ Scheduling

In this experiment, the goal is to evaluate the interaction of interrupt handler threads with competing real-time execution in the guest. In general, PREEMPT_RT changes interrupt processing by moving execution into schedulable kernel threads. This feature allows interrupt handlers to participate in the same scheduling hierarchy as regular processes, thereby giving system administrators greater control.

The experiment evaluates the end-to-end latency of the two-vCPU PREEMPT_RT

configuration. The protection replier is fixed at `SCHED_FIFO` priority 95, whilst a competing real-time CPU workload is pinned to the same latency-critical vCPU1 executing at priority 90. The network IRQ threads are then assigned either priority 50 or priority 98. All other configuration parameters were kept constant. The real-time CPU workload is generated similarly to the last subsection using `stress-ng` with the same parameters.

Without any competing real-time workload, both IRQ handler priority configurations process all 100,000 packets without loss. Raising the IRQ priority reduced the normal-path latency by approximately 2 µs: the median decreases from approximately 43.0 µs to 40.7 µs, whilst the 99.9th percentile decreases from approximately 48.5 µs–49.0 µs to 46.4 µs–47.1 µs.

However, when competing real-time work is introduced, the difference becomes drastic. With the network IRQ threads left at priority 50, the continuously runnable, priority 90 workload can prevent the network IRQ threads from making progress, resulting in network buffers filling, unacceptable latency, and lost frames. Across the three repeated runs, only approximately 83.4 % to 84.5 % of the expected responses were received. The median latency increases to approximately 90 ms to 94 ms, and approximately 95 % of the received responses exceed the 5 ms target. Observed maxima range from 318 ms to 476 ms.

Raising the priority of the IRQ threads to 98 resolves the issues. With the intended priority hierarchy restored, the IRQ threads preempt the real-time workload before waking the protection application. Packet loss is eliminated, and end-to-end latency returns to low-microsecond levels.

As an additional measurement, we applied the same real-time workload to the generic non-PREEMPT_RT guest. Unlike the PREEMPT_RT priority-50 configuration, the generic kernel remained stable. All frames were returned, and the median and 99.9th percentiles were approximately 40.1 µs and 46 µs respectively. This shows that the workload is not responsible for these issues on its own; rather, they result from the execution model of PREEMPT_RT combined with an unstable priority hierarchy.

Figure 5.2 depicts a plot of the latency ECDF up to the 99th percentile of the configurations that remain stable, whilst the table details key quantities related to selected configurations and the failing configuration.

Results

This experiment again demonstrates that PREEMPT_RT is not intrinsically faster than the generic kernel. Rather, PREEMPT_RT provides a level of control that is very useful for constructing the specific scheduling hierarchy defined in Section 4.3 and, more generally, for controlling IRQ threads in a way not possible with the generic kernel.

However, the priority ordering must be carefully considered in light of the individual use case and the system's properties. Improperly configured IRQ thread priorities can undermine the benefits and ultimately perform worse than a generic kernel.

5.2.3 Fair-Task Contention on the RT vCPU

The evaluated guest Linux kernel is compiled without `CONFIG_RT_GROUP_SCHED`. Consequently, the system-wide `sched_rt_runtime_us` and `sched_rt_period_us` parameters do not directly limit `SCHED_FIFO` execution in this configuration. Instead, the kernel provides a per-CPU configuration mechanism, which reserves time for fair-scheduled tasks when real-time work would otherwise monopolize a CPU; namely, the per-CPU fair server.

On guest vCPU1, the default fair-server configuration reserves 50 ms within a 1 s period. To characterize the effect of this mechanism, four conditions were evaluated, differing in fair-server period and runtime, and whether any fair-scheduled ordinary load was runnable on vCPU1. Table 5.3 depicts the four conditions evaluated in this section.

It is important to note that, in this case, RT throttling via fair-server intervention occurs only when the RT workload monopolizes the CPU. Our protection application executes only for a short period of time and then blocks, waiting for the next input frame. Therefore, to properly analyze the mechanism in question, a 95 % CPU-usage RT `stress-ng` workload was launched on vCPU1 across all four conditions to establish a monopoly of RT workload over the RT vCPU.

Condition	Fair runtime	Fair period	SCHED_OTHER load on vCPU1
C1	50 ms	1 s	No
C2	50 ms	1 s	Yes
C3	25 ms	0.5 s	Yes
C4	0 ms	1 s	Yes

Table 5.3: Fair-server configurations used to evaluate mixed scheduling classes on the isolated RT vCPU. C2 and C3 reserve the same 5 % CPU bandwidth using different periods.

C1 In C1, no fair-scheduled workload was placed on the RT vCPU. Although the default fair server remained configured with a runtime per period available to fair-scheduled tasks, no periodic latency disturbance was observed. This is in direct contrast to the traditional `sched_rt_period_us` and `sched_rt_runtime_us` sysctl parameters,

which throttle RT applications when the runtime limit is reached, even if no fair-scheduled work is awaiting execution. Across the three runs, mean latency remained approximately 42 µs, and the 99.9th percentile remained below 50 µs. This experiment shows that a configured fair-server alone does not automatically incur periodic latency interference. If no fair task requires CPU time, the RT tasks are allowed to continue execution.

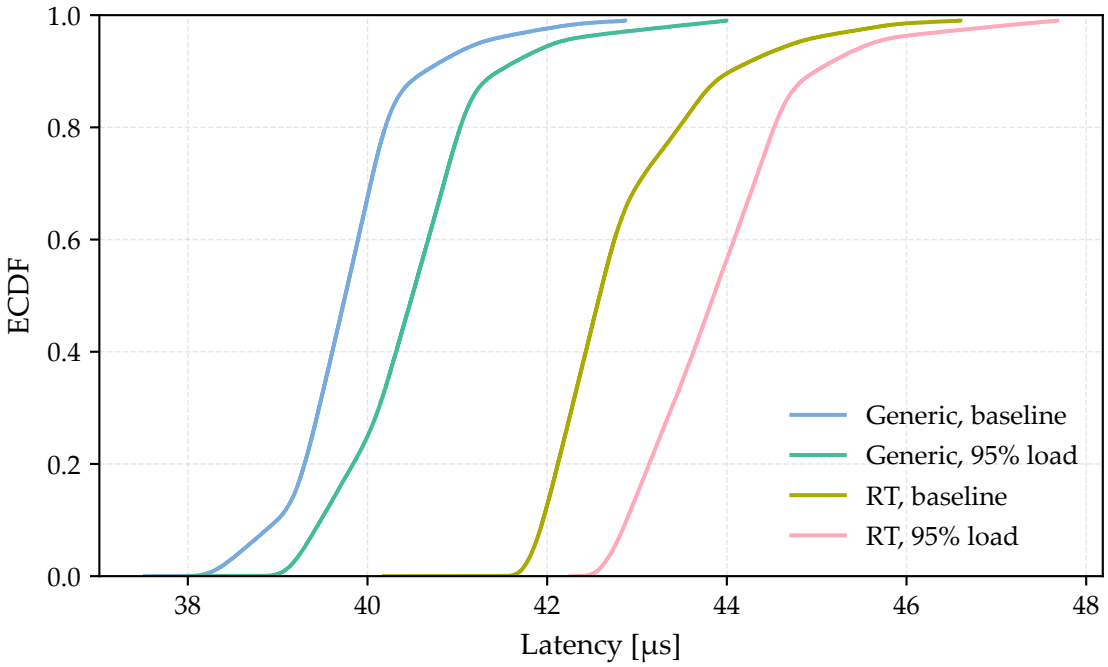
C2 C2 adds a continuously runnable `SCHED_OTHER` process to the RT vCPU. Under this condition, large periodic latency spikes appear, with the spike frequency corresponding to the fair-server configured period of 1 s and the spike magnitude corresponding to the fair-server configured runtime of 50 ms. A representative run reaches a maximum of 51.1 ms, with a 99th percentile of 40.6 ms and a 99.9th percentile of 49.4 ms. The median, however, remains at only 41.8 µs, indicating highly periodic interference instead of a shift of the latency distribution.

C3 For C3, the reserved fair-scheduled fraction of time per period is kept constant at 5 %, but the fair-server period and runtime are both halved. The latency pattern changes accordingly: The latency spike frequency has doubled to a spike occurring every half second, and the magnitude of the spike has halved to 25 ms. A representative run reaches a maximum of 25.4 ms, with a 99th percentile of 20.3 ms and a 99.9th percentile of 24.8 ms. The change in frequency and magnitude compared to C2, while preserving the same reserved CPU fraction, strongly associates the observed latency interference with fair-server execution.

C4 Finally, C4 disabled the fair-server on vCPU1, by setting the runtime to zero, whilst retaining both real-time and fair-scheduled workloads. The periodic latency spikes disappear, and the distribution returns to C1 levels. Across the three C4 runs, the mean latency remains at approximately 42.1 µs, the 99.9th percentile remains below 50 µs, and no packet exceeds 1 ms.

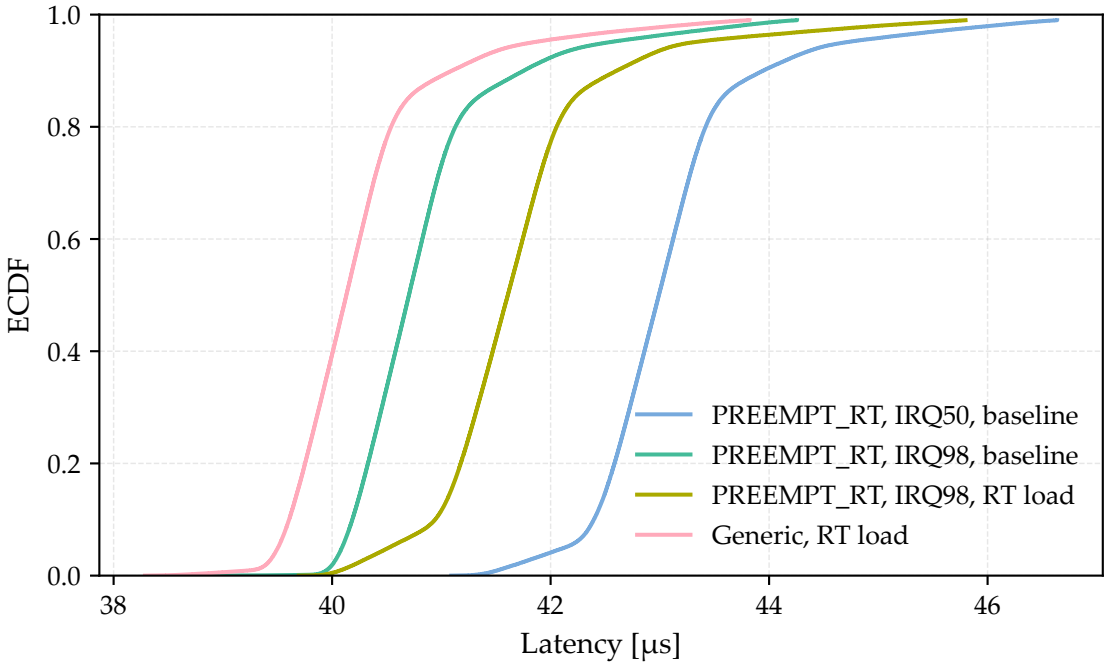
These results demonstrate that large periodic latency delays can arise when fair-scheduled work is simultaneously runnable on the isolated RT vCPU. This result directly supports the design principle of separating guest housekeeping and auxiliary functionality from the mission-critical real-time workload.

Disabling the fair-server is also a feasible solution for allowing ordinary processes to execute alongside real-time processes on a dedicated vCPU. However, this disables the safety mechanism that prevents potentially faulty RT processes from monopolizing a CPU; therefore, this option requires careful system design.



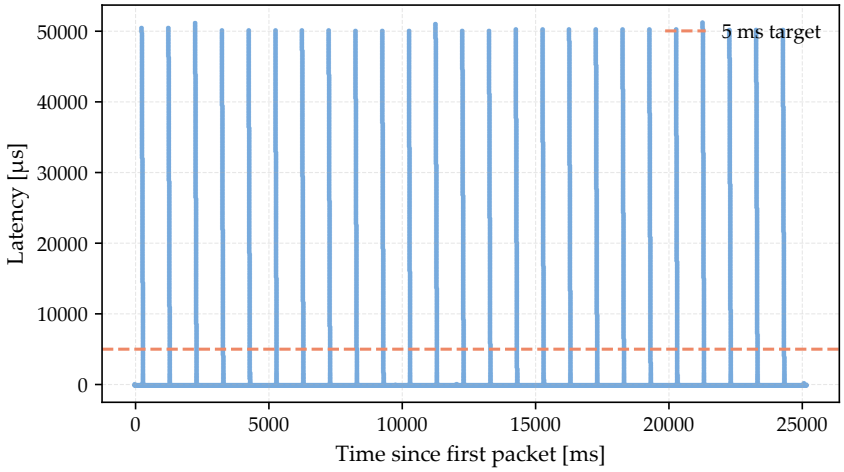
Condition	**Median**	**p99 [µs]**	**p99.9**	**> 1 ms**
Generic, baseline	39.7–39.8	42.8–42.9	44.5–45.1	0
Generic, 95 % load	40.4–40.5	43.9–44.1	45.5–46.3	0
PREEMPT_RT, baseline	42.5–42.6	46.4–46.7	48.4–49.0	0
PREEMPT_RT, 95 % load	43.7–44.0	47.5–47.9	49.5–49.7	0

Figure 5.1: End-to-end latency for the generic and PREEMPT_RT guests with and without ordinary CPU load. The table summarizes the range across the three repeated runs for each condition. The plot depicts the ECDF up to the 99th percentile.

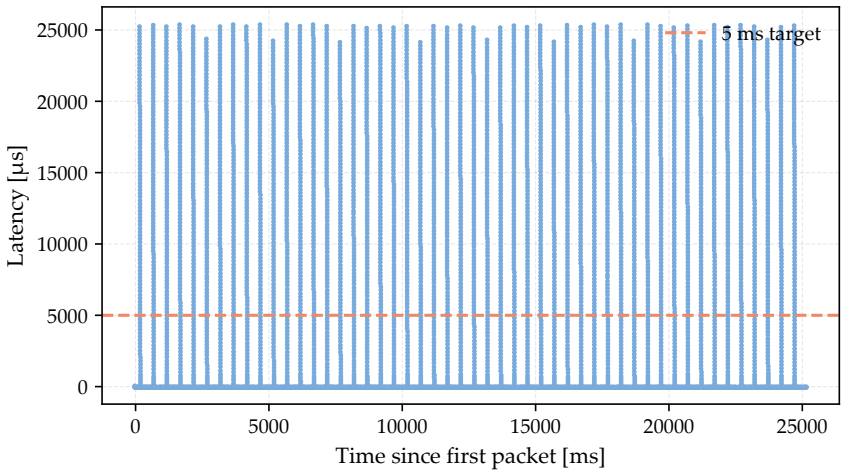


Condition	Received	Median	p99.9	> 5 ms
PREEMPT_RT, IRQ50, RT load	~84 %	90 ms–94 ms	313 ms–456 ms	94.9 %–95.2 %
PREEMPT_RT, IRQ98, RT load	100 %	41.6 µs	48.0 µs–48.9 µs	0 %
Generic, RT load	100 %	40.1 µs	45.4 µs–45.9 µs	0 %

Figure 5.2: Effect of network interrupt scheduling under competing real-time execution. The PREEMPT_RT IRQ50 configuration places critical network processing below the synthetic RT workload, causing severe delay and packet loss.



(a) **C2:** 50 ms fair runtime within a 1 s period.



(b) **C3:** 25 ms fair runtime within a 0.5 s period.

Condition	Median	p95	p99	Max.
C1	41.7 µs	44.0 µs	47.2 µs	49.5 µs
C2	41.8 µs	1.76 ms	40.6 ms	51.1 ms
C3	42.0 µs	0.90 ms	20.3 ms	25.4 ms
C4	41.9 µs	43.8 µs	47.1 µs	49.7 µs

Figure 5.3: Time-series latency plots showing the effect of the per-CPU fair-server when fair-scheduled and real-time workloads contend on the isolated RT vCPU. Halving both runtime and period preserves the 5 % fair bandwidth while approximately doubling disturbance frequency and halving the maximum disturbance duration.

5.2.4 Resulting Scheduling Configuration

Through the experiments, we identify CPU isolation and explicit scheduling relationships as the central mechanisms for maintaining stable latencies.

The generic guest kernel itself already achieves very low scheduler and end-to-end latency when a low-latency vCPU is properly isolated. PREEMPT_RT provides explicit control over the scheduling of IRQ handler threads and reduces the extent of non-preemptible kernel execution, yielding more stable and reliable wake-up latency tails under specific workload conditions, which is generally desirable for predictable real-time systems. However, if improper priorities are selected for a specific system, serious problems can arise. In our instance, it was the improper assignment of the critical network IRQ thread priorities, which led to severe delays and frame loss, which, for our system, is unacceptable. However, assigning proper priorities resolves the problems and yields the PREEMPT_RT benefits; therefore, the priority hierarchy and PREEMPT_RT are retained in the reference design.

Furthermore, the fair-server experiments show that ordinary guest activity should not share the dedicated low-latency RT vCPU. Under such a managed configuration, the default fair-server configuration has no negative effect on the latency distribution in our two-vCPU design. We have seen that if `SCHED_OTHER` tasks and real-time tasks are co-located, periodic latency spikes can occur, resulting in unpredictable latency interference. The final two-vCPU design therefore keeps housekeeping and ordinary workloads on vCPU0 and reserves vCPU1 exclusively for latency-critical applications.

Taken together, these results answer RQ2: The experiments show that stable execution on the reference configurations is primarily achieved through specific CPU isolation, separation of processing and scheduling classes, and a proper scheduling and priority hierarchy, with PREEMPT_RT yielding tail-stability benefits and enabling the explicit instantiation of the designed priority hierarchy. However, simply enabling PREEMPT_RT does not inherently yield lower baseline latency than the generic kernel and may, on the contrary, introduce significant problems if improperly configured; therefore, consideration of the complete system configuration is crucial.

5.3 Processing Costs and Virtualization Effects

> **RQ3:** What latency is contributed by representative protection processing, and how do virtualization events such as VM exits affect guest timing?

After evaluating several scheduling mechanisms that affect measured end-to-end latencies, this section examines two potential contributors within the protection path.

First, we quantify the computational cost of the implemented representative hot-path measurement-preprocessing and protection functions. Second, we investigate the effect of VM exits on guest timing. Together, these experiments distinguish the cost of protection processing and virtualization-induced delays.

5.3.1 Representative vPAC Processing Costs

In the interest of quantifying what fraction of the measured end-to-end latency is associated with the representative vPAC computation, the main computation stages were benchmarked independently. This will also provide insight into how much total potential computation time will remain for other tasks on the same vCPU.

This is directly resulting from the fact that protection-relevant measurement samples arrive at a constant rate of 4,000 samples per second, as described in Subsection 2.1.1, i.e., one frame every 250 µs. This 250 µs interval should not be seen as a strict per-frame execution-time deadline. Individual executions may exceed an input period without necessarily causing overall persistent delays if sufficient capacity remains in subsequent input periods. However, if the average processing demand approaches or exceeds the inter-frame arrival time of 250 µs, the protection application would be unable to keep pace with the incoming data stream, causing delays to accumulate.

The benchmarks were executed in the two-vCPU PREEMPT_RT VM on the dedicated vCPU1, scheduled using `SCHED_FIFO` priority 95. This environment was chosen to minimize potential unrelated interference.

The benchmarking methodology and implementation were introduced in Section 4.5. The primary protection measurements use 20,000 function invocations per timed batch ($N_{\text{batch,p}} = 20000$) for 40,000 timed batches per protection function ($N_{\text{samples,p}} = 40000$). The measurement preprocessing benchmark uses $N_{\text{batch,m}} = 2000$ and $N_{\text{samples,m}} = 30000$. Each primary experiment was repeated three times, and to verify that results are not an artifact of the selected batch size, additional verification measurements were performed with the batch sizes reduced by a factor of ten, i.e., $N_{\text{batch,p}} = 2000$ and $N_{\text{batch,m}} = 200$.

For batch i, the average execution cost per invocation of the benchmarked function is estimated as

$$t_{\text{call},i} = \frac{t_{\text{batch},i}}{N_{\text{batch}}}.$$

The mean values obtained from the benchmarks are summarized in Table 5.4. The reported primary mean is the average of the three repeated individual measurements. The validation column reports the corresponding benchmark results using a tenfold smaller batch size.

Processing element	Primary mean [µs/call]	Validation mean [µs/call]
Measurement processing	1.0372	1.0370
ANSI 50	0.0030	0.0030
ANSI 51 DTL	0.0104	0.0104
ANSI 51 IDMT	0.0615	0.0614
ANSI 67	0.0158	0.0157
ANSI 21	0.0162	0.0162
ANSI 87	0.0476	0.0477
Protection functions combined	**0.1545**	–
Measurement + protection	**1.1917**	–

Table 5.4: Mean execution costs of the representative measurement and protection processing. Primary values are averaged across three repeated measurements. Validation measurements use a tenfold smaller batch size.

The six implemented protection functions require approximately 0.155 µs on average per frame when their independent costs are summed. Furthermore, not all protection functions require the same computation time; ANSI 51 IDMT is the most expensive, and ANSI 50 the least expensive in our implementation. In general, the measurements show that the protection decision logic itself is computationally inexpensive.

In comparison, the measurement preprocessing computation requires approximately 1.037 µs per invocation on average. It therefore accounts for approximately 87 % of the estimated sequential measurement and protection processing. This is consistent with the implementation; the measurement stage performs an 80-sample phasor calculation, whilst the resulting quantities are consumed by comparatively small protection kernels.

Summing the independently measured mean costs yields an estimated average processing demand of approximately:

$$1.0372\,\mu s + 0.1545\,\mu s = 1.1917\,\mu s$$

As mentioned, the data stream uses a 250 µs period; the measured average computation costs therefore demand only approximately

$$\frac{1.1917}{250} \approx 0.48\,\%$$

of this interval. The measured mean computation cost is more than two orders of magnitude below this throughput constraint, thereby providing substantial headroom.

The reduced-batch verification measurements further support batching. Even though the number of invocations was reduced by a factor of ten, the measured means do not change significantly. The three primary executions similarly produced nearly identical mean execution times. Inspection of the time series shows no increase or time-dependent drift in execution cost. Smaller batches only expose more short-term variation and for several functions distinct execution-time levels, but do not change the estimated mean cost.

However, the batch measurement still imposes an important, noteworthy limitation on the interpretation of outliers. A long duration observed for one batch does not indicate whether the delay stems from a single function invocation or was distributed across several invocations. Consequently, dividing a batch maximum or high percentile by the batch size does not yield an observed worst-case execution time for an individual function. Thus, we primarily use the benchmark results to quantify the mean computational cost of our representative protection implementation.

Consequently, we establish that the estimated protection processing time of approximately 1.2 µs is relatively small compared with the end-to-end latencies of roughly 40 µs to 50 µs observed in our system configurations. Thus, the measurement and protection computation is not a dominant contributor to the measured latencies. Therefore, the following section examines virtualization-induced timing effects, particularly the influence of VM exits on guest execution.

5.3.2 VM-Exit Effects

As the previous subsection showed, the computational cost of the protection processing itself is comparatively small compared with observed end-to-end latencies. The second part of RQ3 is therefore concerned with a different source of delay: guest vCPU execution interruption by virtualization activity. A vCPU can exit guest execution for many reasons, but the frequency of such VM exits does not directly correspond to the latencies experienced by the protection application. Therefore, the objective of this experiment is to investigate the interference visible to high-priority guest applications produced by different classes of VM-exit-generating workloads. Furthermore, we investigate both one-vCPU and two-vCPU topologies to determine whether the two-vCPU architecture can isolate such activity from the dedicated RT vCPU.

We combine two measurements, one on the host and one inside the guest. On the host, `perf stat` counts the `kvm:kvm_exit` and `kvm:kvm_userspace_exit` tracepoints. Using these counters, we can quantify total KVM exit activity while distinguishing between the types of VM exits: kernel exits to KVM and userspace exits to QEMU. In

the guest, `cyclictest` measures the experienced wake-up latency using a `SCHED_FIFO` thread at priority 95, matching our protection application scheduling configuration. The thread is memory-locked and uses a wake-up period of 1 ms for a total experiment duration of 120 s.

The synthetic stress is generated via `stress-ng` scheduled with `SCHED_OTHER`. Fair-scheduling, as investigated in Subsection 5.2.3, is enabled with the default configuration and does not interfere with these experiments, because the `cyclictest` RT workload never exceeds the allowed RT quota.

The 1 ms activation interval for `cyclictest` was selected, because reducing the period would substantially increase virtualization activity generated by `cyclictest` itself. Reducing the interval from 1 ms to 100 µs increased measured VM-exit activity from approximately 4,000 exits per second to around 19,000 exits per second, whilst the average cyclictest latency remained at approximately 3 µs. The selected interval still produces around 120,000 wake-up samples per run while limiting the effects introduced by the measurement itself.

We deliberately exclude detailed VM exit to VM entry timing with `perf kvm stat record` from the final measurements, as previous validation showed that the recording process can itself be scheduled on isolated host CPUs and introduce measurement-induced delays. Particularly, VM exits due to the `EXTERNAL_INTERRUPT` reason were found to not be present anymore in runs without this detailed tracing, whilst remaining prevalent in runs with that tracing. In the final measurements, the `perf` process is restricted to the host housekeeping CPUs, whilst the mentioned tracepoints are counted on the pCPUs executing the guest vCPUs. The resulting data quantify the VM exit type, frequency, and latency experienced by `cyclictest`. However, we do not claim that this latency represents the duration of individual VM exits.

In the one-vCPU topology, both `cyclictest` and `stress-ng` execute on vCPU0, pinned to host pCPU10. In the two-vCPU topology, `cyclictest` is placed on the RT vCPU1/pCPU10, and `stress-ng` on vCPU0/pCPU9. Some stressors are additionally moved to the RT vCPU1 in separate control measurements. The QEMU emulator remains pinned to pCPU11 (cf. Chapter 4).

The evaluated workload classes are summarized in Table 5.5. CPU and memory stress simulate non-exit-specific workloads. Disk activity exercises the configured virtual disk-I/O path. CPUID is a deliberate experiment of a high-frequency VM exit stream. Port I/O exercises legacy userspace-emulated I/O and serves as a deliberately pathological contrast workload compared to representative vPAC operation.

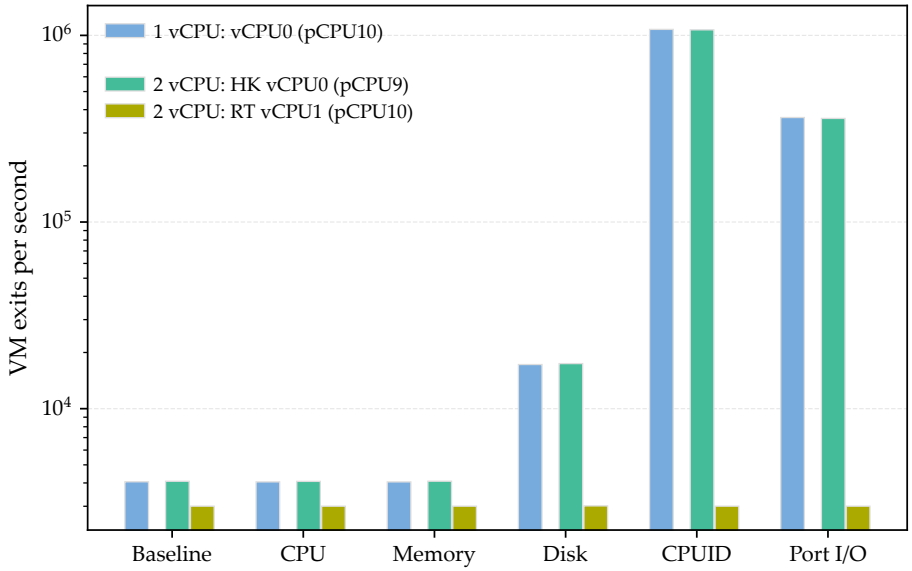
Workload	`stress-ng` mechanism	Purpose	Two-vCPU placement
Baseline	—	Reference virtualization activity without stressor	—
CPU	`--cpu`	CPU-utilization control without deliberately generating specific VM-exits	vCPU0
Memory	`--vm`	Guest memory and page-touch activity	vCPU0
Disk	`--hdd`	Virtual block-I/O activity using direct and synchronized I/O	vCPU0
CPUID	`--x86cpuid`	Synthetic high-frequency VM-exit workload	vCPU0; additional vCPU1 control
Port I/O	`--ioport`	Synthetic legacy port-I/O workload exercising a QEMU-userspace-emulated path	vCPU0; additional vCPU1 control

Table 5.5: Workloads used to characterize VM-exit activity and guest-visible scheduling effects. The two-vCPU measurements additionally execute CPUID and port I/O on RT vCPU1 as placement controls.

Exit Frequency and vCPU Placement

Figure 5.4 compares the VM exit rates generated by the six workload classes in exits per second. For each workload class, the one-vCPU configuration is shown alongside both pCPUs of the two-vCPU configuration.

In both topologies, the produced exit rates on the loaded vCPUs are nearly identical, generating comparable virtualization activity. CPU and memory workloads produce exit rates close to the baseline at around 4,000 exits per second. Disk activity increases that rate to approximately 17,200 exits per second. CPUID produces the highest exit frequency, at approximately 1,080,000 exits per second. Finally, port I/O produces approximately 360,000 exits per second.



	QEMU userspace exit share					
Workload placement	**Baseline**	**CPU**	**Memory**	**Disk**	**CPUID**	**Port I/O**
1 vCPU: vCPU0	0 %	0 %	0 %	0 %	0 %	98.88 %
2 vCPU: housekeeping vCPU0	0 %	0 %	0 %	0 %	0 %	98.86 %

Figure 5.4: VM-exit rates for different workloads. The logarithmic axis shows that workload-generated VM exit activity remains concentrated on the stressed vCPU. The table reports the fraction of exits on the stressed vCPU that return to userspace QEMU. Port-I/O values aggregate the corresponding repeated host-count runs.

The difference between port I/O and the other workloads is important, since none of the other workloads produce VM exits that return to the QEMU userspace except for port I/O. Approximately 98.9 % of all port-I/O VM exits are userspace QEMU exits, therefore exercising substantially different virtualization paths than the other stressors.

Importantly, for the two-vCPU topology, the exit rate on the non-stressed RT vCPU1 remains at approximately 3,000 exits per second throughout all workloads examined. Even the CPUID workload, which increases housekeeping vCPU exits by orders of magnitude, does not produce a corresponding increase on the RT vCPU.

The placement control experiments depicted in Figure 5.5 confirm that this behavior follows the guest vCPU which is executing the VM-exit generating workload. This confirms that the two-vCPU topology enables this VM-exit-placement capability, in

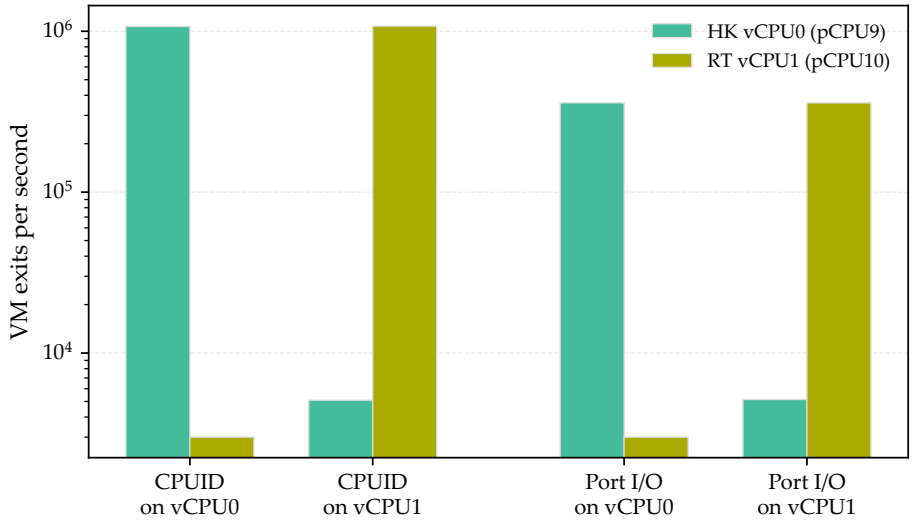


Figure 5.5: Placement of VM-exit activity in the two-vCPU configuration. Moving CPUID or port-I/O stress from housekeeping to RT vCPUs moves the primary exit stream from host pCPU9 to pCPU10. This result confirms that the large workload-generated exit activity can be kept off the RT vCPU through explicit workload placement.

contrast to the one-vCPU topology. However, exit placement alone does not determine changes in the experienced wake-up latency of RT guest workloads. Thus, the next subsection concerns the evaluation of the observed `cyclictest` latencies.

Guest-Visible Scheduling Effects

The host-side measurements establish how much and what type of virtualization activity occurs, and where it occurs. The relevant question for the protection workload, however, is whether this virtualization activity adversely affects the protection application and results in delayed execution within it. Therefore, the guest-side `cyclictest` measurements are performed. Table 5.6 summarizes the corresponding measurements. Again, port I/O rows pool all available repetitions of the measurement. Repetitions were performed specifically for the port I/O workload to capture more rare, high-latency tail samples. The remaining conditions each contain one 120 s run.

The CPUID results provide the clearest evidence that VM exit frequency alone is not a sufficient predictor of RT interference. In the one-vCPU guest, more than one million exits per second occur on the same vCPU as `cyclictest`, yet the observed maximum wake-up latency is only 8 µs, with the mean wake-up latency being 2.35 µs. A very high

Configuration	Workload	Samples	Mean	Median	p99.9
One-vCPU configuration: stressor and cyclictest on vCPU0					
Baseline	Baseline	120,000	2.00 µs	2 µs	3 µs
Shared vCPU	CPU	120,000	2.01 µs	2 µs	3 µs
Shared vCPU	Memory	120,000	2.77 µs	2 µs	7 µs
Shared vCPU	Disk	120,000	3.00 µs	3 µs	5 µs
Shared vCPU	CPUID	120,000	2.35 µs	2 µs	3 µs
Shared vCPU	Port I/O	839,999	2.99 µs	3 µs	4 µs
Two-vCPU configuration: cyclictest on RT vCPU1					
Baseline	Baseline	120,000	2.01 µs	2 µs	4 µs
Stressor on vCPU0	CPU	120,000	2.01 µs	2 µs	4 µs
Stressor on vCPU0	Memory	120,000	2.34 µs	2 µs	5 µs
Stressor on vCPU0	Disk	120,000	2.18 µs	2 µs	5 µs
Stressor on vCPU0	CPUID	120,000	2.08 µs	2 µs	4 µs
Stressor on vCPU0	Port I/O	360,000	2.37 µs	2 µs	4 µs
Stressor on vCPU1	CPUID	120,000	4.56 µs	5 µs	7 µs
Stressor on vCPU1	Port I/O	719,984	5.20 µs	5 µs	7 µs

Table 5.6: Guest-visible `cyclictest` wake-up latency under the different observed workloads. In the two-vCPU VM stressors execute on vCPU0 and `cyclictest` on vCPU1. CPUID and port I/O stressors are additionally placed on vCPU1. Port I/O statistics pool the corresponding individual repeated guest measurements.

exit frequency can therefore coexist with low guest-visible scheduling latency, if the exit path is short.

The two-vCPU results additionally display the effect of workload placement. CPUID on vCPU0 records a mean of 2.08 µs and a 99.9th percentile of 4 µs. Moving the same workload to vCPU1 increases the mean observed wake-up latency of `cyclictest` to 4.56 µs and the 99.9th percentile to 7 µs. The median increases from 2 µs to 5 µs. These results, therefore, show that exit placement has a measurable common-case effect; however, it does not produce a substantial latency tail.

CPU, memory, and disk activity show similarly low guest-visible latency. Notably, disk stress increases the exit rate by approximately a factor of four relative to the baseline, yet latencies remain comparatively small. Across both topologies and all workload groups, excluding port I/O, the observed maximum latency was 13 µs. Thus,

the results distinguish CPU utilization and VM exit frequency from their actual timing implications: neither is sufficient to induce poor RT behavior.

Similar to the CPUID workload, which creates a placement-dependent latency effect, the port I/O workload exhibits this behavior as well. Nevertheless, the observed values remain small compared to the application deadline. However, the interesting aspect of the port I/O workload is that the common percentiles do not capture the complete port I/O behavior.

Legacy Port I/O and Rare Tail Behavior

Port I/O is the only workload in the final experiments that results in substantially longer wake-up delays. Table 5.7 displays a comparison between port I/O and the other observed workloads with respect to observed rare tail-latencies in the form of a cumulative count table.

Condition	**Samples**	$> 20\,\mu s$	$> 50\,\mu s$	$> 100\,\mu s$	$> 200\,\mu s$	$> 500\,\mu s$	$> 1\,ms$	$> 5\,ms$	**Max.**
All non-port-I/O workloads	1,320,000	0	0	0	0	0	0	0	13 µs
Port I/O, 1 vCPU	839,999	9	7	6	5	4	1	0	1.299 ms
Port I/O, 2 vCPU, stressor on vCPU0	360,000	6	6	4	2	0	0	0	310 µs
Port I/O, 2 vCPU, stressor on vCPU1	719,984	4	3	3	2	1	1	1	16.995 ms

Table 5.7: Cumulative counts of elevated `cyclictest` wake-up latencies. Port-I/O conditions contain additional repetitions because rare events were sampled more extensively; the counts, therefore, characterize only the observed tails and must not be interpreted as a comparison of tail probabilities between configurations.

Across 1,320,000 wake-up measurements from all other non-port-I/O workloads, no latency above 13 µs is observed. Port I/O is the only evaluated workload class to produce observations above 50 µs, even above 1 ms.

In the one-vCPU configuration, seven observations exceed a latency of 50 µs across 839,999 samples. Four exceed 500 µs and the largest observed value is 1.299 ms.

In the two-vCPU configuration with port I/O placed on the housekeeping vCPU0, six of 360,000 measurements exceed 50 µs, and no observations exceed 500 µs, with a maximum observed latency of 310 µs. It might seem that the second vCPU, therefore, substantially reduces the largest delays observed in this particular fresh measurement set. However, these high-latency observations are very rare; thus, this result may simply

be an artifact of the small sample size and should not be interpreted as a result of the topology and task pinning.

The most extreme observation occurs when port I/O executes on the RT vCPU1 in the two-vCPU topology. Three of 719,984 measurements exceed 100 µs, with one reaching a latency maximum of 16.995 ms. The latter even exceeds the latency protection target of 5 ms. Furthermore, this maximum was observed in a guest-only repetition, without the simultaneous host-measurement `perf stat` counterpart. The elevated sample, therefore, cannot be attributed solely to the lightweight host counter used in the main measurements.

The port I/O workload is deliberately pathological, not representative of our established vPAC processing. It repeatedly exercises a legacy I/O path for which KVM returns execution to QEMU userspace. The experiment's value lies in serving as a stress case, showing that the type of VM exit matters more than the exit count and frequency alone. CPUID produces approximately three times as many exits per second without a comparable tail.

The measurements do not identify the exact internal mechanisms responsible for these rare delays. The wake-up latency can include several mechanisms before the RT task can continue executing, such as multiple virtualization events, host scheduling, QEMU device processing, or other shared-state synchronization in the host or guest. Isolating these components would require detailed tracing of the host, the QEMU execution path, and the guest, which, by itself, can perturb the measurements, as we have seen with `perf kvm stat record`.

For the operational configuration, the relevant conclusion is consequently to avoid unnecessary legacy userspace-emulated I/O on the latency-critical path, which our reference protection workload does not require. Furthermore, a minimized, paravirtualized device configuration as described in Chapter 4 is to be maintained. The two-vCPU topology also permits assigning ordinary exit-generating workloads to the additional housekeeping vCPU0, thereby further isolating the RT vCPU. This does not inherently constitute a formal guarantee against all interference mechanisms, but the corresponding CPU, memory, disk, and CPUID results demonstrate effective isolation of the RT vCPU1 from VM exits. However, the actual experienced latency was very similar in both topologies.

Overall, the experiments show that VM exit frequency and count are not, by themselves, adequate measures of virtualization-induced RT interference. CPUID issuing more than one million VM exits per second still retains low-microsecond wake-up latencies, whilst legacy port I/O workloads introduce an unpredictable high-latency tail. Both topologies exhibit very similar latencies, whilst the benefit of the two-vCPU configuration is the ability to assign specific workloads to specific vCPU processing classes. This result complements the topology trade-off evaluated in the following

section (cf. Section 5.4): the additional vCPU primarily provides isolation flexibility.

5.4 One-vCPU vs. Two-vCPU Topologies

> **RQ4:** What are the trade-offs between one-vCPU and two-vCPU VM configurations?

The previous experiments showed that low latency does not result solely from the number of guest vCPUs. Multiple factors influence whether the latency-critical path can execute without interference, including CPU isolation, the priority hierarchy, and interactions between real-time and fair-scheduled work. The previously detailed one-vCPU and two-vCPU configurations differ fundamentally in how these execution classes are arranged.

Whilst the one-vCPU topology minimizes the CPU resources allocated to each VM, it necessarily places all guest execution onto that one scheduler run queue. This execution includes the real-time protection processing, interrupt handling, kernel execution, ordinary guest housekeeping, and any auxiliary work. The two-vCPU topology instead separates ordinary and latency-critical workloads onto distinct vCPUs. This section is therefore concerned with evaluating the two topologies with respect to their end-to-end latency, scheduling-class separation, and behavior under load.

We first revisit the fair-scheduling mechanism investigated in Subsection 5.2.3, particularly in the context of the one-vCPU topology. Based on the results, we select latency-oriented configurations for each topology and evaluate them under increasing but bounded real-time utilization. At last, we intentionally investigate unbounded RT workloads to detail starvation and failure containment in both topologies.

5.4.1 Co-location of Real-Time and Fair-Scheduled Work

Subsection 5.2.3 demonstrated that fair-server intervention can introduce large periodic latency spikes when highly utilized RT work and runnable fair-scheduled work share the same CPU. In the experiments, we deliberately introduced such workloads onto the isolated RT vCPU of the two-vCPU guest in order to investigate the mechanism. On the one-vCPU topology, however, this interaction becomes more relevant because fair-scheduled housekeeping and auxiliary work necessarily share the CPU with real-time work.

Three configurations were therefore evaluated, as summarized in Table 5.8. The workload was generated via `stress-ng`. The RT workload executes at `SCHED_FIFO` priority 90 and consumes approximately 95 % CPU time, while an additional `SCHED_OTHER` workload requests 10 % CPU time, simulating an auxiliary runnable workload. The

known priority hierarchy for latency-critical network IRQ handlers and the protection application remains intact. The results are depicted in Figure 5.6.

Condition	vCPUs	Fair server	Workload	Placement
F1	1	50 ms / 1 s	95 % RT + 10 % fair	Both workloads on vCPU0
F2	1	Disabled	95 % RT + 10 % fair	Both workloads on vCPU0
F3	2	50 ms / 1 s	95 % RT + 10 % fair	RT on vCPU1, fair work on vCPU0

Table 5.8: Conditions used to evaluate co-location of real-time and fair-scheduled work. F2 removes the fair-server reservation, whereas F3 avoids the interaction by separating the scheduling classes onto distinct vCPUs.

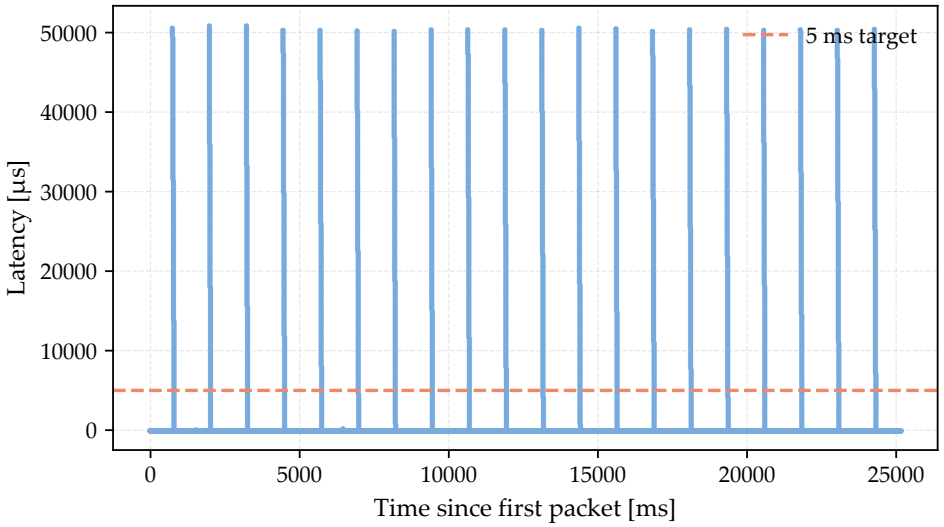
F1 With the default fair server enabled in F1, the periodic latency spikes identified in Subsection 5.2.3 appear again. A very similar latency time series plot appears again, with latencies spiking to 50 ms approximately every second.

F2 Disabling the fair server in F2 removes this periodic behavior and stops real-time throttling via the fair server, even though both workloads are retained on the single vCPU. Across the three pooled runs, the median is 40.63 µs and the 99.9th percentile is 47.08 µs, with no observation exceeding 1 ms.

F3 The two-vCPU configuration offers an alternative solution. In F3, the RT work remains on vCPU1, while the fair-scheduled work is placed on the housekeeping vCPU0, as intended by the topology. The default fair server can therefore remain enabled. Both workloads can coexist within the VM, and the resulting latency distribution remains similarly stable, with a median of 41.22 µs and a 99.9th percentile of 48.56 µs. Again, no latency observation exceeded 1 ms.

Results

The comparison exposes an important difference between the topologies. In a one-vCPU guest, workload-classes cannot be separated onto distinct vCPUs. Disabling the fair server is therefore an effective way of protecting the latency-critical real-time workload from this particular interference mechanism. This configuration is used for the latency-oriented one-vCPU experiments below.



Condition	Median	p99	p99.9	Max.
F1: 1 vCPU, fair server	40.74 µs	38.25 ms	49.28 ms	50.83 ms
F2: 1 vCPU, fair server disabled	40.63 µs	44.92 µs	47.08 µs	94.71 µs
F3: 2 vCPU, separated	41.22 µs	45.52 µs	48.56 µs	203.24 µs

Figure 5.6: Effect of co-locating high RT utilization and fair-scheduled work. F1 shows the known periodic latency pattern with the default fair server. F2 suppresses this mechanism by disabling the fair server, while F3 avoids it by separating fair and RT execution across two vCPUs. F1 details the results from the first run of 100,000 frames, F2 and F3 pool three runs of 100,000 frames each.

This configuration also involves a trade-off. The fair server exists to prevent RT tasks from monopolizing a CPU and allow fair-scheduled tasks to execute as well. Disabling it, therefore, removes a safety mechanism for guest execution and is appropriate only if the RT workload is properly bounded and controlled.

The two-vCPU topology avoids this trade-off via intent. Fair-scheduled tasks remain on vCPU0, while the latency-critical RT path executes on vCPU1, allowing the fair server to remain enabled without it interfering with latency-critical processing. However, as demonstrated in Subsection 5.2.3, the configuration and placement of tasks within the VM still require careful attention, since fair work assigned to the RT-dedicated vCPU can reproduce the same mechanism. The advantage is therefore the ability to deliberately prevent co-location.

Condition	vCPUs	RT load	Fair server on RT CPU	RT workload placement
T1	1	0 %	Disabled	vCPU0
T2	1	80 %	Disabled	vCPU0
T3	1	90 %	Disabled	vCPU0
T4	1	95 %	Disabled	vCPU0
T5	2	0 %	Default	vCPU1
T6	2	80 %	Default	vCPU1
T7	2	90 %	Default	vCPU1
T8	2	95 %	Default	vCPU1

Table 5.9: Conditions used for comparison of both topologies under bounded auxiliary RT load

5.4.2 End-to-End Latency under Bounded RT Load

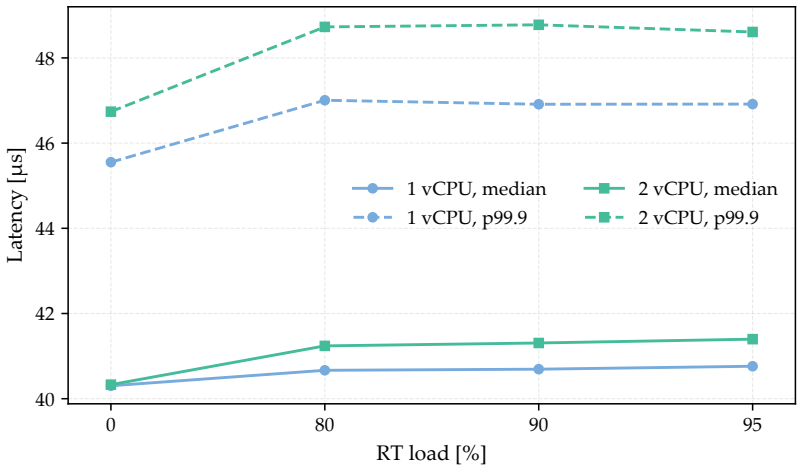
Having now established a latency-oriented configuration for each topology, the goal of the next experiment is to evaluate whether either configuration degrades as auxiliary RT utilization approaches higher levels, simulating computationally demanding real-time workloads. In the following experiments, no fair-scheduled CPU work is introduced. In addition to a baseline, a `SCHED_FIFO` `stress-ng` workload at priority 90 is placed on the RT vCPU at utilizations of 80 %, 90 %, and 95 %.

The experiment conditions are summarized in Table 5.9. The fair server is disabled in the one-vCPU guest, while it remains enabled in the two-vCPU guest.

Each condition consists of three runs of 100,000 frames. All three runs are pooled into one 300,000-sample empirical distribution. Figure 5.7 shows the median and 99.9th percentile latencies as RT load increases. Both configurations remain stable throughout the tested range, receiving and responding to every frame, with no frame latency exceeding 1 ms. The median and 99.9th percentile do not significantly degrade in either configuration.

Results

Therefore, the measurements do not display any capacity-related advantage for either topology within the tested bounds. In particular, the one-vCPU guest does not develop any increasing queueing delays or packet loss as RT load approaches 95 %. Once configured to avoid the fair-server interaction, the implementation remains viable for our representative workload.



	1 vCPU		2 vCPU	
RT load	**Median**	**p99.9**	**Median**	**p99.9**
	[µs]			
0 %	40.304	45.552	40.330	46.738
80 %	40.667	47.006	41.240	48.728
90 %	40.693	46.912	41.308	48.777
95 %	40.762	46.916	41.398	48.609

Figure 5.7: End-to-end latency under increasing bounded RT load. Solid and dashed curves show median and 99.9th percentiles, respectively, for each topology. Markers denote measured load levels; connecting lines are only visual guides and do not imply measurements at intermediate loads. Each point pools three runs of 100,000 frames. No packet loss or latency above 1 ms was observed in any run.

Likewise, though, the additional housekeeping vCPU in the two-vCPU topology does not provide any significant latency advantage in this experiment. The small differences between the two distributions are on the order of one microsecond and do not justify choosing one topology over the other based on the latency observed in this experiment. The more relevant result is that both configurations remain stable throughout the tested range.

The purpose and benefit of the second vCPU must therefore be evaluated primarily in terms of the use-case, separation capability, and robustness rather than increased computational capacity based on these experiments.

5.4.3 Overload, Priority Starvation, and Failure Containment

The bounded measurements deliberately stop at 95 % RT load. For `stress-ng` CPU loads below 100 %, the worker alternates between computation and idle phases in order to approximate the requested load[4]. These idle periods provide opportunities for ordinary workloads and, more importantly, lower-priority real-time tasks to make progress.

A continuously running fixed-priority real-time task, therefore, represents a different condition. Hence, three experiments were conducted using a 100 % CPU-load `SCHED_FIFO` priority-90 `stress-ng` worker to characterize the consequences of faulty or unbounded RT tasks and to assess the extent to which each topology can handle the resulting starvation of other processes. Table 5.10 summarizes the evaluated experiment conditions and qualitative results.

Condition	vCPUs	Continuous RT workload	Protection	Existing SSH	New SSH
O1	1	FIFO90 on vCPU0	Works	No	No
O2	2	FIFO90 on RT vCPU1	Works	Yes	Yes
O3	2	FIFO90 on housekeeping vCPU0	Works	No	No
D1	1	FIFO40 on vCPU0	Works	Yes	No

Table 5.10: Evaluated experimental conditions and qualitative behavior under deliberate 100 % RT workloads. O1–O3 use priority 90. D1 lowers the priority below the normal IRQ-thread priority to investigate the starvation mechanism.

O1 In O1, the 100 % RT workload is executed on the only vCPU in the one-vCPU guest. The latency-critical packet processing remains properly functional. This follows from the configured hierarchy: critical network IRQ threads and the protection application itself run at higher real-time priorities than the `stress-ng` worker. The guest, therefore, continues to properly receive, process, and return protection frames.

However, the guest becomes operationally unmanageable. Existing Secure Shell (SSH) sessions stop responding, and new connections cannot be established. Although the `stress-ng` worker was started with a timeout, it is never terminated after the timeout has passed, instead resuming the 100 % CPU-utilizing worker thread indefinitely. This behavior is not a system failure, but a consequence of the topology and our

[4] `https://manpages.debian.org/bullseye/stress-ng/stress-ng.1.en.html#l`

configuration. Since all guest execution shares vCPU0, the one-vCPU topology provides no separate execution context from which the overloaded vCPU can be managed.

O2 The O2 experiment demonstrates the fault containment enabled by allocating a dedicated vCPU to RT processing in the two-vCPU topology. The `stress-ng` worker is now placed on the RT vCPU1, but guest housekeeping and management remain on vCPU0. Similar to O1, protection processing continues proper operation. The main difference is that existing SSH connections, the establishment of new SSH connections, and workload termination and management all remain functional. Therefore, a saturation of the real-time environment does not remove guest management capacity.

O3 In O3, the complementary control experiment is provided. The `stress-ng` workload is now executed on the housekeeping vCPU0. Again, the protection processing continues unhindered execution. However, the system's qualitative control resembles the state of O1: On the housekeeping side, SSH connections, both new and established, do not function, and `stress-ng` does not terminate after the timeout expires. Therefore, simply using the two-vCPU topology does not ensure the safety of arbitrary RT scheduling configurations. The benefit lies in the topology providing dedicated, separate execution domains, so the effects of overloaded workloads can be contained by their placement.

D1 A further diagnostic experiment, D1, lowers the priority of the `stress-ng` worker thread from 90 to 40. This priority is deliberately less than the default priority of IRQ threads in a PREEMPT_RT system. Lowering the priority below this level restored functionality of existing network connections and allowed this workload to be manually interrupted via an existing SSH session. This supports the interpretation that the loss of connectivity via existing SSH connections results from starvation of lower-priority interrupt processing.

However, the priority 40 workload resumes failing to terminate within its configured timeout period, and new SSH connections cannot be established. This may be due to timer-related RT execution in PREEMPT_RT systems, which runs at an even lower priority. Particularly, on a PREEMPT_RT kernel, timers are executed within the `ktimersd` thread, which runs at the lowest possible real-time priority 1 [13]. A continuously runnable FIFO40 task can therefore permit higher-priority IRQ processing, enabling SSH access, whilst still preventing lower-priority kernel and userspace progress. Therefore, to ensure overall guest liveness, simply preserving network IRQ execution is insufficient, and tasks and scheduling priorities must be managed carefully.

These overload experiments furthermore also show why the 95 % conditions in

Subsection 5.4.2 remain usable. The bounded RT load periodically yields execution, enabling lower-priority tasks to run and progress. A permanently runnable task, such as the 100 % `stress-ng` worker in this set of experiments, does not provide these opportunities. Consequently, administration of RT workloads must also consider whether RT utilization is bounded.

5.4.4 Topology Trade-off

The experiments in this section have revealed that the distinction between the two topologies is not latency-based and inherently obvious. When correctly configured, both topologies offer proper protection processing with zero packet loss and similar latency distributions. Both topologies are thus viable for the representative workload. Therefore, to answer RQ4, the trade-offs between the two configurations are detailed below.

The main distinction between the two topologies is a trade-off among resource consolidation, fault tolerance, and enhanced management control. The one-vCPU topology offers higher resource consolidation capability by requiring only one vCPU per VM. However, this resource efficiency is obtained at the cost of stronger scheduling assumptions. Namely, real-time workloads and ordinary and auxiliary housekeeping tasks must share a single processing unit. One consequence of this co-location is revealed by the fair-server experiments: retaining the fair-server can introduce large, potentially unacceptable latency. Disabling the fair-server removes this source of latency but, consequently, removes a safety guarantee for fair-scheduled tasks.

The one-vCPU topology must, thus, be treated as a consolidation-oriented design, in which scheduling priorities and RT workloads must be strictly controlled. It offers less protection against mistakes or faulty unbounded RT tasks.

On the contrary, the two-vCPU design consumes an additional vCPU but permits workload classes to be separated on a per-processing-unit basis. In our design, housekeeping and fair-scheduled tasks remain on vCPU0, whilst critical real-time tasks execute on the dedicated vCPU1. This allows the default fair-server behavior to be retained and preserves a management execution context via vCPU0 if the RT vCPU1 is overloaded.

The additional vCPU is therefore more of an isolation and fault-containment mechanism than a means of providing lower-latency guest-protection processing. Essentially, for consolidation-density-maximizing deployments, the one-vCPU topology is a viable option. If stronger separation of workload classes, stronger operational manageability, and tolerance of errors and faults are required, the two-vCPU topology provides a more robust configuration option. This is further complemented by the results of the VM exit exploration detailed in Subsection 5.3.2. The final end-to-end latency behavior of

both configurations is evaluated in Section 5.5. And the behavior of both configurations under multiple concurrently deployed VMs is investigated in Section 5.6.

5.5 Final End-to-End Latency

> **RQ1:** Can PREEMPT_RT-based KVM guests provide sufficiently low and stable end-to-end latency for representative vPAC workloads?

In the previous sections, we have investigated individual mechanisms that can affect latency and vCPU topological differences. The experiments were therefore designed to expose those specific mechanisms and generally used comparatively short measurement intervals. In this final experiment, however, we aim to evaluate our primary question: whether the systems we have implemented provide low, stable end-to-end latency during sustained operation.

Both VM topologies are included in the test. In Section 5.4, we showed that the additional vCPU in the two-vCPU topology is not to be interpreted as an inherent latency optimization; both topologies operate at a trade-off. Therefore, the final long-running experiment evaluates whether the previously observed low latencies are maintained by both topologies over longer measurement intervals.

5.5.1 Long-Run Measurement Method

For each topology, three independent runs are performed. Each run transmits 14.4 million frames, corresponding to approximately one hour of sustained operation. The complete experiment thus contains 86.4 million end-to-end measurements in total, with 43.2 million per topology. The evaluation path itself remains unchanged. No additional synthetic workloads or host tracing is active during these measurements.

Previous experiments retained each frame and its latency separately in a single CSV file; this would require writing large files containing tens of millions of individual samples. The long-run receiver was therefore adapted: Latencies below 1 ms are accumulated in 0.1 µs histogram bins, whilst the minimum, maximum, mean, and variance are calculated online during the measurement. Latencies of at least 100 µs, packet loss, and duplicate/reordering counters are also stored as exact exceptional events. All memory is bounded, preallocated, and pre-faulted; no dynamic allocation is necessary.

To evaluate whether performance potentially degrades over time, the receiver additionally maintains statistics for one-minute windows. These windows allow a distinction between system degradation and localized disturbances. All data is only written once

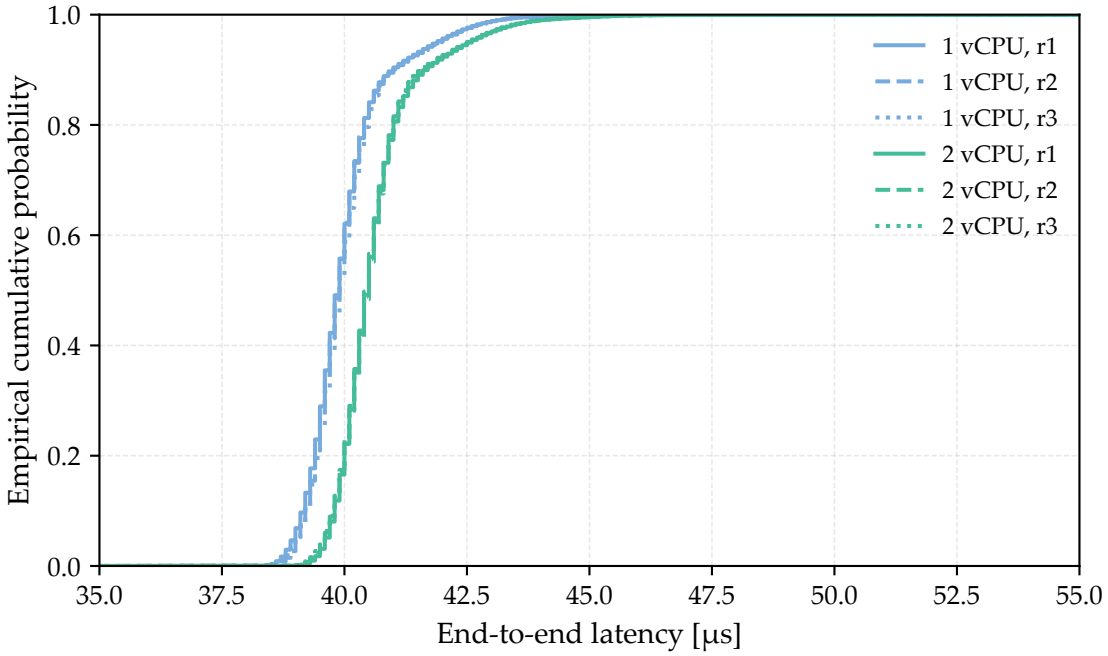


Figure 5.8: Empirical end-to-end latency distributions for three one-hour repetitions of each VM topology. Each run contains 14.4 million received responses. The displayed range focuses on the dense latency distribution; rare larger observations are reported separately in Table 5.11 and analyzed over time in Subsection 5.5.3.

the measurement is finished, to prevent recurring file or terminal I/O from interfering with the receive path.

5.5.2 Latency Distribution and Reproducibility

Figure 5.8 shows ECDFs for all six one-hour runs. The three curves belonging to the same topology are visually almost indistinguishable, indicating that the dense distributions are very reproducible. The corresponding numerical results are summarized in Table 5.11. Across all runs, all frames have been received, no duplicated or reordered responses, and no latency observation above our target 5 ms was observed.

The dense distributions remain close to those measured in previous shorter experiments. Across the one-vCPU runs, the mean ranges only from 39.980 µs to 40.020 µs, while the 99.9th percentile ranges from 45.1 µs to 45.5 µs. The two-vCPU results are similarly stable, retaining the 99.9th percentiles between 46.1 µs and 46.4 µs. The larger

number of latency samples exceeding 100 µs in the first runs of both topologies is primarily due to the localized transient discussed below. There is no persistent shift of the dense distribution, as can be seen in Figure 5.8. The slightly higher mean of the first run of the two-vCPU topology is also caused by this.

Configuration	Run	Mean	Median	p99.9	$\geq$ 100 µs	$\geq$ 1 ms	Max.
1 vCPU	r1	39.980 µs	39.9 µs	45.5 µs	203	111	3.076 ms
1 vCPU	r2	39.989 µs	39.9 µs	45.1 µs	41	0	387.503 µs
1 vCPU	r3	40.020 µs	39.9 µs	45.1 µs	38	0	625.275 µs
1 vCPU	pooled	39.996 µs	39.9 µs	45.2 µs	282	111	3.076 ms
2 vCPU	r1	40.791 µs	40.5 µs	46.4 µs	2108	1329	4.335 ms
2 vCPU	r2	40.572 µs	40.5 µs	46.2 µs	33	0	395.432 µs
2 vCPU	r3	40.584 µs	40.5 µs	46.1 µs	56	0	401.006 µs
2 vCPU	pooled	40.649 µs	40.5 µs	46.2 µs	2197	1329	4.335 ms

Table 5.11: Numerical long-running end-to-end latency results. Each individual run contains 14.4 million measurements; each pooled row therefore represents 43.2 million measurements. All runs completed without packet loss or duplicate/reordered responses. Tail counts are cumulative within each row.

Pooling the three runs for both topologies yields a median of 39.9 µs and 40.5 µs, and a 99.9th percentile latency of 45.2 µs and 46.2 µs for the one-vCPU and two-vCPU guests, respectively. The latency for both configurations firmly remains far below the protection timing requirement throughout the dense distribution.

5.5.3 Temporal Stability and Rare Transients

However, an ECDF alone cannot reveal whether rare elevated observations are distributed throughout the complete run or concentrated in a short interval. This distinction is, however, important for long-running measurements and their interpretation. Figure 5.9 displays the one-minute mean latency and the number of samples exceeding 100 µs throughout the runs.

Outside of both localized transient intervals, the minute-scale latency remains highly stable. There is no monotonic increase in latency with elapsed runtime during sustained execution.

One localized disturbance occurs in the first run of each topology. In the first runs, the number of samples exceeding 100 µs was 203 for the one-vCPU topology and 2108 for the two-vCPU topology. For one-vCPU, 138 of the 203 samples occur within an interval of approximately 1.35 s around 527 s after the start of the measurement. This interval

contains all 111 observations above 1 ms and reaches the observed maximum of 3.076 ms. For two-vCPU, the one-minute window observations place 1235 samples exceeding 100 µs in one window and 827 in the following window, leaving 46 observations throughout the rest of the run. In contrast, two-vCPU r2 and r3 contain only 33 and 56 observations exceeding 100 µs in total. Neither remaining runs of one-vCPU and two-vCPU contain millisecond-scale latency observations.

Within the localized r1 events, consecutive frame latencies frequently decrease by our configured send interval of approximately 250 µs. For example, one delayed response can be followed by successive frames, whose latencies decrease by approximately 250 µs each. This structure is consistent with a temporary backlog followed by queue draining. However, the available experiment arrangement does not determine where such a backlog formed.

Both timestamps are taken on *River*, so the end-to-end latencies include sender execution, sender host networking, the physical network path, *Christina's* networking and virtualization path, guest execution, the return path, and scheduling on *River*. The localized excursions can therefore not be attributed to one specific element in the chain without additional measurement instrumentation. The same queueing phenomenon is observed once, for a brief period, in each topology, and is not reproduced in any of the remaining runs or measurement times. Therefore, these disturbances are treated as rare end-to-end disturbances of unresolved origin, rather than a recurring latency mechanism that is topology-specific.

However, the brief disturbance events do not cause frame loss and remain below the protection timing target of 5 ms. Across all 86.4 million latency samples, the largest observed latency is 4.335 ms.

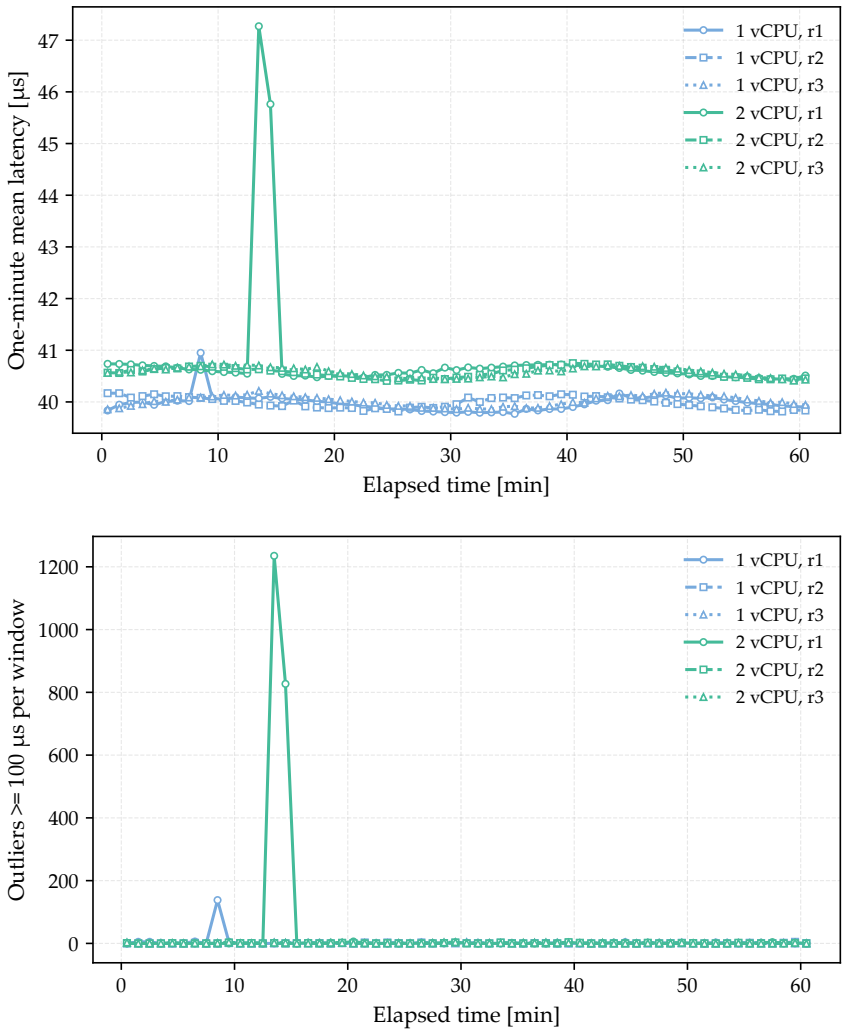


Figure 5.9: Time-related behavior of the six measurements. The upper panel shows mean latency in consecutive one-minute windows; the lower panel shows the number of observations exceeding 100 µs in each window. The measurements show no increase in latency over elapsed runtime. Elevated observations are instead concentrated in localized intervals in r1 of each topology.

5.5.4 Long-Run Findings and Implications

The long-running measurements provide empirical evidence that PREEMPT_RT-based KVM guests can maintain sufficiently low and stable end-to-end latency for the representative vPAC workload evaluated in this thesis. Across six runs, both VM topologies lose no frames and retain their reproducible dense latency distributions at approximately 40 µs to 46 µs. No progressive degradation over time is observed either.

However, the measurements also display why this conclusion must remain empirical rather than be interpreted as a hard real-time guarantee. Both VM topologies contain a short, millisecond-scale latency disturbance. Neither results in a violation of the 5 ms target; however, they demonstrate that rare end-to-end interference might not be fully captured by the dense percentiles alone. The experiment thus established neither a formal worst-case end-to-end latency nor an absolute upper latency bound.

Furthermore, the results of this experiment again support the topology trade-off established in Section 5.4. The one-vCPU configuration retains slightly lower percentile latencies, whereas the two-vCPU configuration provides additional flexibility. This experiment shows that both configurations can sustain the representative vPAC protection workload with low and stable end-to-end latency.

5.6 Multi-VM Scalability

> **RQ5:** Do the evaluated configurations remain viable when multiple IED VMs are deployed concurrently?

The preceding experiments primarily considered individual VMs and their behavior. However, the deployment model we investigate, i.e., one IED per VM (cf. Section 3.4), allows multiple IEDs to be consolidated on a single system by instantiating multiple VMs. This final section, therefore, evaluates whether the implemented reference protection workload remains viable when executed by several concurrent VMs.

Details regarding CPU placement, specific multi-VM changes to topologies, and changes to the measurement implementation are detailed in Subsection 4.1.3, Subsection 4.2.3, and Section 4.6.

5.6.1 Multi-VM Experiment Configuration

The final one-vCPU configuration is evaluated with 1, 2, 3, 4, 6, and 8 VMs simultaneously active. Each VM retains a dedicated pCPU backing its vCPU with QEMU and virtualization support work consolidated onto pCPU11 across VMs.

The final two-vCPU configuration is evaluated with 1, 2, 4, and 7 VMs simultaneously active. Similarly, the real-time vCPU1 is backed by a dedicated pCPU for each VM,

while the housekeeping vCPU0 threads and QEMU and virtualization support tasks are consolidated onto pCPU10 and pCPU11, respectively. The idle configuration, which allows the housekeeping vCPUs to be consolidated, is detailed in Subsection 4.1.3.

Each measurement run consists of 100,000 transmitted measurement frames. Every participating VM receives this frame and is expected to reply. The experiment records pooled and per-VM end-to-end latency, and missing responses.

To characterize how far apart responses arrive for the same measurement frame, we calculate the per-frame response spread. This is defined for frame i and the set of participating VMs V, as:

$$S_i = \max_{v \in V} T_{i,v} - \min_{v \in V} T_{i,v},$$

where $T_{i,v}$ denotes the measured end-to-end latency of the response from VM v. The spread, therefore, measures the difference in time between the first and last response of one emitted measurement frame.

Before the final consolidation placement was fixed, an exploratory one-vCPU configuration assigned both the vCPU and the virtualization processing of every VM to separate, dedicated pCPUs. The configuration behaved normally up to three concurrently responding VMs, however, at four responders, a high reproducible upper-tail appeared, but no responses were missing. When all four VMs were concurrently receiving the measurement frames, but only one VM was actively replying, the latencies of all VMs acting as the replier were similar. Reassigning CPU placement and increasing the receiver's scheduling priority on *River* did not explain this behavior.

The anomaly, therefore, required all VMs to reply concurrently, rather than solely being present in the broadcast delivery to the guests. Furthermore, the replies captured at the receiver were mostly in a repeatable arrival order, with the largest delays originating from VMs, which were later in that arrival order. This behavior is consistent with potential queueing or serialized processing at a shared component in the return path, possibly combined with batching or scheduling effects. The measurements do not identify the responsible components, and further optimizations in the packet delivery path and host networking are out of scope of this thesis (cf. Section 3.1).

Therefore, the final scalability feasibility evaluation focuses on the consolidated configurations described above.

5.6.2 Consolidated Multi-VM Results

The numerical results of the scaling experiments are summarized in Table 5.12. All expected responses were received for all conducted experiment runs in both topologies.

Topology	VMs	End-to-end latency			Response spread	
		Median	p99.9	Max. [µs]	Median	p99.9
1 vCPU	1	40.491	45.936	86.832	0	0
1 vCPU	2	45.388	58.941	73.008	12.214	15.712
1 vCPU	3	55.603	63.250	167.519	13.527	16.823
1 vCPU	4	61.841	74.441	171.952	14.829	25.960
1 vCPU	6	66.300	92.255	240.170	28.716	40.938
1 vCPU	8	75.634	110.765	340.922	38.070	53.123
2 vCPU	1	40.750	45.882	73.157	0	0
2 vCPU	2	46.295	59.689	131.690	12.224	15.578
2 vCPU	4	62.496	75.974	217.016	14.769	25.924
2 vCPU	7	70.327	98.219	169.503	38.210	48.928

Table 5.12: Multi-VM end-to-end latency and per-frame response-spread results. Each measurement point contains 100,000 transmitted frames and therefore $100000 \times M$ expected responses for M participating VMs. No expected responses were lost in any run.

Figure 5.10 compares the end-to-end latency distributions of both topologies. As more concurrently replying VMs are added, the distributions move toward higher latencies.

One-vCPU scalability For the one-vCPU configuration, the median latency increases from 40.491 µs with one active VM to 75.634 µs with eight active VMs. The 99.9th percentile increases from 45.936 µs to 110.765 µs. The largest observed latency in the final experiment is 340.922 µs, although these runs remain short and thus do not allow an accurate characterization of potentially rare outliers. In total, even though the additional VMs increase the overall response latency, the maximum tested one-vCPU configuration with eight active VMs remains well below the 5 ms target.

The corresponding response spread also increases continuously. This indicates that the increase in pooled latency is accompanied by a greater temporal separation between the responses of VMs to the same input frame.

As a coarse utilization check, inspection of pCPU utilization with `htop` during a run with eight responding VMs showed an approximate 25 % to 26 % utilization of the shared QEMU and virtualization support pCPU11. This indicates that no sustained saturation or overload was apparent during the evaluation of the reference workload.

Two-vCPU scalability The two-vCPU configuration shows the same general behavior. At the maximum tested scale of seven concurrently responding VMs, the pooled median is 70.327 µs, the 99.9th percentile is 98.219 µs, and the largest observed latency is 169.503 µs.

The response spread likewise increases with scale. At seven VMs, the median spread reaches 38.210 µs, with a 99.9th percentile of 48.928 µs. Figure 5.11 depicts the per-frame spread for the two-vCPU measurements. The spread does not change across runs over time but remains concentrated within bands that depend on the number of responding VMs.

Similarly to the one-vCPU experiments, `htop` was used during a seven-VM run to inspect pCPU utilization. The housekeeping pCPU10 showed approximately 13 % utilization, and the virtualization support pCPU11 approximately 22 %, again showing no signs of shared processor exhaustion from the implemented reference workload.

5.6.3 Scalability Findings

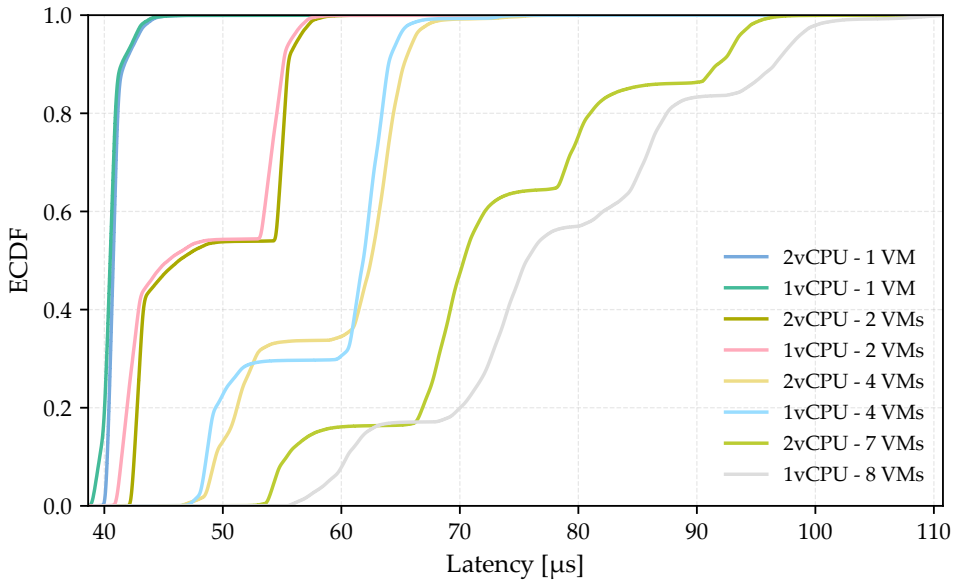
An important result is that the one-vCPU and two-vCPU topologies produce nearly identical latency distributions at equal VM counts, which extends the topology trade-off. The additional housekeeping vCPU does not require an additional pCPU for every deployed VM, but multiple VMs can be consolidated effectively. For the evaluated workload, this consolidation does not introduce a meaningful latency penalty.

Therefore, the experiments answer RQ5 positively within the evaluated scope. The evaluated configurations remain viable when multiple IED VMs are deployed concurrently, whilst the observed sub-millisecond end-to-end latency is maintained, with no missed responses. We evaluated a maximum of eight one-vCPU and seven two-vCPU VMs, with consolidation as described previously.

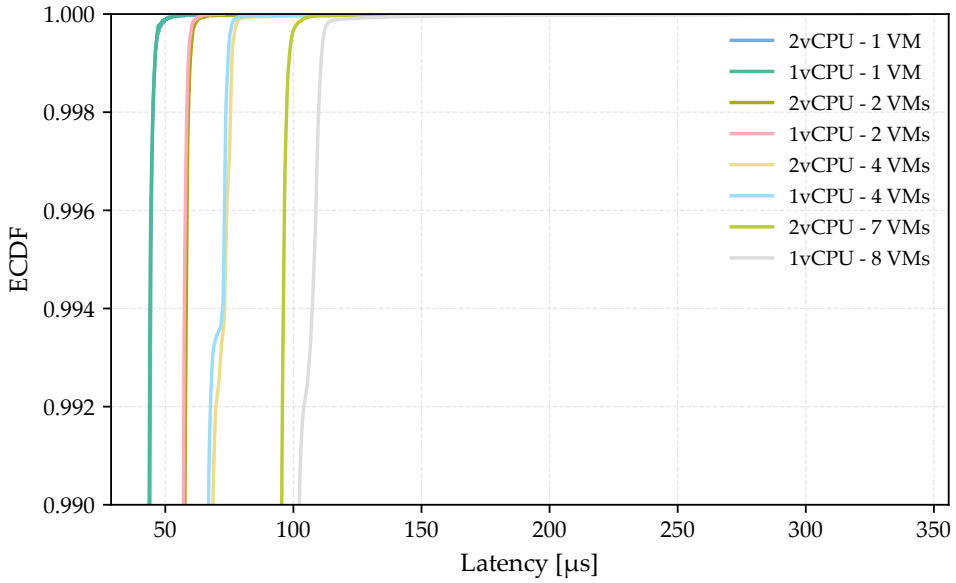
The increasing latency and response spread show that additional VMs are not free. However, the measurements include the complete Linux bridge, TAP, virtio/vhost, and *River* receive path, and the previously mentioned exploratory four-VM experiment demonstrates that the behavior of this shared processing path can influence multi-VM latency. This may also apply in the consolidated configuration, as network frame processing relies on a single pCPU across all VMs, which may contribute to the observed response spread. This might be a reason why the results show this increasing latency spread with increasing VM counts. As mentioned, detailed investigation and optimization of packet delivery and host networking is not in the scope of this thesis (cf. Section 3.1).

To conclude, it is important that the tested scales not be interpreted as a general platform-capacity limit. The results establish the feasibility of consolidation and multi-IED deployment within the scope of the implemented representative protection

workload and hardware and software. Production IEDs may include additional functionality, such as logging, storage, and management, which can increase the workload on shared processors. Determining capacity limits for such other IED implementations requires additional analysis.



(a) Dense end-to-end latency distributions.



(b) Upper tail beginning at the 99th percentile.

Figure 5.10: End-to-end latency distributions for the consolidated multi-VM configurations. Equal-count one-vCPU and two-vCPU configurations are shown together for 1, 2, and 4 VMs; the maximum tested scales of 8 one-vCPU and 7 two-vCPU VMs are also included.

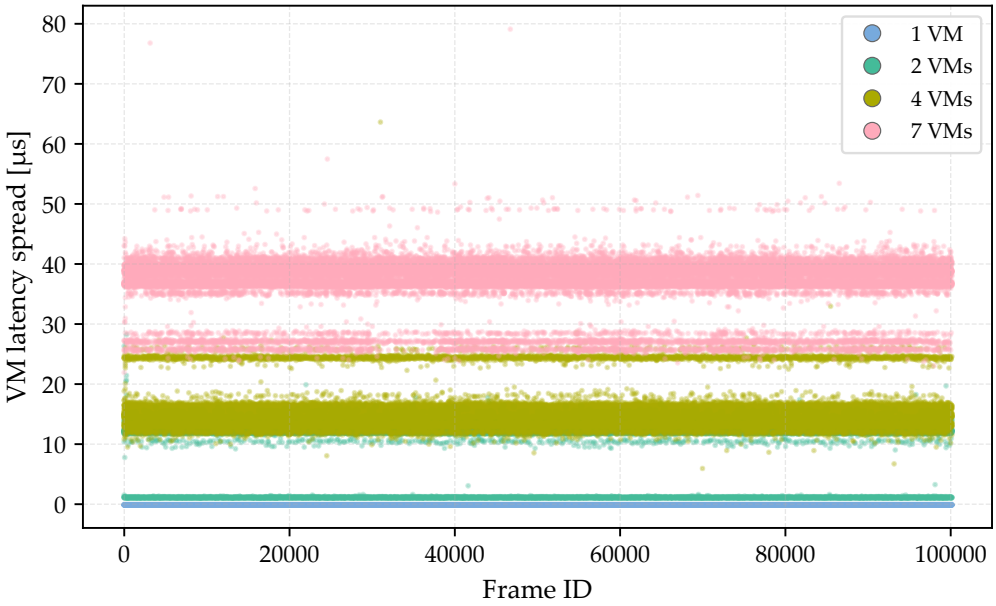


Figure 5.11: Per-frame latency spread for the consolidated two-vCPU multi-VM experiments. The spread is the latency difference between the first and last VM response belonging to the same transmitted frame. Increasing the number of VMs produces larger but stable response separation.

6 Discussion

6.1 Determinism and Real-Time Guarantees

The term *deterministic* in the context of this thesis does not imply a formally proven upper bound on end-to-end latency. Instead, it refers to the ability of the designed and implemented system to provide low, reproducible, and predictable timing behavior under the defined system configurations and representative vPAC workload. This distinction is important because the evaluated platform includes a general-purpose operating system, KVM/QEMU-based virtualization, virtualized I/O, and a complete networked end-to-end packet path. We characterize the timing behavior of this system empirically, rather than via formal schedulability proofs for each component involved in the interactions.

Within this empirical scope, the results show a high degree of predictability. The long-running measurements in Section 5.5 contain 86.4 million end-to-end latency samples across both evaluated VM topologies. The dense distribution remains reproducible between independent one-hour runs, with normal latencies concentrated around 40 µs to 46 µs. Furthermore, no packet loss or latency degradation is observed over time, and no measured response exceeds the protection target of 5 ms.

However, the measurements also indicate why these results must not be interpreted as a hard upper bound. A short, localized, time-related disturbance with millisecond-scale latency is contained in one run of each topology. Even though the maximum latency of both events remains below the protection target and is not reproduced in any other run, their presence displays that rare interference can still occur outside the dense percentile region. Thus, our empirically observed maximum is not a guarantee that a larger event cannot occur during the longer operation or under conditions not covered by the evaluation.

This timing behavior depends on the configuration of the entire system and the end-to-end path, as shown by the mechanism-oriented experiments in Chapter 5. On the one hand, incorrect priorities or task allocation can produce substantially higher latency and packet loss. On the other hand, properly configured mechanisms allow retaining a narrow and repeatable latency. Thus, *determinism* in this system should be understood as a property of the complete configured end-to-end execution environment and not of individual mechanisms in isolation.

Therefore, the evaluation supports the conclusion that PREEMPT_RT-based KVM guests can provide sufficiently predictable end-to-end timing for the representative vPAC workload investigated in this thesis. The evidence is empirical, not a formal, proven hard-real-time guarantee or upper bound on end-to-end latency. Such a guarantee would require analysis of all components contributing to the end-to-end path, which is not provided by the measurement-based methodology and scope used in this thesis.

6.2 One-vCPU vs. Two-vCPU Trade-offs

The experiments performed and their results do not identify either VM topology as superior. Instead, both operate on a consolidation-density versus isolation-flexibility trade-off. As shown in Section 5.4 and Section 5.5, both configurations are able to maintain low and stable protection latency.

The one-vCPU topology uses fewer vCPUs and consequently requires less physical host computing resources, allowing greater consolidation density of VMs on the host. Furthermore, its dense end-to-end latency is slightly lower in the evaluated system, by approximately 1 µs, which in general does not represent a meaningful advantage. The important limitation is that all guest work must ultimately share a single processing unit; this includes guest housekeeping, the real-time workload, and any auxiliary work. This implies the need for strong real-time admission control, as otherwise real-time workloads can overload the guest system and prevent fair-scheduled work from making progress, as demonstrated in Section 5.4.

The two-vCPU topology consumes an additional vCPU and thus more physical computing resources, but permits a deliberate separation of latency-critical real-time, and ordinary housekeeping and auxiliary work. As shown in Section 5.6, the additional vCPU does not require an additional pCPU for each VM; the housekeeping vCPU threads can be effectively consolidated across multiple VMs. Furthermore, this separation allows the fair-server to remain enabled, unlike in the one-vCPU configuration. It also improves failure containment: the experiments showed that a saturated real-time vCPU does not prevent management and housekeeping activity from progressing on the housekeeping vCPU. The VM-exit experiments in Subsection 5.3.2 provide a related benefit, by being able to localize VM-exit effects to the pCPU backing the vCPU on which the exit-generating workload executes.

Thus, the resulting trade-off is primarily architectural in nature. One-vCPU guests can be attractive if physical core efficiency and VM consolidation density are important and the real-time workloads are bounded and controlled. Two-vCPU guests offer more flexible workload execution and management when sufficient cores are available on

the host machine. On modern many-core COTS systems, this additional flexibility can be valuable, but the one-vCPU topology remains valid if the stronger workload assumptions are acceptable.

6.3 Remaining Tail Latency

After applying the scheduling and isolation mechanisms identified in Chapter 5, the dominant, dense end-to-end behavior is highly stable, as detailed in Section 5.5. However, the measurements still contain rare observations well above the dense latency region.

In Chapter 5, we identify multiple mechanisms whose effects were reproducible under controlled conditions and can be attributed to these mechanisms, such as IRQ thread priorities and fair-server mechanisms in Section 5.2, and VM-exit effects in Subsection 5.3.2. It is important to distinguish the identified mechanisms from these remaining events.

The remaining localized disturbances observed in the final end-to-end experiment cannot be associated with the same confidence. Within these high-latency periods, consecutive frame latencies often decrease approximately by the 250 µs inter-send interval. This phenomenon is consistent with a temporary backlog followed by queue draining. The measurement setup does not reveal where this backlog may have formed. In particular, investigation of the host networking on *Christina* was not in the scope of this thesis (cf. Section 3.1). Therefore, queueing in the bridge, tap, virtio, or related host-side network processing remains a potential contributor.

The exploratory experiment detailed in Section 5.6, in which four VMs are concurrently replying to one input frame, where the QEMU and virtualization support threads of each VM also receive their own dedicated pCPU, displayed further anomalous behavior. This behavior appears to reside somewhere in the packet return path, specifically when four VMs reply concurrently, potentially providing further evidence that the shared packet-processing path can exhibit tail behavior.

Furthermore, *River* introduces another source of uncertainty. As the measurement boundary includes *River* and the final timestamps are taken inside of user-space applications on the host, the reported end-to-end latency includes scheduling and networking on *River* as well. Temporary delays in either of these areas can lead to the observed latency behavior, even if the guest itself had already produced the response in time. Consequently, the remaining millisecond-scale latency events should be interpreted as disturbances to the whole end-to-end path, rather than just delayed guest execution or virtualization delays.

As mentioned in Section 3.1, networking-specific optimization and analysis are

addressed in the accompanying work on the communication path between virtualization host and VM guests [25]. The evaluation in this thesis focuses on the scheduling and virtualization mechanisms that can be controlled and reproduced directly.

From the general application perspective, the remaining tails are nevertheless important even though they occur rarely. They are orders of magnitude larger than the normal percentile regions and cannot be ignored. Therefore, the results support low and stable empirical end-to-end latency while also showing that the remaining latency tail can be influenced by components across the whole end-to-end path and cannot yet be attributed to a single specific cause.

6.4 Configuration Guidance

Our evaluation does not identify a single optimal configuration; rather, the results suggest a set of configuration principles necessary for maintaining low, reproducible latencies on the evaluated platform. This section details the identified mechanisms and configuration controls.

Section 5.2 shows the benefits of PREEMPT_RT and that its proper arrangement of scheduling priorities into a priority hierarchy matters. Severe latency growth and packet loss can result from misconfiguration. The whole latency-critical protection path must be considered. Our resulting hierarchy places the relevant network IRQ threads above the protection process and places auxiliary real-time work below it. Ordinary fair-scheduled work falls below the real-time classes.

In general, CPU isolation and pinning principles should be used to keep unrelated execution away from the latency-sensitive path. On the host, the pCPUs backing the real-time vCPUs should be isolated from operating system noise and other processes. Within a two-vCPU guest, latency-critical protection processing can be placed on a dedicated, isolated vCPU, whilst other work is performed on the housekeeping vCPU. The QEMU emulator and virtualization threads are also placed on host CPUs, which do not back the real-time vCPUs. For the deployment of multiple VMs, the results in Section 5.6 show that consolidating QEMU threads and, in the two-vCPU topology, the housekeeping vCPUs, across VMs, is a viable option for consolidation density.

The treatment of fair-scheduled work and the resulting fair-server configuration depends on the selected VM topology, cf. Section 6.2 and Section 5.4.

Further optimization is achieved by avoiding unnecessary sources of wake-up and device emulation overhead. Hosts and guests use the performance CPU-frequency governor, and the host disables processor idle states. The guests use polling idle behavior on the latency-critical vCPUs to avoid repeated virtual `HLT` wake-ups on the latency-sensitive path. An important note here is that the evaluated configuration of the

two-vCPU topology in the single-VM experiments differs from the configuration in the multi-VM experiments (cf. Subsection 4.1.3). The `haltpoll` configuration is a promising replacement for the global `idle=poll` parameter, but the `haltpoll` configuration has not been tested across the single-VM experiments. However, in general, these choices trade energy efficiency for lower wake-up latency, but our timing requirements justify this choice. Similarly, virtual device configuration should favor paravirtualized mechanisms and avoid unnecessary legacy user-space emulated I/O, as seen in Subsection 5.3.2.

Therefore, both evaluated VM topologies are valid deployment options when configured with these principles in mind (cf. Section 6.2). The recommendations described in this section are specific to the evaluated hardware, software, configuration, and representative protection workload. They should be interpreted as deployment guidance derived from measured mechanisms, not as universal real-time configuration guidelines for arbitrary application scenarios.

6.5 Limitations and Threats to Validity

As described in Section 6.1, the results of this thesis are based on empirical measurements and must be interpreted as such. Although repeated stress tests and long-running measurements are included, they still observe only a finite subset of possible executions. Section 5.5 provides substantial evidence for stable timing behavior, but the existence of still rarer events cannot be excluded. Consequently, all observed latencies and results are not formally established as upper bounds or worst-case times.

Similarly, the experiments conducted establish how the evaluated mechanisms behave under controlled workloads, but do not prove that every possible combination of tasks, interrupts, host activity, and virtualization will satisfy the protection deadline. Therefore, the results apply to the bounded workload assumptions imposed by the experiments.

The protection workload evaluated in this thesis was specifically implemented to be representative of IED protection computation, as researched in open-source repositories. The implemented protection is not a complete commercial IED software stack. The implementation contains a protection-critical workload path, including measurement preprocessing and several representative ANSI protection functions. However, input generation is deterministic, and synthetic Sampled Values (SV) and Generic Object-Oriented Substation Events (GOOSE) payloads are used instead of a full IEC 61850 implementation. A larger commercial application could potentially introduce additional tasks, communication services, logging, or other work. Therefore, the absolute latency and execution time values should not be generalized to arbitrary vPAC software without additional evaluation.

As detailed in Section 6.4, the experiment results depend on specific hardware and software. The identified mechanisms in the evaluation are expected to remain relevant beyond our specific platform, but their measurable effects might differ across other hardware and software versions.

Furthermore, the selected end-to-end measurement boundary spanning over *Christina* to *River* introduces additional ambiguity in the interpretation and analysis of rare tail events, as detailed in Section 6.3. More detailed decomposition requires additional timestamping or tracing points along the end-to-end path and can itself risk introducing delays or perturbations. As mentioned, networking-specific optimization is out of scope for this thesis and is addressed separately in the accompanying networking work [25].

Similarly, other measurement instrumentation may itself be a potential source of interference. For example, detailed host-side VM-exit tracing was found to introduce VM-exit delays itself and was therefore avoided in the final measurements. This is the primary reason why experiments were implemented in a way that attempts to limit potential measurement interference, such as omitting logging I/O during experiment runs, whilst retaining enough information to characterize the resulting outcomes. However, this means that not enough information may be retained to reconstruct every event that has led to rare outliers, as shown in Section 5.5.

The synthetic stressors used throughout Chapter 5 are furthermore deliberately chosen to expose particular scheduling and virtualization mechanisms. They should not be interpreted as normal production behavior. Furthermore, they do not establish the probability with which such conditions would occur in a deployed vPAC system. For example, the large delays under pathological legacy I/O motivate avoiding such device paths in practice, but should not be interpreted as a benchmark for representative normal protection workloads.

Lastly, regarding the conclusions concerning consolidation and scalability of multiple IEDs, i.e., multiple concurrently operating VMs: The results are limited to the VM counts and CPU placements evaluated in Section 5.6, which are limited to the actual core count and resource availability on our test hardware. Even if several VMs can coexist in the testbed configuration, this does not imply unbounded scalability. The previously mentioned restrictions on tested combinations of tasks and system load continue to apply; in particular, our representative vPAC workload was tested with no additional auxiliary workloads. The multi-VM results should be interpreted as a demonstration of viability within our representative protection workload, rather than general capacity limits and conclusions.

7 Related Work

7.1 Virtual IED and vPAC Systems

Early work already investigated whether general-purpose virtualization could provide suitable communication for protection and automation systems. Ferreira and de Oliveira [29] evaluated an architecture using KVM/QEMU and OpenDDS[1] for communication, primarily comparing communication between hosts and VMs. Reported latencies between components are used to assess the feasibility of soft real-time applications. The authors explicitly mention integration of an actual IEC 61850 application into the timing evaluation as subsequent work. This study provided early evidence for the feasibility of virtualized communication, but did not yet characterize the timings of a complete protection path [29].

Industry reference architectures move from communication benchmarks toward more complete virtual protection relays (VPRs). The Intel/Kalkitech VPR whitepaper [30] details implementation designs of virtualized IEDs onto VMs. One of the described architectures resembles our chosen deployment model (cf. Section 3.4) by hosting each discrete IED in its own VM. However, an alternative design that involves shared services among VMs is also presented. The reference implementation contains IEC 61850 Sampled Values (SV) and Generic Object-Oriented Substation Events (GOOSE) communication, signal conditioning (also called measurement preprocessing), and several representative ANSI protection functions. Laboratory tests measure the resulting protection response time. This whitepaper, therefore, demonstrates a virtual protection application stack with related functionality and illustrates the one-IED-per-VM model as an established architectural option [30].

Further studies demonstrate the feasibility of virtualized protection under substantial consolidation and realistic application load. Wójtowicz et al. [31] develop a Linux/KVM-based platform with SR-IOV networking and static processor allocation, operating tens of VMs hosting virtual IEDs. They connect the virtualized environment to physical test equipment and evaluate the operating time of virtual overcurrent protection. Carvalho et al. [32] similarly develop a KVM-based protection and control environment utilizing PREEMPT_RT, and a tuned system. Their evaluation combines four real-time and three

[1] `https://opendds.org/`

general-purpose VMs and measures IEC 61850-like GOOSE and SV communication latency. Schönborn et al. [33] evaluate industry protection and control software using PREEMPT_RT, explicit resource isolation, CPU affinity, cache allocation, and optimized virtual networking. The work primarily focuses on containerized protection and control applications, however, a brief section describes additional evaluation inside of VMs using both KVM and VMware ESXi. The reported experiments include multi-hour timing measurements and a field deployment at an MV substation, where two virtual protection and control instances have been running for over one year. These results provide strong evidence that virtualized protection and control is practically feasible when resources are carefully provisioned [31, 33].

Some more recent work focuses more on the evaluation of a complete software-defined substation in accordance with IEC 61850. Kabbara et al. [34] combine virtualized protection and software-defined networking. Their testbed uses dedicated VMs for overcurrent protection processing, deploying one virtualized IED per VM, whilst also introducing containerized IEDs for scalability experiments within the same VMs. They evaluate changes in the IEC 61850 communication protocol, system scale, and protection functionality. The authors note that their setup is not optimized for real-time determinism and attribute one observed outlier to said absence, stating it could be avoided with proper RT tuning. RT optimizations are described as a potential avenue for future work. They identify measurements such as cyclic execution and jitter as additional, if the system is to be optimized for RT evaluation.

In addition to the published work, several open-source implementations of virtual or software-based IEC 61850 and protection are available. To properly implement representative protection processing in this thesis, six such implementations were examined [4–9]. Recurring elements and the general structure of the protection-processing path were investigated. One of these, namely `iec61850_open_server` [9], is also used as the basis of the virtualized IEDs evaluated by Kabbara et al. [34].

Overall, the existing work establishes that virtualized protection is feasible across several architectures and scales [29–34]. The studies furthermore detail that proper resource isolation and networking are important for real-time behavior. However, the primary focus is generally on architectural viability or scalability of the resulting system. Detailed experimental decomposition of the individual scheduling and virtualization mechanisms that can produce stable and predictable timing is less central to this work. The following sections, therefore, consider related real-time Linux and real-time virtualization literature.

7.2 Real-Time Linux

PREEMPT_RT is an established approach for improving the predictability of Linux by modifying several kernel mechanisms, resulting in a reduction of kernel code that can delay a high-priority task [11–13, 35]. One important consequence is that much interrupt processing is deliberately moved into schedulable threads, thus enabling precise control over execution via scheduling mechanisms. However, the literature also emphasizes that simply using a PREEMPT_RT kernel is not sufficient to achieve low and reproducible scheduling latency. Kernel, application, interrupt, and CPU placement must be configured correctly to achieve reliability [11]. This observation is important to this thesis, as the scheduling of IRQ threads relevant to the real-time workload is itself part of the evaluation.

Low-latency Linux designs, therefore, commonly combine PREEMPT_RT with additional kernel isolation mechanisms. Examples in the literature include CPU isolation mechanisms, directing interrupt and kernel activity to specific cores, suppressing timer ticks and RCU activity on isolated cores, disabling SMT, and restricting dynamic CPU frequency changes and sleep states [22, 32, 33, 36]. These principles closely correspond to the baseline configuration mechanisms used in this thesis. Such system tuning, however, can also introduce trade-offs, such as disabling energy-saving mechanisms, which has been shown to significantly increase power consumption, especially when a system is idle [22]. Furthermore, numerous online resources exist, detailing real-time configuration and application management, for example [37–40].

Empirical evaluations of real-time Linux commonly focus on scheduling latency and jitter, potentially under additional, suitable stress loads. `Cyclictest` is widely used as a benchmark for scheduling latencies, whilst stressors attempt to increase the probability of unfavorable delays [11, 35]. Schönborn et al. [33] apply the same general methodology, combining `cyclictest` with `hackbench`. However, they also note that such synthetic scheduling measurements are too simplistic to establish reliable performance on their own, and thus complement them with measurements from actual protection applications.

Empirical, measurement-based characterization must nevertheless be distinguished from formal timing guarantees. Reghenzani et al. [11] detail how the PREEMPT_RT literature has heavily relied on empirical measurements, due to the complexity of Linux and the nondeterministic behavior of COTS hardware, which strongly complicate timing analyses. However, formal analysis is not entirely absent: Oliveira et al. [35] derive sound bounds to scheduling latency utilizing a formal model of kernel behavior together with measured model variables. This result is narrower than a bound on complete application latencies, but demonstrates the possibility of more formal analyses compared to black-box measurements alone.

The real-time Linux literature provides mechanisms and experimental methodology on which the guest configuration in this thesis builds. Virtualization, however, introduces another scheduling and execution layer, with additional sources of latency; thus, related work on real-time virtualization is considered next.

7.3 Real-Time Virtualization

Virtualization adds another scheduling layer between a real-time application and the physical processor. In KVM/QEMU, each vCPU is represented by a host Linux thread and is thus subject to host scheduling [41]. Therefore, configuring only the guest as a real-time system will not remove delays originating from host virtualization mechanisms [41]. Accordingly, previous work treats reliable guest timings as a property of both the host and guest configuration [18, 41].

A common approach is to explicitly partition processor resources. Earlier KVM work reports low guest scheduling latency by dedicating physical CPUs to back vCPUs, while later work has also explicitly investigated regular scheduling of vCPU threads on shared pCPUs [18, 41]. Riel [18] details a KVM configuration with vCPU threads being pinned to dedicated isolated pCPUs, while housekeeping is located on separate pCPUs. The same separation principle is also extended into the guest, differentiating between real-time and housekeeping vCPUs [18]. This approach and its principles closely resemble those used in this thesis.

The real-time behavior and the interference introduced by the added virtualization layer are again commonly evaluated through guest-visible scheduling latency. As in the real-time Linux literature, `cyclictest` is commonly used within VMs whilst varying background stress workloads are applied [41–44]. Experiments demonstrate that the priorities and placement of vCPU and QEMU threads can significantly impact measured latency [42], while Zuo et al. [43] additionally emphasize CPU shielding and a priority ordering for their experiments, further reinforcing the principle that low guest latency depends on host-side configuration rather than solely on enabling PREEMPT_RT. Again, numerous online resources exist, providing guidance on real-time host and VM configuration with KVM/QEMU, for example [45, 46].

Furthermore, earlier studies identify virtualization overhead not as a single, uniform cost. Userspace I/O paths and device emulation are identified as potential sources of higher latency [18, 42]. Kiszka [42] reports latency effects from configurations involving QCOW2 storage and emulated display or serial I/O. Riel [18] identifies QEMU userspace emulation as a slow path for real-time guest execution. Thus, these results motivate a more nuanced distinction among virtualization events and execution paths, rather than treating virtualization overhead as constant interference.

Overall, prior work demonstrates that KVM and virtualization can support low-latency real-time guests when hosts and guests are carefully configured. The principal evaluation metric in this literature is generally guest-visible scheduling, similar to real-time Linux evaluations.

7.4 Comparison with This Work

The preceding work establishes the feasibility and applicability of virtualized protection, as well as the importance of careful real-time configuration. Existing vIED and vPAC studies evaluate complete architectures and scalability, sometimes using tuned PREEMPT_RT and KVM environments. However, these configuration mechanisms are generally applied within the evaluated platform, while the resulting performance is measured on the overall application level. In contrast, the real-time Linux and virtualization literature investigates specific mechanisms in greater detail but typically relies on generic benchmarks. A less explored connection is between such mechanism-level behavior and the resulting timing of virtualized protection.

This thesis connects these areas by implementing and evaluating a representative protection pipeline workload derived from published and open-source implementations. The evaluation sits at the intersection of the described areas, by investigating specific effects of scheduling, co-location of workloads, different vCPU topologies, and different VM-exit-generating activity on virtualization. Mechanism-oriented experiments are combined with the representative protection computation, using generic benchmarking where appropriate, such as the study of VM-exit interference, whilst other mechanisms are investigated using the end-to-end latency of our implemented benchmark setup. Additionally, we separate the computational cost of protection computation from the resulting end-to-end latency measurements and evaluate the final system latency over extended operation. Overall, the contribution is an experimental characterization of the interactions between specific real-time Linux and virtualization mechanisms within the specific context of vPAC workloads.

8 Conclusion and Future Work

8.1 Summary and Answers to the Research Questions

This thesis investigated whether PREEMPT_RT-based KVM virtualization on COTS hardware can provide sufficiently low and stable timing for representative vPAC workloads. The work considered the complete scheduling environment, including CPU placement, interrupt handling, real-time and fair-scheduled workloads, guest topology, representative protection processing, and virtualization events. The resulting configurations were then evaluated using end-to-end latency measurements to determine whether the protection deadline could be met, and specific mechanisms were also microbenchmarked, such as the representative workload implementation and VM-exit effects. The experimental results provide the following answers to the research questions introduced in Section 1.2.

RQ1 PREEMPT_RT-based KVM guests can provide low and highly reproducible end-to-end latency for the representative vPAC workload evaluated in this thesis. Across six approximately one-hour measurements of both designed guest configurations, 86.4 million protection exchanges were completed without packet loss or exceeding the protection target of 5 ms. The dense distribution is concentrated around a latency of 40 µs to 46 µs. The largest observed latency, 4.335 ms, occurs during a short, localized disturbance and appears only in one repetition. Furthermore, as discussed in Section 6.3, the outliers themselves might arise in components deliberately left out of the thesis scope, such as host networking. Therefore, the general results demonstrate high empirical predictability, but do not constitute formal upper bounds to end-to-end latency.

RQ2 Low and stable latencies depend on explicit scheduling and isolation. In particular, we implemented a scheduling hierarchy inside the guests for latency-critical protection processing. CPU isolation and affinity are likewise important for keeping unrelated execution away from latency-sensitive components. Furthermore, proper configuration of workloads within the guests is imperative, otherwise the fair-server might introduce severe latency effects. PREEMPT_RT is valuable because it enables

greater preemption and explicit scheduling of otherwise implicit kernel execution, potentially yielding a lower wake-up latency tail.

RQ3 The representative protection processing itself only contributes a small fraction of the measured timings. In our representative workload, the total sequential protection processing in the guests requires approximately 1.19 µs per invocation. Virtualization events are more nuanced: The frequency alone of these events does not predict their effect on guest timing. High VM exit rates may remain very compatible with low-latency execution, whereas lower VM exit rates, which trigger different execution paths in the virtualization layer, namely requiring an exit to the QEMU userspace, may impose rare, unpredictable millisecond-scale guest-visible delays. The practical implication, therefore, is not to minimize every VM exit, but to avoid expensive exits, particularly to QEMU userspace, for example, because of unnecessary device emulation. Furthermore, controlling where exit-generating workloads execute can benefit the latency-critical path.

RQ4 Both evaluated configurations are viable for executing the representative protection workload and maintaining low, stable latencies, but they offer different trade-offs. The one-vCPU configuration requires fewer host resources, thereby enabling a higher consolidation density. However, all guest tasks, including ordinary processes and real-time, latency-critical work, must ultimately share the same vCPU, hence requiring stronger control over the workloads admitted and executed. The two-vCPU configuration consumes additional host resources, but allows explicit separation of the latency-critical and housekeeping tasks. Under potential real-time overload, this separation provides better failure containment. The multi-VM evaluation shows that this separation does not require an additional dedicated pCPU for the housekeeping vCPUs of each deployed VM: the housekeeping vCPU threads across multiple VMs can be consolidated onto a shared pCPU. The choice between the two configurations is primarily an architectural trade-off between the simpler, more resource-efficient one-vCPU design and the stronger workload separation and fault containment provided by the two-vCPU design.

RQ5 The evaluated configurations remain viable when multiple IED VMs are deployed concurrently within the tested scale. The final experiments evaluate up to eight one-vCPU and up to seven two-vCPU VMs. All runs produce no missing responses, with end-to-end latencies well below the protection threshold. At these maximum tested scales, the 99.9th percentile remains approximately 111 µs for the one-vCPU configuration and 98 µs for the two-vCPU configuration. Increasing the number of active

VMs increased both the pooled end-to-end latency and the latency spread between responses. The results primarily establish the feasibility of multiple virtualized IEDs executing concurrently within the designed system that underlies the implemented representative protection workload. No general capacity limits for arbitrary production IEDs are established.

Taken together, the results show that deterministic virtualization, as introduced in this thesis in Section 6.1, is feasible on the evaluated COTS platform. The achieved behavior arises from the complete configuration developed and evaluated in this thesis and not simply from PREEMPT_RT, so that protection processing can make progress promptly by limiting interference from lower-priority and auxiliary activities. Under the conditions provided in the thesis, both evaluated topologies and configurations can maintain low, reproducible end-to-end latencies with a significant margin over the chosen protection deadline during sustained single-VM operation and remain viable when several instances of the representative protection workload are consolidated on the same physical host.

Simultaneously, the experiments detail important limitations of this conclusion. Rare end-to-end latency tail events remain; latency also increases as more VMs share parts of the packet-processing path; and the measurements provide empirical rather than formal evidence. Thus, the results are to be interpreted as a demonstration of feasibility for this platform and system design, identifying effective configuration principles derived from the evaluated workload and platform, and should not be interpreted as hard-real-time guarantees or general capacity bounds. Relevant source code and configuration files are publicly available [1].

8.2 Future Work

The evaluation investigates configurations on a single hardware-software platform. A first direction for future work might therefore be broader validation, including longer-duration experiments and repetitions over more hardware platforms. These would provide more evidence about how well the mechanisms and principles detailed in this thesis generalize to other platforms.

Another open question is the remaining brief end-to-end tail latency observed in Section 5.5. The end-to-end boundary used in this thesis deliberately measures the entire RTT path but cannot identify where the disturbances described occur. Additional timestamping or tracing at selected points in the hosts and guests may help provide insight into the introduced delays.

The virtualization host itself offers another avenue for investigation that may correlate with the observed tail latencies. The evaluated *Christina* host uses a generic, non-

PREEMPT_RT kernel. Applying PREEMPT_RT and related principles might potentially reduce scheduling delays for latency-critical host execution, such as vCPU threads, emulator, or networking threads. However, as mentioned, host networking optimization is already being investigated by accompanying work [25].

Similarly, since *River* is included in the end-to-end latency measurements, tightening benchmarking on that physical host may yield insight into delays introduced by the measurement instrumentation on *River* itself. Delays introduced due to scheduling on *River* are currently indistinguishable from those of the system under test.

The representative workload and virtualized IED systems can be extended to a potentially more complete production environment, including currently omitted services such as auxiliary logging and management interfaces. This is especially relevant to the multi-VM results, as the current measurements use the representative protection workload. Additional auxiliary activity could increase the utilization of the shared processors, thereby yielding different consolidation outcomes. Evaluating more complete vPAC software could test whether the principles remain effective as the software stack becomes more complex and how far the observed consolidation behavior generalizes.

Similarly, integration with the accompanying networking work [25] would allow the complete communication and virtualization system to be evaluated as a whole. In general, in long-running measurements and in multi-VM scalability experiments, this integration would help separate IED execution from effects introduced by the current Linux networking path.

Finally, the empirical methodology of this thesis could be complemented by formal methods to characterize potential upper bounds on end-to-end latency. Such models may include schedulability analyses, task execution models, interrupt handling, or virtualization activity within KVM/QEMU. However, developing such models for the complete protection path seems non-trivial, especially because it crosses guest, hypervisor, host, networking, and further components. Nevertheless, they could provide a useful complement to the established empirical evidence.

As a final potential topic of interest, in Section 3.4 we adopted the design of deploying one IED per VM, thereby establishing a clear isolation boundary. To potentially maximize consolidation density, it might be interesting to investigate how multiple IEDs might be consolidated inside a single VM. This would include investigating dynamic provisioning strategies to determine whether an IED is deployed into an existing VM or assigned a dedicated VM. Additionally, this will raise further questions about task admission, isolation, and related issues, which the one-IED-per-VM model currently avoids.

Abbreviations

IED Intelligent Electronic Device

vPAC virtualized protection and control

COTS commercial off-the-shelf

VM virtual machine

KVM Kernel-based Virtual Machine

vCPU virtual CPU

pCPU physical CPU

MU merging unit

CB circuit breaker

SV Sampled Values

GOOSE Generic Object-Oriented Substation Events

DFT Discrete Fourier Transform

IRQ interrupt request

XML extensible markup language

DFT Discrete Fourier Transform

ECDF empirical cumulative distribution function

SSH Secure Shell

List of Figures

2.1 Simplified IEC 61850-based substation architecture and the communication path relevant to this thesis. 5
2.2 Example vPAC substation architecture. 7
2.3 Protection processing path inside an IED. The solid path represents the latency-critical protection path (hot-path), while auxiliary functionality executes outside the hot-path. 8

3.1 High-level system architecture used in this thesis. One physical host emulates the measurement source and trip receiver, while a second physical host acts as the COTS virtualization host where virtualized IEDs execute the protection workload as VMs and return a response to the first host. 22

5.1 End-to-end latency for the generic and PREEMPT_RT guests with and without ordinary CPU load. The table summarizes the range across the three repeated runs for each condition. The plot depicts the ECDF up to the 99th percentile. 54
5.2 Effect of network interrupt scheduling under competing real-time execution. The PREEMPT_RT IRQ50 configuration places critical network processing below the synthetic RT workload, causing severe delay and packet loss. 55
5.3 Time-series latency plots showing the effect of the per-CPU fair-server when fair-scheduled and real-time workloads contend on the isolated RT vCPU. Halving both runtime and period preserves the 5 % fair bandwidth while approximately doubling disturbance frequency and halving the maximum disturbance duration. 56
5.4 VM-exit rates for different workloads. The logarithmic axis shows that workload-generated VM exit activity remains concentrated on the stressed vCPU. The table reports the fraction of exits on the stressed vCPU that return to userspace QEMU. Port-I/O values aggregate the corresponding repeated host-count runs. 63

5.5 Placement of VM-exit activity in the two-vCPU configuration. Moving CPUID or port-I/O stress from housekeeping to RT vCPUs moves the primary exit stream from host pCPU9 to pCPU10. This result confirms that the large workload-generated exit activity can be kept off the RT vCPU through explicit workload placement. 64
5.6 Effect of co-locating high RT utilization and fair-scheduled work. F1 shows the known periodic latency pattern with the default fair server. F2 suppresses this mechanism by disabling the fair server, while F3 avoids it by separating fair and RT execution across two vCPUs. F1 details the results from the first run of 100,000 frames, F2 and F3 pool three runs of 100,000 frames each. 70
5.7 End-to-end latency under increasing bounded RT load. Solid and dashed curves show median and 99.9th percentiles, respectively, for each topology. Markers denote measured load levels; connecting lines are only visual guides and do not imply measurements at intermediate loads. Each point pools three runs of 100,000 frames. No packet loss or latency above 1 ms was observed in any run. 72
5.8 Empirical end-to-end latency distributions for three one-hour repetitions of each VM topology. Each run contains 14.4 million received responses. The displayed range focuses on the dense latency distribution; rare larger observations are reported separately in Table 5.11 and analyzed over time in Subsection 5.5.3. 77
5.9 Time-related behavior of the six measurements. The upper panel shows mean latency in consecutive one-minute windows; the lower panel shows the number of observations exceeding 100 µs in each window. The measurements show no increase in latency over elapsed runtime. Elevated observations are instead concentrated in localized intervals in r1 of each topology. 80
5.10 End-to-end latency distributions for the consolidated multi-VM configurations. Equal-count one-vCPU and two-vCPU configurations are shown together for 1, 2, and 4 VMs; the maximum tested scales of 8 one-vCPU and 7 two-vCPU VMs are also included. 86
5.11 Per-frame latency spread for the consolidated two-vCPU multi-VM experiments. The spread is the latency difference between the first and last VM response belonging to the same transmitted frame. Increasing the number of VMs produces larger but stable response separation. 87

List of Tables

2.1 Representative protection functions considered in this thesis. 10
2.2 Selected Linux mechanisms relevant to isolating latency-critical execution. 15

4.1 Virtualization software versions on Christina. 30
4.2 Host kernel parameters used for CPU isolation and latency-oriented configuration. 32
4.3 Guest kernel parameters used for the one-vCPU and two-vCPU configurations. 33
4.4 VM execution-context placement on *Christina* for the one-vCPU and two-vCPU configurations. 36
4.5 Scheduling hierarchy used inside the PREEMPT_RT guests. 38
4.6 Reference one-vCPU and two-vCPU system configurations used in the evaluation. 44

5.1 Physical hardware used for the evaluation. 45
5.2 Guest-visible scheduling latency with and without synchronous disk-I/O activity. Each condition pools three 120 s `cyclictest` runs with approximately 120,000 wake-ups per run. `cyclictest` executes with `SCHED_FIFO` priority 95. Run maxima report the range of the individually observed maxima rather than an averaged maximum. 50
5.3 Fair-server configurations used to evaluate mixed scheduling classes on the isolated RT vCPU. C2 and C3 reserve the same 5 % CPU bandwidth using different periods. 52
5.4 Mean execution costs of the representative measurement and protection processing. Primary values are averaged across three repeated measurements. Validation measurements use a tenfold smaller batch size. 59
5.5 Workloads used to characterize VM-exit activity and guest-visible scheduling effects. The two-vCPU measurements additionally execute CPUID and port I/O on RT vCPU1 as placement controls. 62

5.6 Guest-visible `cyclictest` wake-up latency under the different observed workloads. In the two-vCPU VM stressors execute on vCPU0 and `cyclictest` on vCPU1. CPUID and port I/O stressors are additionally placed on vCPU1. Port I/O statistics pool the corresponding individual repeated guest measurements. 65
5.7 Cumulative counts of elevated `cyclictest` wake-up latencies. Port-I/O conditions contain additional repetitions because rare events were sampled more extensively; the counts, therefore, characterize only the observed tails and must not be interpreted as a comparison of tail probabilities between configurations. 66
5.8 Conditions used to evaluate co-location of real-time and fair-scheduled work. F2 removes the fair-server reservation, whereas F3 avoids the interaction by separating the scheduling classes onto distinct vCPUs. . . 69
5.9 Conditions used for comparison of both topologies under bounded auxiliary RT load 71
5.10 Evaluated experimental conditions and qualitative behavior under deliberate 100 % RT workloads. O1–O3 use priority 90. D1 lowers the priority below the normal IRQ-thread priority to investigate the starvation mechanism. 73
5.11 Numerical long-running end-to-end latency results. Each individual run contains 14.4 million measurements; each pooled row therefore represents 43.2 million measurements. All runs completed without packet loss or duplicate/reordered responses. Tail counts are cumulative within each row. 78
5.12 Multi-VM end-to-end latency and per-frame response-spread results. Each measurement point contains 100,000 transmitted frames and therefore $100000 \times M$ expected responses for M participating VMs. No expected responses were lost in any run. 83

Bibliography

[1] Leon Simoniants. *vpac-thesis-artifacts. Supplementary source code and configuration artifacts for the Bachelor's Thesis "Scheduling for Deterministic Virtualization of Virtualized Protection and Control Workloads on COTS Hardware"*. 2026. URL: https://github.com/leonsmnts/vpac-thesis-artifacts (visited on 09/20/2026).

[2] Juan C. Lozano, Keerthi Koneru, Neil Ortiz, and Alvaro A. Cardenas. "Digital Substations and IEC 61850: A Primer." In: *IEEE Communications Magazine* 61.6 (2023), pp. 28–34. DOI: 10.1109/MCOM.001.2200568.

[3] Jakub W. Konka, Colin M. Arthur, Francisco J. Garcia, and Robert C. Atkinson. "Traffic generation of IEC 61850 sampled values." In: *2011 IEEE First International Workshop on Smart Grid Modeling and Simulation (SGMS)*. 2011, pp. 43–48. DOI: 10.1109/SGMS.2011.6089025.

[4] Alailton-jr. *virtual-ied-vpac. Software-based IEC 61850 protection IED*. URL: https://github.com/Alailton-jr/virtual-ied-vpac (visited on 09/13/2026).

[5] stevenblair. *rapid61850. Rapid-prototyping protection and control schemes with IEC 61850*. URL: https://github.com/stevenblair/rapid61850 (visited on 09/13/2026).

[6] elshazlio. *Definite-Time-Lag-Overcurrent-Relay-with-Load-Shedding-using-Arduino. A protection relay circuit implementing DTL, a smoothing filter-enabled current sensor, and example load shedding behaviour*. URL: https://github.com/elshazlio/Definite-Time-Lag-Overcurrent-Relay-with-Load-Shedding-using-Arduino (visited on 09/13/2026).

[7] pg-ot. *virtual_substation_OT. A complete virtual substation implementation demonstrating IEC 61850 GOOSE communication between Protection IED and Breaker IED using libiec61850 library*. URL: https://github.com/pg-ot/virtual_substation_OT (visited on 09/13/2026).

[8] renato-monaro. *OpenRelay. OpenSource Protection Algorithms for Electric Power System Relays*. URL: https://github.com/renato-monaro/OpenRelay (visited on 09/13/2026).

[9] robidev. *iec61850_open_server. An open source implementation of an IEC61850 IED using lib61850*. URL: https://github.com/robidev/iec61850_open_server (visited on 09/13/2026).

[10] DEIF A/S. *About protections - AGC 150 ATS Designers handbook*. URL: https://documentation.deif.com/r/agc-150-ats-designers-handbook-4189341346-uk/ac-protections/about-protections (visited on 09/13/2026).

[11] Federico Reghenzani, Giuseppe Massari, and William Fornaciari. "The Real-Time Linux Kernel: A Survey on PREEMPT_RT." In: *ACM Comput. Surv.* 52.1 (Feb. 2019). ISSN: 0360-0300. DOI: 10.1145/3297714. URL: https://doi.org/10.1145/3297714.

[12] Sebastian Andrzej Siewior. *Theory of operation*. Version 7.3.0-rc3. The Linux Kernel Development Community. URL: https://docs.kernel.org/core-api/real-time/theory.html (visited on 09/13/2026).

[13] Sebastian Andrzej Siewior. *How realtime kernels differ*. Version 7.3.0-rc3. The Linux Kernel Development Community. URL: https://docs.kernel.org/core-api/real-time/differences.html (visited on 09/13/2026).

[14] The Linux Foundation. *Realtime Documentation*. July 21, 2026. URL: https://wiki.linuxfoundation.org/realtime/preempt_rt_versions (visited on 09/13/2026).

[15] The Linux Kernel Development Community. *Real-Time group scheduling. 2.1 System wide settings*. Version 7.3.0-rc3. URL: https://docs.kernel.org/scheduler/sched-rt-group.html#system-wide-settings (visited on 09/13/2026).

[16] Linux man-pages project. *sched(7) — Overview of CPU Scheduling*. Version 6.18. Feb. 8, 2026. URL: https://man7.org/linux/man-pages/man7/sched.7.html (visited on 09/13/2026).

[17] Ingo Molnar. *[GIT PULL] Scheduler Changes for v6.12*. Linux kernel mailing list. Sept. 19, 2024. URL: https://lkml.iu.edu/2409.2/03239.html (visited on 09/14/2026).

[18] Rik van Riel. "Real-time KVM from the ground up." In: LinuxCon NA 2016. Red Hat. 2016. URL: https://www.linux-kvm.org/images/d/d2/LC-Rick_van_Riel-Real-time_KVM.pdf (visited on 09/13/2026).

[19] The Linux Kernel Development Community. *The kernel's command-line parameters*. Version 7.3.0-rc3. URL: https://docs.kernel.org/admin-guide/kernel-parameters.html (visited on 09/13/2026).

[20] The Linux Kernel Development Community. *CPU Isolation*. Version 7.3.0-rc3. URL: https://docs.kernel.org/admin-guide/cpu-isolation.html (visited on 09/19/2026).

[21] Frederic Weisbecker. *CPU Isolation – Nohz_full troubleshooting: broken TSC/clocksource – by SUSE Labs (part 6)*. SUSE Labs. Dec. 14, 2022. URL: https://www.suse.com/c/cpu-isolation-nohz_full-troubleshooting-tsc-clocksource-by-suse-labs-part-6/ (visited on 09/13/2026).

[22] Sebastian Gallenmüller, Johannes Naab, Iris Adam, and Georg Carle. "5G URLLC: A Case Study on Low-Latency Intrusion Prevention." In: *IEEE Communications Magazine* 58.10 (2020), pp. 35–41. DOI: 10.1109/MCOM.001.2000467.

[23] Rafael J. Wysocki. *CPU Idle Time Management*. Version 7.3.0-rc3. The Linux Kernel Documentation. 2018. URL: https://www.kernel.org/doc/html/latest/admin-guide/pm/cpuidle.html (visited on 09/13/2026).

[24] Rafael J. Wysocki. *CPU Performance Scaling*. Version 7.3.0-rc3. The Linux Kernel Documentation. 2017. URL: https://www.kernel.org/doc/html/latest/admin-guide/pm/cpufreq.html (visited on 09/13/2026).

[25] Radu-Vladimir Raceala. "Deterministic Network Stack for Virtualized Protection and Control Systems." Bachelor's thesis. Technical University of Munich (TUM), 2026.

[26] The Linux Kernel Development Community. *Linux Kernel Source: Power Management sysfs Interface. drivers/base/power/sysfs.c*. URL: https://github.com/torvalds/linux/blob/master/drivers/base/power/sysfs.c (visited on 09/19/2026).

[27] Marcelo Tosatti. *Linux Kernel Source: haltpoll CPUIdle Governor. drivers/cpuidle/governors/haltpoll.c*. The Linux Kernel Development Community. URL: https://github.com/torvalds/linux/blob/master/drivers/cpuidle/governors/haltpoll.c (visited on 09/19/2026).

[28] The Linux Kernel Development Community. *Guest Halt Polling*. Version 7.3.0-rc3. URL: https://docs.kernel.org/virt/guest-halt-polling.html (visited on 09/19/2026).

[29] Roger Daniel Francisco Ferreira and Rômulo Silva de Oliveira. "Cloud IEC 61850: DDS Performance in Virtualized Environment with OpenDDS." In: *2017 IEEE International Conference on Computer and Information Technology (CIT)*. 2017, pp. 231–236. DOI: 10.1109/CIT.2017.17.

[30] Dean Samara-Rubio, Graham McKenzie, Prithpal Khajuria, Roopesh Ariya, Prasanth Gopalakrishnan, and Viju Ravindran. *Virtual Protection Relay: A paradigm shift in power system protection*. White Paper. Intel Corporation, 2024. URL: https://builders.intel.com/docs/networkbuilders/virtual-protection-relay-white-paper-1709299398.pdf (visited on 09/13/2026).

[31] Robert Wójtowicz, Ryszard Kowalik, Désiré D. Rasolomampionona, and Karol Kurek. "Virtualization of Protection Systems - Tests Performed on a Large Environment Based on Data Center Solutions." In: *IEEE Transactions on Power Delivery* 37.4 (2022), pp. 3401–3411. DOI: 10.1109/TPWRD.2021.3128993.

[32] Ricardo Carvalho, Mário Antunes, João Paulo Barraca, Diogo Gomes, and Rui L. Aguiar. "Design and Evaluation of a Low-Latency CPC Environment for Virtual IEDs." In: *2022 IEEE 11th International Conference on Cloud Networking (CloudNet)*. 2022, pp. 272–276. DOI: 10.1109/CloudNet55617.2022.9978874.

[33] Sandro Schönborn, Robert Birke, David Kozhaya, and Thanikesavan Sivanthi. "Real-time performance of virtualised protection and control software." In: *27th International Conference on Electricity Distribution (CIRED 2023)*. Vol. 2023. 2023, pp. 1817–1821. DOI: 10.1049/icp.2023.1028.

[34] Nadine Kabbara, Agrippina Mwangi, Alexandru Ştefanov, and Madeleine Gibescu. "A Real-Time Implementation and Testing of Virtualized Controllers for Software-Defined IEC 61850 Digital Substations." In: *IEEE Open Journal of Industry Applications* 5 (2024), pp. 300–310. DOI: 10.1109/OJIA.2024.3426321.

[35] Daniel Bristot de Oliveira, Daniel Casini, Rômulo Silva de Oliveira, and Tommaso Cucinotta. "Demystifying the Real-Time Linux Scheduling Latency." In: *32nd Euromicro Conference on Real-Time Systems (ECRTS 2020)*. Ed. by Marcus Völp. Vol. 165. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020, 9:1–9:23. ISBN: 978-3-95977-152-8. DOI: 10.4230/LIPIcs.ECRTS.2020.9. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ECRTS.2020.9.

[36] Sebastian Gallenmüller, Florian Wiedner, Johannes Naab, and Georg Carle. "How Low Can You Go? A Limbo Dance for Low-Latency Network Functions." In: *J. Netw. Syst. Manage.* 31.1 (Dec. 2022). ISSN: 1064-7570. DOI: 10.1007/s10922-022-09710-3. URL: https://doi.org/10.1007/s10922-022-09710-3.

[37] Edoardo Barbieri. *Tuning a real-time kernel*. Canonical Ltd. June 19, 2023. URL: https://ubuntu.com/blog/real-time-kernel-tuning (visited on 09/13/2026).

[38] Jaroslav Klech, Sujata Kurup, Marie Doleželová, Jana Heves, Maxim Svistunov, Radek Bíba, David Ryan, Cheryn Tan, Lana Brindley, and Alison Young. *Advanced tuning procedures to optimize latency in RHEL for Real Time*. Red Hat, Inc. Sept. 29, 2020. URL: https://docs.redhat.com/en/documentation/red_hat_enterprise_linux_for_real_time/7/html/tuning_guide (visited on 09/13/2026).

[39] The Linux Foundation. *Realtime Documentation*. Oct. 2, 2023. URL: https://wiki.linuxfoundation.org/realtime/documentation/start (visited on 09/13/2026).

[40] Theodore Ts'o, Darren Hart, and John Kacur. *HOWTO: Build an RT-application*. Archived wiki, no longer updated. Apr. 25, 2019. URL: https://archive.kernel.org/oldwiki/rt.wiki.kernel.org/index.php/HOWTO:_Build_an_RT-application.html (visited on 09/13/2026).

[41] Luca Abeni and Dario Faggioli. "Using Xen and KVM as real-time hypervisors." In: *Journal of Systems Architecture* 106 (2020), p. 101709. ISSN: 1383-7621. DOI: 10.1016/j.sysarc.2020.101709.

[42] Jan Kiszka. "Towards Linux as a Real-Time Hypervisor." In: (Jan. 2009).

[43] Baojing Zuo, Kai Chen, Alei Liang, Haibing Guan, Jun Zhang, Ruhui Ma, and Hongbo Yang. "Performance Tuning Towards a KVM-Based Low Latency Virtualization System." In: *2010 2nd International Conference on Information Engineering and Computer Science*. 2010, pp. 1–4. DOI: 10.1109/ICIECS.2010.5678357.

[44] Jan Ruh and Wilfried Steiner. "The need for deterministic virtualization in the industrial internet of things." In: *Proceedings of the Workshop on Fog Computing and the IoT*. IoT-Fog '19. Montreal, Quebec, Canada: Association for Computing Machinery, 2019, pp. 26–30. ISBN: 9781450366984. DOI: 10.1145/3313150.3313222. URL: https://doi.org/10.1145/3313150.3313222.

[45] Intel Corporation. *Real-Time VMs using KVM*. Aug. 21, 2026. URL: https://eci.intel.com/docs/3.3/development/tutorials/enable-rt-vms.html (visited on 09/13/2026).

[46] Libvirt Virtualization Knowledgebase. *KVM Real Time Guest Config*. URL: https://libvirt.org/kbase/kvm-realtime.html (visited on 09/13/2026).